Sisteme de operare

6 septembrie 2009

Timp de lucru: 70 de minute

NOTĂ: toate răspunsurile trebuie justificate

1. Un sistem dispune de următoarele caracteristici

- magistrala de date pe 64 de biți
- overhead-ul impus de un page fault este de 1ms
- nu dispune de memorie cache
- durata unei operații cu memoria este de 100ns

Pe sistem rulează un sistem de operare în cadrul căruia biblioteca standard C folosește un buffer intern de 64K la nivelul fiecărui handle de fișier.

Care din operațiile marcate aldin (bold) de mai jos durează mai mult?

`char buf[32*1024];`
`f = fopen("a.dat", "wb");`
`/* fill buffer */`
`…`
`fwrite(f, buf, 32*1024);`
`fflush(f);`
`fwrite(f, buf, 32*1024);`
	`char buf[32*1024];`
	`int fd;`
	`char *a;`
	`fd = open("a.dat", O_RDWR | O_CREAT | O_TRUNC, 0644);`
	`/* fill buffer */`
	`…`
	`write(fd, buf, 32*1024);`
	`a = mmap(NULL, 32*1024, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);`
	`memcpy(a, buf, 32*1024);`

Operația fwrite înseamnă copierea datelor din buf în bufferul intern al bibliotecii standard C. Întrucât bufferul intern oferă spațiu pentru tot buffer-ul nu va exista nici un apel de sistem și nici pagefault-uri.

Operația memcpy va presupune copierea datelor într-o zonă alocată cu mmap. Durata de copiere este identică celei de mai sus, dar au loc page fault-uri la fiecare pagină. Aceasta se întâmplă pentru că mmap alocă memorie virtuală pură (demand paging). Primul acces la o pagină va conduce la page fault.

În concluzie, operația memcpy durează mai mult decât fwrite.

2. În cadrul problemei celor 5 filozofi se folosește următoarea implementare (în pseudo-C) a funcției eat():

```
mutex_t global_mutex;

void eat(int fork1, int fork2)
{
	lock(global_mutex);
	take_fork(fork1);
	take_fork(fork2);
	do_eat();
	put_fork(fork1);
	put_fork(fork2);
	unlock(global_mutex);
}
```

Care este neajunsul acestei implementări?

Folosirea mutexului global înseamnă că un singur filozofi din cei 5 poate mânca la un moment dat. Fiind 5 furculițe, soluția eficientă permite ca doi filozofi să mănânce simultan.

3. Pe un sistem rulează 50 de procese. La un moment dat este pornită o mașina virtuală VMware Server pe care rulează 50 de procese. Câte procese vor rula pe sistemul gazdă? Dar în cazul pornirii unei mașini virtuale OpenVZ pe care rulează 50 de procese?

O mașină virtuală VMware Server este reprezentată pe sistemul gazdă de un singur proces. Vor exista, în total, 51 de procese.

O mașină virtuală (container) OpenVZ are, pe sistemul fizic, un corespondent pentru fiecare proces. Vor exista, în total, 100 de procese.

4. Pe un sistem de fișiere MINIX se execută operațiile

```
lseek(fd, SEEK_SET, 32*1024);
write(fd, buffer, 1024);
```

Descrieți operațiile asociate asupra structurilor sistemului de fișiere (inode, bitfield, data block, dentry etc.)

lseek nu modifică structurile interne ale sistemului de fișiere. Este poziționat cursorul de fișier (corespondent unei structuri din memorie) la offsetul specificat.

*Se calculează blocul aferent adresei 32*1024 prin parcurgerea pointerilor de bloc din inode-ul MINIX. Întrucât se depășește spațiul referibil prin referință directă se va citi blocul aferent pentru referință indirectă.*

Se parcurge bitfield-ul și să găsește primul bloc liber. Adresa acelui bloc este scrisă pe poziția aferentă din blocul de referință indirectă. Se citește blocul de pe disc în memorie și se scrie informația furnizată din user-space. Se actualizează câmpul size din inode. Ulterior se va face flush la bloc din memorie pe disc.

5. Un sistem folosește un planificator round-robin non-preemptiv. În cadrul sistemului rulează 5 procese care execută următoarea secvență:
`[ 10ms rulare | 10ms așteptare | 10ms rulare ]`
Cât durează planificarea celor 5 procese?

Planificare round-robin înseamnă că fiecare proces este planificat pe rând. Planificarea se face astfel:
0-10ms: procesul 1
10ms-20ms: procesul 2 (procesul 1 așteaptă)
20ms-30ms: procesul 3 (procesul 2 așteaptă, procesul 1 este gata de execuție)
30ms-40ms: procesul 4 (procesul 3 așteaptă, procesul 1 și procesul 2 sunt gata de execuție)
40ms-50ms: procesul 5 (procesul 4 aștepată, procesele 1, 2 și 3 sunt gata de execuție)
....
Durata de planificare este de 100ms

6. Cum se modifică spațiul de adresă al unui proces la schimbarea de context între două thread-uri?

Nu se modifică. Thread-urile partajează spațiul de adresă al procesului.

7. Are sens folosirea operațiilor asincrone în locul celor sincrone în situația de mai jos (pseudo-cod)?

```
AIO_TYPE aioArray[32];
InitializeAsyncIoArray(aioArray);
for (i = 0; i < 32; i++)
       StartAsyncIo(aioArray[i]);
WaitForAllObjects(aioArray);
```

Cele 32 de operații asincrone sunt pornite în același timp. Durata de așteptare este durata de rulare/planificare a celei mai lente operații. În cazul unei operații sincrone (blocante, secvențiale). Durata de așteptare ar fi fost suma duratelor de rulare/planificare a tuturor operațiilor.

8. Descrieți o situație în care operația:
`memcpy(a, “12345678901234567890”, 20);`
durează mai mult, respectiv mai puțin, decât operația
`getpid();`
Operația getpid rezultă într-un apel de sistem. Overhead-ul unui page fault este de 5ms, iar a unui apel de sistem de 7ms.

Dacă zona indicată de a (20 de octeți) este poziționată într-o singură pagină overhead-ul este de 5ms în cazul unui page fault (pagina nu este prezentă în memoria fizică) sau neglijabil în cazul în care pagina este prezentă în memorie. Durează, astfel, mai puțin decât getpid().

Dacă zona indicată de a (20 de octeți) este poziționată pe două pagini (spre exemplu 8 octeți în prima, 12 în a doua), overhead-ul (în cazul absenței paginilor din memoria fizică) este de 10ms.

9. Pe o arhitectură x86, care registre generale (eax, ebx, ecx, edx, esi, edi, ebp, esp) sunt schimbate în cazul unei schimbări de context între două thread-uri? Dar în cazul unei schimbări de context între două procese?

În ambele cazuri se schimbă tot setul de registre, întrucât definesc un nou context.

10. Se presupune că se implementează la nivel hardware o tehnică ce împiedică accesarea zonelor de memorie nealocate la nivel de octet. Cum poate fi folosită această tehnică pentru prevenirea atacurilor de tip buffer overflow la nivelul stivei?

Nu previne. Atacul de tip buffer overflow înseamnă suprascrierea stivei sau a altei regiuni deja alocate și a adresei de retur (și aceasta alocată pe stivă). Protecția la accesarea unor zone nealocate nu împiedică acest tip de atac.

11. Într-un sistem de operare, **4** (patru) procese execută operațiile

```
fd1 = open(“a.txt”, O_RDONLY);
fd2 = open(“b.txt”, O_RDWR);
a1 = mmap(NULL, 4*1024, PROT_READ, MAP_SHARED, fd1, 0);
a2 = mmap(NULL, 4*1024, PROT_READ | PROT_WRITE, MAP_SHARED, fd2, 0);
printf(“%c”, *a1);
*a2 = ‘a’;
```

Câte pagini de memorie fizică, respectiv virtuală vor fi ocupate în urma operațiilor de mai sus?

Fiecare proces accesează cele două pagini alocate. În urma accesului se realizează un page fault și se mapează paginile virtuale peste paginile fizice.

*Se vor aloca 4 procese * 2 pagini virtuale = 8 pagini virtuale*

*Pentru fiecare pointer (a1 sau a2) se mapează **aceeași** pagină fizică, Maparea este partajată (MAP_SHARED) și toate procesele vor vedea același conținut. În cazul particular al mapării a2 (PROT_WRITE) scrierile din cadrul unui proces vor fi vizibile în celelalte procese.*

Se vor aloca 1 pagini fizică (prin a2) + 1 pagină fizică (prin a1) = 2 pagini fizice

Sisteme de operare

10 septembrie 2009

Timp de lucru: 70 de minute

NOTĂ: toate răspunsurile trebuie justificate

1. Care din următoarele acțiuni consumă cel mai mult timp în cazul unei schimbări de context între două thread-uri ale aceluiași proces:

- schimbarea registrelor
- flush TLB
- schimbarea tabelei de pagini
- schimbarea tabelei de descriptori de fișier

În cazul schimbării de context între două thread-uri ale aceluiași proces nu se face flush la TLB, nu se schimbă tabela de pagini, nu se schimbă tabela de descriptori de fișier. În consecință, deși foarte rapidă, acțiunea care consumă cel mai mult timp este schimbarea registrelor.

2. Un sistem dispune de magistrală de date și registre pe 32 de biți (sizeof(unsigned long) = 32). Sistemul folosește paginare simplă (non-ierarhică) și nu are memorie cache, nici TLB. Știind că un acces la memorie durează 50ns iar o pagină este de 4KB, cat va dura secvența de mai jos? Se presupune că vectorul buffer este alocat în RAM (memoria fizică) și că valoarea contorului *i* se păstrează într-un registru (nu folosește memoria).

```
unsigned long buffer[32*1024];

for (i = 0; i < 32 * 1024; i++)
        buffer[i] = i;
```

*În absența TLB fiecare acces la memoria fizică necesită accesarea tabelei de pagini (aflată tot în memoria fizică). Astfel, pentru fiecare dintre cele 32*1024 de accese la buffer vor exista încă 32*1024 accese la tabele de pagini. Rezultă 64*1024 accese la memorie cu o durată totală de 64*1024*50ns.*

3. Care dintre variantele chroot, respectiv OpenVZ oferă un grad mai mare de securitate?

chroot oferă "încapsulare" (securizare) doar la nivelul sistemului de fișiere. OpenVZ oferă securizare la nivelul proceselor, memoriei, procesorului, rețelei, utilizatorilor etc. OpenVZ oferă, așadar un grad mai mare de securitate.

4. În cadrul unui proces cu mai multe thread-uri, un thread execută următoarea secvență de cod (pseudo C):

```
int esp;
int stack_val;

/* se obtine valorea registrului esp (registru de stiva) al thread-ului planificat anterior */
esp = get_former_esp();
stack_val = *(esp + 4);
```

Care va fi rezultatul execuției secvenței de mai sus pe un sistem în care stiva crește în jos?

Întrucât stiva crește în jos, valoarea esp+4 va puncta către o zonă alocată din stiva fostului thread. Execuția de mai jos va rezulta în obținerea acelei valori (nu se va obtine segmentation fault decât dacă thread-ul anterior și-a încheiat execuția).

5. Pe un sistem de fișiere MINIX se execută operația:

```
fd = open("a.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
```

Precizați ce se întâmplă la nivelul sistemului de fișiere (inode, inode bitmap, zone bitmap, data block, dentry etc.) în cazul în care fișierul există sau nu există pe disc.

Dacă fișierul există se găsește dentry-ul acestuia și se obține inode-ul aferent și se citește inode-ul în memorie.
Dacă fișierul nu există se creează un inode nou. Pentru aceastase parcurge bitmapul de inode-uri și se alocă un inode. Se completează cu 1 poziția liberă găsită. Se creează un dentry cu numele "a.txt".
Dacă fișierul exista este trunchiat. Se parcurg pointer-ii de blocuri ai inode-ului și se marchează cu NULL (sau ceva echivalent). Se parcurge bitmapul de blocuri și marchează cu 0 pozițiile aferente acelor blocuri. Dacă fișierul avea mai mult de 7 blocuri se citește și completează cu NULL blocul pentru dereferențieri simple.

6. Precizați o soluție de sincronizare pentru problema formării moleculei de oxid de fier (Fe_2O_3) (ca alternativă la problema formării apei).

```
void fe_fun()
{
        m.enter();
        fe_count++;
        if (o_count >= 3) {
                if (fe_count < 2)
                        m.fe_cond.wait();
                else {
                        m.o_cond.signal();
                        m.o_cond.signal();
                        m.o_cond.signal();
                        m.fe_cond.signal();
                        fe_count -= 2;
                }
        }
        else
                m.fe_cond.wait();
        m.leave();
}
```

```
void o_fun()
{
        m.enter();
        o_count++;
        if (o_count >= 3 && fe_count >= 2) {
                m.o_cond.signal();
                m.o_cond.signal();
                m.fe_cond.signal();
                m.fe_cond.signal();
                if (o_count > 3) {
                        m.o_cond.signal();
                        m.o_cond.wait();
                }
        }
        else
                m.o_cond.wait();
        m.leave();
}
```

7. Biblioteca standard C oferă programatorului funcția calloc (alocare cu zeroing). De ce este nevoie și de oferirea funcției malloc?

Funcția malloc este mai rapidă – nu face zeroing și permite demand paging.

8. Pe un sistem care dispune de 3 pagini fizice (frames) și folosește un algoritm de înlocuire a paginii de tip NRU se execută următoarea secvență:

```
1r, 2w, 4r, 1w, 3r, 2w, 1w, 4w, 3r, 3w, 1r, 2r, 5r, 2w, 6r, 3w, 1w, 2r
```

Câte page fault-uri au loc? 1r înseamnă operație de citire în cadrul paginii virtuale 1; 2w înseamnă operație de scriere în cadrul paginii virtuale 2.

Conținutul celor 3 pagini fizice, împreună cu evenimentul aferent este prezentat, evolutiv, în tabelul de mai jos; la fiecare două pagefault-uri se resetează bitul referenced):

frame 1	1r (R)	1r (R)	1r	1w (W)	3r (R)	3r (R)	3r	3r (R)	3w (RW)	3w (W)	2r (R)	2r (R)	2w (W)	6r (R)	6r (R)	6r	2r (R)
frame 2		2w (W)	2w (W)	2w (W)	2w (W)	2w (W)	4w (W)	4w (W)	4w (W)	4w (W)	4w (W)	5r (R)	5r (R)	5r	3w (W)	3w (W)	3w (W)
frame 3			4r (R)	4r (R)	4r	1w (W)	1w (W)	1w (W)	1w (W)	1w (RW)	1w (RW)	1w (W)	1w (W)	1w (W)	1w (W)	1w (W)	1w (W)
	PF	PF	PF	PF	PF	PF	PF		PF		PF	PF	PF	PF	PF		PF

9. Fie secvența de program de mai jos:

```
int main(void)
{
        char *a;
        int i;

        for (i = 0; i < 10; i++)
                a = malloc(1);

        return 0;
}
```

La rulare se observă (prin folosirea unui profiler) că primul apel malloc durează semnificativ mai mult decât celelalte 9. Care este motivul? Toate apelurile reușesc (întorc o adresă validă) și nu există nici o modificare adusă apelului malloc.

Primul apel malloc generează un page fault, urmarea fiind alocarea unei pagini fizice întregi (chiar dacă se solicită alocarea unui singur octet). Următoarele apeluri vor aloca octeți din cadrul aceleiași pagini – nu mai este generat un page fault și nu se aloca alte pagini.

10. Are sens folosirea operațiilor de tipul Overalapped I/O pe un sistem care dispune de un singur hard-disk?

Da. Operațiile overlapped I/O permit o planificare mai eficientă a operațiilor de I/O la nivelul nucleului și permit aplicației să ruleze

11. Descrieți o situație în care două procese partajează o pagină virtuală (din spațiul virtual de adrese).

Fiecare proces are propriul spațiu de adrese. Nu există noțiunea de partajare a unei pagini virtuale.

Sisteme de operare

25 iunie 2009

Timp de lucru: 70 de minute

NOTĂ: toate răspunsurile trebuie **justificate**

1. "Sistemele de operare moderne nu au probleme de fragmentare externă a memoriei fizice alocate din user-space." Indicați și motivați valoarea de adevăr a propoziției anterioare.

Sistemele de operare moderne folosesc suportul de paginare pus la dispoziție de sistemul de calcul. Folosirea paginării înseamnă că se pot aloca ușor pagini de memorie fizică acolo unde sunt libere. Mecanismul de memorie virtuală asigură faptul că o alocare rămâne virtual contiguă. În felul acesta dispar problemele de fragmentare externă – adică de găsire a unui spațiu continuu pentru alocare (rămân însă problemele de fragmentare internă).
Excepție fac alocările din kernel-space care pot solicita alocare de memorie fizic contiguă sau alocările impuse de hardware (de exemplu DMA).

2. Un sistem de operare dispune de un planificator de procese care folosește o cuantă de 100ms. Durata unei schimbări de context este 1ms. Este posibil ca planificatorul să petreacă jumătate din timp în schimbări de context? Motivați.

Da, este posibil în situațiile în care procesele planificate execută acțiuni scurte și apoi se blochează determinând schimbări de context. Acest lucru se poate întâmpla în cazul sincronizării între procese (un proces P1 execută o acțiune, apoi trezește procesul P2 și apoi se blochează, procesul P2 execută o acțiune, apoi trezește procesul P1, etc.), sau în cazul comunicației cu dispozitive de I/O rapide (procesul P1 planifică o operație I/O și se blochează, operația se încheie rapid și trezește procesul etc.).
O altă situație este schimbarea rapidă a priorității proceselor care determină schimbarea de context pentru rularea procesului cu prioritatea cea mai bună.

3. Dați exemplu de funcție care este reentrantă, dar nu este thread-safe. Dați exemplu de funcție care este thread-safe, dar nu este reentrantă.

Toate funcțiile reentrante sunt thread-safe. Exemplu de funcție care este thread-safe dar nu reentrantă este malloc. Un exemplu generic este o funcție care folosește un mutex pentru sincronizarea accesului la variabile partajate între thread-uri: funcția este thread-safe, dar nu este reentrantă (nu pot fi executate simultan două instanțe ale acestei funcții). Proprietatea de reentranță sau thread-safety se referă la implementarea și interfața funcției, nu la contextul în care este folosită (o funcție reentrantă poate fi folosită într-un context unsafe din punct de vedere al sincronizării, dar nu înseamnă că este non-thread safe).

4. Într-un sistem de fișiere FAT un fișier ocupă 5 blocuri: 10, 59, 169, 598, 1078. Știind că:

- un bloc ocupă 1024 de octeți
- o intrare în tabela FAT ocupă 32 de biți
- tabela FAT NU se găsește în memorie
- copierea unui bloc în memorie durează 1ms

cât timp va dura copierea completă a fișierului în memorie?

Un bloc ocupă 1024 de octeți, o intrare în tabela FAT 4 octeți, deci sunt 256 intrări FAT într-un bloc. În tabela FAT intrările 10, 59, 169 se găsesc în primul bloc, intrarea 598 în al treilea bloc și 1078 în al cincilea bloc. Vor trebui, astfel, citite 3 blocuri asociate tabelei FAT. Fișierul ocupă 5 blocuri, deci vor fi citite, în total, 8 blocuri. Timpul total de copiere este 8ms.

5. Două procese P1, respectiv P2 ale aceluiași utilizator sunt planificate după cum urmează:

`fd = open("/tmp/a.txt", O_CREAT | O_RDWR, 0644);`
`write(fd, "P1", 2);`
`--- schedule ---`	

	`--- schedule ---`
	`fd = open("/tmp/a.txt", O_CREAT | O_RDWR, 0644);`
	`write(fd, "P2", 2);`

Ce va conține, în final, fișierul /tmp/a.txt? Ce va conține fișierul în cazul în care se folosesc thread-uri în loc de procese?

Două apeluri open întorc descriptori către structuri distincte de fișier deschis. Acest lucru înseamnă că fiecare descriptor va folosi un cursor de fișier propriu. Al doilea apel open va poziționa cursorul de fișier la începutul fișierului și va suprascrie mesajul primului proces. În final în fișier se va scrie P2. În cazul folosirii thread-urilor situația este neschimbată pentru că se vor folosi, din nou, cursoare de fișier diferite.

6. Are sens folosirea unui sistem de protejare a stivei (stack smashing protection, canary value) pe un sistem care dispune de și folosește bitul NX?

Da, are sens. În general, sistemele de tip stack overflow suprascriu adresa de retur a unei funcții cu o adresă de pe stivă. Bitul NX previne execuția de cod pe stivă. Dar adresa de retur poate fi suprascrisă cu adresa unei funcții din zona de text (return_to_libc attack) sau o adresă din altă zonă care poate fi executată (biblioteci, heap).

7. Pe un sistem quad-core și 4GB RAM rulează un proces care planifică 3 thread-uri executând următoarele funcții:

```
thread1_func(initial_data)
{
    for (i = 0; i < 100; i++) {
        work_on_data();
        wake_thread2();
        wait_for_data_from_thread3();
    }
}
```

```
thread2_func()
{
    for (i = 0; i < 100; i++) {
        wait_for_data_from_thread1();
        work_on_data();
        wake_thread3();
    }
}
```

```
thread3_func()
{
    for (i = 0; i < 100; i++) {
        wait_for_data_from_thread2();
        work_on_data();
        wake_thread1();
    }
}
```

Care este dezavantajul acestei abordări? Propuneți o alternativă.

Codul de mai sus este un cod serial. Folosirea celor trei thread-uri este ineficientă pentru că se execută mai ușor în cadrul unui singur thread (apar overhead-uri de creare, sincronizare și schimbare de context între thread-uri). Soluția este folosirea unui singur thread sau reglarea algoritmului folosit pentru a putea fi cu adevărat paralelizat.

8. În spațiul de adrese al unui proces, zona de cod (*text*) este mapată read-only. Acest lucru este avantajos din punct de vedere al securității, întrucât împiedică suprascrierea codului executat. Ce alt avantaj important oferă?

Fiind read-only zona poate fi partajată între mai multe procese limitând spațiul ocupat în RAM.

9. Folosind o soluție de virtualizare, se dorește simularea unei rețele formată din: două sisteme Windows, un gateway/firewall OpenBSD și un server Linux. Opțiunile sunt VMware Workstation, OpenVZ și Xen. Care variantă de virtualizare permite rularea unui număr cât mai mare de instanțe de astfel de rețele pe un sistem dat?

OpenVZ nu poate fi folosit pentru că este OS-level virtualization: toate mașinile virtuale folosesc același nucleu deci pot fi folosite mașini virtuale care rulează același sistem de operare ca sistemul gazdă. Xen este o soluție rapidă dar rularea unui sistem nemodificat (gen Windows) este posibilă doar în situația în care hardware-ul peste care rulează oferă suport (Intel VT sau AMD-V). Vmware Workstation este o soluție mai lentă, în general, decât Xen dar permite rularea oricărui tip de sistem de operare guest.

10. Un sistem de operare dat poate fi configurat să folosească un split user/kernel 2GB/2GB al spațiului de adresă al unui proces sau un split 3GB/1GB. Sistemul fizic dispune de 1GB RAM. Un proces rulează secvența:

```
for (i = 0; i < N; i++)
    malloc(1024*1024);
```

Pentru ce valori (aproximative) ale lui N *malloc* va întoarce NULL în cele două cazuri de split?

În exemplul de cod de mai sus, malloc alocă memorie pur virtuală (fără suport de memorie fizică). Alocarea de memorie fizică se va realiza la cerere (demand paging). malloc va întoarce NULL în momentul în care procesul rămâne fără memorie virtuală în user-space. N va avea, așadar, valori aproximative de 2048 și 3072. Dimensiunea memoriei RAM a sistemului este nerelevantă în această situație.

11. Un proces dispune de o tabelă de descriptori de fișiere cu 1024 de intrări. În codul său, procesul deschide un număr mare de fișiere folosind *open*. Totuși, al 1010-lea apel *open* se întoarce cu eroare, iar *errno* are valoarea *EMFILE* (maximum number of files open). Care este o posibilă explicație?

Procesul realizează 1009 apeluri open cu succes, rezultând în 1009 file descriptori deschiși. stderr, stdout, stdin sunt 3 descriptori inițiali, rezultând 1012 descriptori. Restul au fost creați prin alte metode. File descriptorii pot fi creați și altfel: creat (creare fișiere), dup, socket, pipe. O altă situație este aceea în care procesul moștenește un număr de file descriptori de la procesul părinte.

Sisteme de operare

26 iunie 2009

Timp de lucru: 70 de minute

NOTĂ: toate răspunsurile trebuie **justificate**

1. Știind că operațiile de lucru cu pipe-uri sunt atomice, implementați în pseudocod un mutex cu ajutorul pipe-urilor.

lock: read(pipefd[0], &a, 1);
unlock: write(pipefd[1], &a, 1);

a este un char; pipefd este un pipe

2. Durata unei schimbări de context este de 1ms iar overhead-ul unui apel de sistem de 100µs. Totuși, la un moment dat, apelul down(&sem); durează doar 1µs. Apelul se realizează cu succes. Care este explicația?

Apelul down este implementat în user-space (de exemplu o implementare de tip futex). Dacă valoarea semaforului este strict pozitivă, atunci apelul down va decrementa valoarea semaforului și va continua execuția (fără apel de sistem și fără schimbare de context). În cazul în care valoarea este egală cu 0 va avea loc un context switch. În cazul unei implementări de thread-uri kernel-level, acest lucru va presupune și un apel de sistem (planificatorul este implementat în kernel-space).

3. De ce este mai avantajos ca, pe un sistem uniprocesor, după un apel *fork* să fie planificat primul procesul fiu?

Pentru a evita posibilele copieri inutile datorate copy-on-write. De multe ori, după fork procesul copil execută exec, rezultând în schimbarea completă a spațiului de adresă. Dacă procesul părinte ar fi planificat primul, atunci apelurile de scriere ale acestuia vor rezulta în duplicarea paginilor (overhead temporal și consum memorie) datorită copy-on-write. Dacă procesul copil face exec, atunci acele duplicate au însemnat un consum inutil de resurse.

4. Există vreo diferență între implementarea simbolului *errno* în contextul unui proces single-threaded față de un proces multi-threaded? Argumentați.

Da, există diferență. Fiecare thread trebuie să aibă acces la o variabilă errno proprie, astfel că errno va fi de obicei implementat ca variabilă per-thread. Acest lucru se poate realiza cu ajutorul TLS/TSD. O variabilă comună errno pentru toate thread-uri ar conduce la incosistența informațiilor referitoare la erorile apărute.

5. Un sistem uniprocesor (single-core) dispune de 64KB L1 cache, 512KB L2 cache și un TLB cu 256 intrări. Pe un sistem de operare cu suport în kernel pentru thread-uri, ce durează mai mult: schimbarea de context între două thread-uri sau între două procese?

Schimbarea de context între două procese va dura tot timpul mai mult decât schimbarea de context între două procese. În momentul schimbării de context între două procese se schimbă întreg spațiul de adresă și resursele asociate. Se schimbă astfel tabela de pagini, se face flush la TLB etc. În cazul thread-urilor o schimbare de context presupune doar schimbarea registrelor și a informațiilor specifice unui thread.

6. Comanda pmap afișează informații despre spațiul de adrese al unui proces. În urma rulării de mai jos a comenzii *pmap* se observă următoarele informații despre biblioteca standard C:

```
# pmap 1
base address          size    rights     name
[...]
00007f8c480e6000    1320K    r-x--      libc-2.7.so
00007f8c4842f000      12K    r----      libc-2.7.so
00007f8c48432000       8K    rw---      libc-2.7.so
```

Presupunând că în sistem rulează 50 de procese care folosesc biblioteca standard C, care este spațiul total de memorie RAM ocupat de bibliotecă?

*Ultima zona este o zonă read-write și nu poate fi partajată între două procese. Celelalte două zone sunt read-only și vor fi partajate. Biblioteca va ocupa, așadar, 50*8K + 12K + 1320K.*

7. Descrieți și explicați în pseudo-asamblare cum acționează suportul de SSP (Stack Smashing Protection) pe un sistem în care stiva crește în sus (de la adrese mici la adrese mari).

Pe un sistem pe care stiva crește în sus nu se poate realiza stack overflow din stack-frame-ul curent ci din stack frame-ul apelantului. Astfel, dacă o funcție apelează strcpy și un argument este un buffer al funcției, acest buffer poate fi folosit pentru a suprascrie (prin overflow) adresa de retur a funcției strcpy. Valoarea de tip canary trebuie stocată la o adresă mai mică decât adresa de retur a funcției strcpy (practic, la fel ca la o stivă care crește în jos). Întrucât apelantul este cel care construiește stack frame-ul apelatului, acesta va trebui să marcheze valoarea de tip canary. În schimb apelatul (aici strcpy), înainte de întoarcere va verifica suprascrierea adresei de tip canary.
Stack frame-ul este cel de mai jos:

```
[ strcpy local  ]
[      ...      ]
[ ret address   ]
[ old_ebp       ]  callee (strcpy) stack frame
[ canary value  ]----
[ strcpy param1 ]
[ strcpy param2 ]
[      ...      ]
[ local buffer  ]  caller stack frame
[      ...      ]
[ local buffer  ]
```

8. Pe un sistem de fișiere dat un dentry are următoarea structură:
- 1 octet - lungimea numelui
- 251 octeți – numele
- 4 octeți - numărul inode-ului

O instanță a unui astfel de sistem de fișiere deține un director rădăcină, 5 subdirectoare, iar fiecare subdirector conține 5 fișiere. Câte dentry-uri deține sistemul de fișiere?

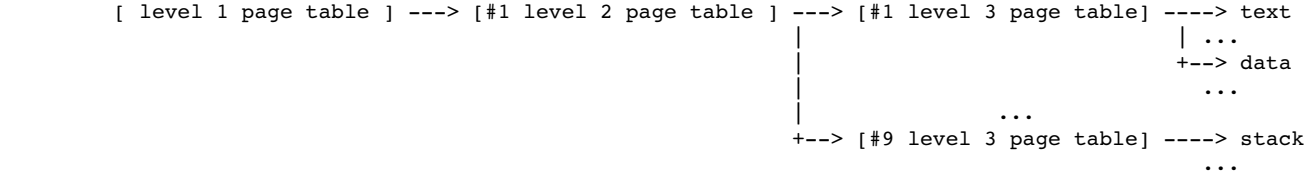
*Fiecare intrare în sistemul de fișiere (director sau fișier) conține cel puțin un dentry. Ignorând intrările speciale . și .. rezultă (1 +) 5 + 5*5 = 30 (31) intrări. Directorul rădăcină poate să nu aibă dentry. Considerând intrările speciale, rezultă un plus de 1 + 2*5 intrări = 11 intrări (directorul rădăcină nu are referință ..).*

9. Un sistem pe 64 de biți folosește pagini de 8KB și 43 de biți pentru adresare într-o schemă de adresare ierarhică pe trei niveluri cu împărțirea (10 + 10 + 10 + 13). Un proces care rulează în cadrul acestui sistem are, la un moment dat, următoarea componență a spațiului de adrese (se începe de la adresa 0):

```
[ text ]             - 16 pagini
[ data ]             - 8 pagini
[ spațiu nealocat ]  - 8168 pagini
[ stivă]             - 8 pagini
```

Știind că o intrare în tabela de pagini ocupă 64 de biți, câte pagini ocupă tabelele de pagini pentru procesul dat?

O intrare în tabela de pagini ocupă 64 de biți = 8 octeți. Există, astfel, 1024 de intrări într-o pagină. Zona text și data ocupă 24 de pagini deci vor există 24 de intrări valide în prima pagină de tabelă de pe nivelul 3. Următoarele 8168 pagini vor completa intrările din prima tabelă de pe nivelul 3 și vor mai folosi 7 pagini de tabele. Întrucât este spațiu nealocat, cele 7 pagini de tabele nu vor fi nici ele alocate. A 9-a pagină de tabela va folosi primele 8 intrări pentru a referi paginile de pe stivă.
Prima pagină de tabelă de pe nivelul 2 va avea valide doar prima și a 9-a intrare (care vor referi prima și a 9-a pagină de tabelă). Pagina de tabelă de pe nivelul 1 va avea validă doar prima intrare către pagina de tabelă de pe nivelul 2. Vor fi, astfel, folosite, doar 4 pagini.
Schematic, reprezentarea este următoarea:



10. Dați exemplu de situație în care, pentru comunicația cu dispozitivele de I/E, se preferă folosirea polling în loc de întreruperi.

Pollingul se preferă în situațiile în care întreruperile previn funcționarea eficientă a sistemului. Acest lucru se întâmplă în cazul în care întreruperile sunt transmise foarte des și procesorul petrece mult timp în rutinele de tratare a întreruperilor. Soluția este dezactivarea temporară a întreruperilor și folosirea polling. Acest lucru se întâmplă la dispozitivele de rețea foarte rapide, spre exemplu plăcile de rețea.

11. Un program execută secvența de cod din coloana din stânga tabelului de mai jos. În coloana din dreapta este prezentat rezultatul rulării programului:

```
/* init array to 2, 0, 0, 0 ... */
static int data1[1024*1024] = {2, };

static void print_time(char *msg)
// ...

static void init_array(int *a, size_t len)
{
    size_t i;

    for (i = 0; i < len; i += 1024)
        a[i] = 2009;
}

int main(void)
{
    int *data2 = malloc(1024*1024 * sizeof(int));

    print_time("before init data1");
    init_array(data1, 1024*1024);
    print_time("after  init data1");

    print_time("before init data2");
    init_array(data2, 1024*1024);
    print_time("after  init data2");

    return 0;
}
```

```
before init data1: 1245582962s, 753431us
after  init data1: 1245582962s, 767496us

before init data2: 1245582962s, 767524us
after  init data2: 1245582962s, 776012us




---
Se observa ca:
-   durata initializare data1 - 14065us
-   durata initializare data2 -  8488us
```

Cum explicați faptul că inițializarea vectorului *data1* durează mai mult decât inițializarea vectorului *data2*?

Data1 se găsește în .data și este stocat în executabil. Zona .data a executabilului va fi mapată în memorie folosind demand paging. Drept consecință, un page-fault în momentul inițializării vectorului data1 va forța citirea de pe disc (din executabil). De partea cealaltă, data2 va fi alocat direct în RAM la cerere (tot prin demand-paging).

1. Care din următoarele instrucțiuni **ar putea** suprascrie adresa de retur a unei funcții? (my_func este o funcție) Motivați și precizați contextul în care se poate întâmpla. (Poate fi un singur răspuns, răspunsuri multiple, nici unul, toate răspunsurile)

```
long *a = malloc(30);  /* definire si alocare */

/* instructiuni */
a)  a = my_func;
b)  *(&a + 4) = my_func;
c)  *(a + 0x4000000) = my_func;
d)  memcpy(my_func, a, sizeof(void *));
```

Cuvânt cheie: ar putea. Unele situații sunt improbabile dar posibile.

Înainte de toate:
- *a este o variabilă (de tip pointer)*
- *a rezidă pe stivă*
- *&a este adresa pe stivă a variabilei a*
- *a (valoarea a) este o adresă de heap (punctează către zona de 30 de octeți alocată folosind malloc)*
- *în general, heap-ul crește în sus și stiva crește în jos*
- *my_func este o funcție deci rezidă în .text (zona de cod)*

a) nu are un efect; se suprascrie valoarea lui a cu adresa funcției my_func
*b) posibil; dacă există unele variabile între [ret][ebp] și [a] atunci &a+4 poate puncta către adresa unde se găsește valoarea de retur și *(&a + 4) o poate suprascrie*
c) posibil; a+0x4000000 înseamnă adunarea a 0x4000000 la o adresă de heap (valoarea lui a); se poate ajunge (greu probabil, dar posibil) la o adresă de retur de pe stivă
d) ciudată, se suprascrie o informație din zona de cod (zona read only); pot apărea erori de acces sau comportanent nedeterminist (în nici un caz nu suprascrierea adresei de retur dintr-o funcție)

2. Un proces este folosit pentru calcularea de transformate Fourier iar un altul este folosit pentru căutarea de informații într-o ierarhie de fișiere. Care dintre cele două procese va avea prioritatea mai mare? De ce?

Proces care calculează transformate Fourier – CPU intensive. Proces care caută informatii într-o ierarhie de fisiere – I/O intensive. În general, procesele I/O intensive au prioritate mai mare. Motivele sunt:
- *creșterea interactivității*
- *împiedicarea starvation (fairness); dacă nu ar fi astfel prioritizate, procesele CPU intensive s-ar transforma în "processor hogs" și ar folosi resursele sistemului*
- *procesele I/O intensive ocupă timp puțin pe procesor deci întârzierea provocată altor procese este mică*

3. Un sistem dispune de un TLB cu 128 de intrări; care este capacitatea maximă a memoriei fizice și a memoriei virtuale pe acel sistem?

Nu există nici o legătură. TLB-ul menține mapări de pagini fizice și pagini virtuale. Conține un subset al tabelei de pagini. Memoria fizică și memoria virtuală pot fi oricât de mici/mari. Nu sunt afectate de dimensiunea TLB-ului.

4. Completați zona punctată de mai jos cu (pseudo)cod Linux (POSIX) sau Windows (WIN32) (la alegere) care va conduce la afișarea mesajului "alfa" la ieșirea standard (standard output) și mesajul "beta" la ieșirea de eroare standard (standard error):

```
/* de completat */
[...]
fputs("alfa", stderr);
fputs("beta", stdout);
```

Nu alterați simbolurile standard fputs (functie), stderr și stdout (FILE *).

Problema este, de fapt, o problemă a paharelor ascunsă. Se dorește ca ieșirea standard să folosească descriptorul 2, iar ieșirea de eroare standard să folosească descriptorul 1.

În pseudocod Linux, lucurile stau astfel:

```
int aux_fd;

/* aux_fd punctează către ieșirea standard */
dup2(STDOUT_FILENO, aux_fd);
/* descriptorul de ieșire standard este închis și apoi punctează către ieșirea de eroare standard */
dup2(STDERR_FILENO, STDOUT_FILENO);
/* descriptorul de ieșire de eroare standard este închis și apoi punctează către ieșirea standard (indicată de aux_fd) */
dup2(aux_fd, STDERR_FILENO)

fputs ....
```

5. Fie următoarea secvență de (pseudo)cod:

```
int *a;
a = mmap(NULL, 4100, PROT_READ | PROT_WRITE, MAP_ANONYMOUS | MAP_PRIVATE, -1, 0);
n = read_int_from_user();
a[n] = 42;
n = read_int_from_user();
a[n] = 42;
```

Ce efect au valorile introduse de utilizator asupra programului? (page faults, erori, scrieri în memorie) Discuție. (o pagină ocupă 4 KB; read_int_from user() citește o valoare întregă de la intrarea standard)

Discuția este ceva mai amplă. Prerequisites:

- *mmap lucrează la nivel de pagină*
- *mmap nu alocă memorie fizică "din prima"; se alocă la acces (demand paging) în urma unui page fault*

Nu am considerat necesar să se observe că a este int și că referirea primei pagini se face cu 0 <= n <= 1024. Au fost considerată validă și observația 0 <= n <= 4096 pentru prima pagină.*

Se solicită alocarea a 4100 de octeți (> 4096, < 8192) deci se vor aloca 2 pagini.

Primul read:

- *n < 0, proababil eroare (SIGSEGV) în cazul în care pagina anterioară este nevalidă (destul de probabil)*
- *n >= 2048 (peste cele două pagini), probabl eroare (SIGSEGV)*
- *0 <= n < 1024 page fault și alocare spațiu fizic și validare pagină pentru prima pagină; fără erori*
- *1024 <= n < 2048 la fel ca mai sus pentru a doua pagină; fără erori (chiar și pentru n >= 1025 (4100/4))*

Al doilea read:

- *n < 0 idem*
- *n >= 2048 idem*
- *n este în aceeași pagină ca mai sus nu se întâmplă nimic*
- *n în cealaltă pagină atunci page fault, alocare spațiu fizic și validare pagină*

Practic mmap(..., 4100, ...) este echilvalent cu mmap(..., 8192, ...).

6. Care este avantajul configurării întreruperii de ceas la valoarea de 1ms? Dar la valoarea de 100ms?

1ms

- *timp de răspuns scurt, interactivitate sporită, fairness, sisteme desktop (întreruperi dese, se diminuează timpul de așteptare pentru fiecare proces*

100ms

- *productivitate (throughtput) sporită, sisteme server (mai puține context switch-uri, mai mult timp pentru "lucru efectiv")*

1. Un proces execută la un moment dat:

```
sigaction(SIGUSR1, &sa, NULL);
```

iar la un moment ulterior

```
write(fd, buffer, 4);
```

În care din situații este mai probabilă înlocuirea majorității intrărilor din TLB? Motivați. (Argumentele și modul de folosire a funcțiilor se prespun corecte.)

Problema se tratează cel mai bine de la coadă la cap. Care sunt situațiile în care se înlocuiesc majoritatea intrărilor din TLB (eventual un flush – golire)? Răspuns: în cazul unei schimbări de context. Se schimbă tabelele de pagini între procesul preemptat și cel planificat, mai puțin partea kernel. TLB-ul se golește (dacă arhitectura și/sau sistemul de operare permite) atunci unele intrări rămân active (zone de memorie partajată comune procesulelor, zone din spațiul kernel). De aici cuvântul "majorității".

Când se realizează un context switch?

- *la terminarea unui proces*
- *la expirarea cuantei de rulare a unui proces*
- *în momentul în care prioritatea procesului curent (cel ce rulează) este sub prioritatea unui proces READY*
- *în cazul blocării procesului curent*

În cazul celor două apeluri doar ultima variantă are sens (blocarea procesului curent). Acest lucru se poate întâmpla doar în cazul apelului write, care este un apel blocant.

2. Care este limita de spațiu de swap pentru un sistem cu magistrala de adrese de 32 de biți cu spațiul de adrese împărțit 2GB/2GB (user/kernel). Dar pentru un sistem cu magistrala de adrese de 64 de biți?

Nu există nici o limitare. Singura limitare este cea impusă de hardware.

3. O funcție signal-safe este o funcție care poate fi apelată fără restricții dintr-o rutină de tratare a unui semnal (signal handler). De ce nu este malloc o funcție signal-safe? Oferiți o secvență de cod pentru exemplificare.

După cum s-a menționat în cateva rezolvări (fără a aduce o contribuție în cadrul răspunsului, însă) funcțiile signal-safe sunt practic echivalente cu funcțiile reentrante. O funcție non-signal-safe este o funcție care folosește variabile statice, astfel că, dacă un semnal întrerupe funcția în programul principal și funcția este rulată în handler este posibil să apară inconsistențe (exact cum se întâmplă în momentul în care un thread este întrerupt și rulează alt thread fără asigurarea accesului exclusiv și consistent la date).

Dacă un semnal întrerupe funcția malloc și, în handler, rulează funcția malloc structurile interne folosite de libc pentru gestiunea alocării memoriei procesului vor fi date peste cap. Funcția printf este, în mod similar, o funcție non-signal-safe pentru că folosește buffer-ele interne ale libc. Mai multe informații aici (https://www.securecoding.cert.org/confluence/x/34At)

Scenariul de exemplificare este de forma:

```
void sig_handler(int signo)
{
       void *p = malloc(100);
}

int main(void)
{
              ....
       void *a = malloc(BUFSIZ);    /* aici sosește semnalul */
              ...
}
```

Răspunsul "malloc poate genera SIGSEGV când deja rulează un signal handler" nu este valid. Malloc nu generează SIGSEGV; cel mult rămâne fără memorie și întoarce NULL. SIGSEGV este generat în momentul accesării unei regiuni invalide a memoriei.

4. Un program execută următoarea secvență de cod:

```
for (i = 0; i < BUFLEN; i++)
    printf("%c", buf[i]);
```

iar altul

```
for (i = 0; i < BUFLEN; i++)
    write(1, buf+i, 1);
```

Care secvență durează mai mult? De ce?

Funcția printf folosește buffering-ul din libc. Acest lucru înseamnă că, până la îndeplinirea uneia dintre cele trei acțiuni de mai jos, nu se face apel de sistem:

- *se umplu buffer-ele*
- *se apelează fflush(stdout)*

- *se transmite newline (\n)*

Apelul de sistem write face apel de sistem de fiecare dată rezultând un overhead important.

Pentru convingere puteți rula testul de aici (http://anaconda.cs.pub.ro/~razvan/school/so/test_printf_write.c). Mai jos este un exemplu de rulare, primul folosind printf al doilea folosind write (se alterează macro-ul USE_PRINTF). Rezultatele sunt, în opinia mea, edificatoare.

```
razvan@valhalla:~/school/2008-2009/so/examen$ time ./test_printf_write > out.txt
real   0m0.076s
user   0m0.060s
sys    0m0.020s

razvan@valhalla:~/school/2008-2009/so/examen$ time ./test_printf_write > out.txt
real   0m5.930s
user   0m0.052s
sys    0m5.868s
```

5. Un proces P se găsește în starea READY. Precizați și motivați două acțiuni care determină trecerea acestuia in starea RUNNING.

Fie Q procesul care rulează în acest moment pe procesor. Situații de trecere a lui P din READY în RUNNING:
- *Q se încheie și P este primul din coada de procese READY*
- *lui P îi este crescută prioritatea peste a lui Q*
- *lui Q îi expiră cuanta și P este primul în coada de procese READY*
- *Q efecuează o operație blocantă (trece în blocking) și P este primul proces în coada de procese READY*

6. De ce obținerea ordonată/ierarhică a lock-urilor previne apariția deadlock-urilor, respectiv apariția fenomenului de starvation?

Ordonarea modului de obținere (achiziție) a lock-urilor în particular și a resurselor în general previne apariția de cicluri în graful de alocare a resurselor și deci apariția deadlock-urilor.

În absența ordinii de obținere un proces P1 poate face Lock(1) și apoi Lock(2). Înainte de Lock(2) este preemptat și procesul P2 face Lock(2) și apoi încearcă Lock(1). Ambele procese rămân blocate în așteptare mutuală (deadly embrace) = deadlock.

Nu există nici o legătură directă în lock-uri și fenomenul de starvation. Fenomentul de starvation caracterizează o durată de așteptare foarte mare pentru un proces gata de rulare. Alte procese îi iau tot timpul "fața" și procesul nu ajunge pe procesor. Se spune că sistemul nu este "fair" (echitabil). Principala formă de asigurare a echității este folosirea noțiunii de cuantă și, în lumea Linux, de epocă și folosirea priorității dinamice a proceselor. Orice formă de locking duce la creșterea nivelului de starvation. Un proces care așteaptă la un semafor intrarea într-o regiune critică dar alte procese intră înaintea sa. Asigurarea fairness-ului poate fi asigurată prin strategii de tipul FIFO. Dar, obținerea ierarhică a lock-urilor nu are un efect vizibil diferit față de folosirea în orice fel a lock-urilor din perspectiva starvation.

Sisteme de operare

20 iunie 2010

Timp de lucru: 90 de minute

NOTĂ: toate răspunsurile trebuie justificate

1. Câte inode-uri va folosi un hard-link către un fișier aflat pe un sistem de fișiere diferit?

Nu se pot crea hard-link-uri către un fișier aflat pe un alt sistem de fișiere. Un hard-link conține un nume și un număr de inode. Inode-ul referit corespunde sistemului local de fișiere, nu altui sistem de fișiere (nu există un identificator al sistemului de fișiere, se presupune cel local).

2. Fie următoarea secvență de comenzi:

```
touch a.txt
ln a.txt b.txt
ln -s b.txt c.txt
```

Comanda ln fără opțiuni creează hard link-uri, iar comanda ln cu opțiunea -s creează symbolic link-uri. Câte inode-uri, respectiv dentry-uri vor fi create în urma rulării comenzilor de mai sus?

Un hard-link se asociază cu un dentry. La fel un nume de fișier. Un symbolic link are asociat un inode, inode ce conține numele fișierului referit. Se vor crea astfel următoarele:
touch a.txt → 1 dentry (a.txt) și 1 inode (aferent fișierului proaspăt creat)
ln a.txt b.txt → 1 dentry (b.txt) ca hard-link la a.txt
ln -s b.txt c.txt → 1 dentry (c.txt) și 1inode (aferent simbolic link-ului proaspăt creat)

Se creează 3 dentry-uri și 2 inode-uri.

3. Un proces execută secvența următoare în două situații diferite:

```
a = malloc(5000);
memset(a, 0, 5000);
```

Într-una din situații rezultă două page fault-uri, iar în alta trei page fault-uri. Explicați acest comportament.

Pentru alocarea celor 5000 de octeți se folosește demand-paging. Accesele la acea zonă conduc la generarea de page fault-uri. Se generează un page fault pentru fiecare pagină. Depinzând de alinierea celor 5000 de octeți pot rezulta două sau trei page fault-uri.

De exemplu, în cazul în care se alocă [500 octeți, 4096 octeți, 404 octeți] pe parcursul a trei pagini, vor rezulta trei page fault-uri după ce se accesează octetul cu indexul 0, octetul cu indexul 500, octetul cu indexul 4596.

În cazul în care se alocă [4096, 904] pe parcursul a două pagini vor rezulta două page fault-uri după ce se accesează octetul cu indexul 0 și octetul cu indexul 4096.

4. Un program citește un fișier de pe disc, operație care durează T1. Imediat după prima rulare, se execută din nou programul și durează T2. T2 este semnificativ mai mic decât T1. Cum explicați?

Citirea unui fișier de pe disc presupune, pe sistemele de operare moderne, interacțiunea cu un subsistem de caching în memorie (buffer cache) al datelor de pe disc. La prima rulare, nu există date în cache-ul din memoria fizică (buffer cache) și toate datele sunt citite de pe disc. La a doua rulare, datele se regăsesc în cache și timpul de citire va fi redus – diferența de acces la memorie față de disc este mare.

5. Care proces este părintele proceselor zombie?

Un proces zombie este un proces care și-a încheiat execuția dar a cărui stare nu a fost "analizată" de procesul părinte – adică procesul părinte nu a apelat wait pentru culegerea de informații despre procesul copil. Drept urmare, procesul zombie are același proces părinte ca procesul obișinuit înainte să-și fi încheiat execuția – nu există un proces specializat care să fie părintele proceselor zombie.

6. Precizați o situație în care accesarea unei adrese virtuale valide produce segmentation fault.

În cazul în care pagina referită de adresă este marcată de tip read-only (fără a fi vorba de copy-on write), un acces de scriere la acea pagină va genera un page fault. Sistemul de operare aanalizează tipul de page fault; fiind vorba de un acces de scriere la o adresă dintr-o zonă marcată read-only (non copy-on-write), conchide că este vorba de un acces invalid. Rezultă transmiterea unui semnal SIGSEGV (pe un sistem Unix) către procesul care a generat accesul, adică afișarea unui mesaj de tipul "Segmentation fault".

7. Este utilă folosirea "canary value" (stack smashing protection) în cadrul funcției de mai jos? Justificați.

```
void f(char *msg)
{
        char *buffer = malloc(10);
        strcpy(buffer, msg);
}

...
f("supercalifragilisticexpialidocious");
...
```

Stack smashing protection se referă la protejarea stivei prin detectarea situațiilor în care adresa de retur a unei funcții este probabil să fie suprascrisă. În cazul particular al secvenței de cod de mai sus, se realizează un buffer overflow la nivelul heap-ului, adică la nivelul variabilei buffer (alocată pe heap folosind malloc). Drept urmare, folosirea stack smashing protection nu are nici o utilitate.

8. Un sistem S1 foloseşte segmentare. Timpul de translatare a unei adrese virtuale într-o adresă fizică este T1. Un sistem S2 folosește paginare, iar timpul de translatare este T2. Care dintre timpii T1 și T2 este mai mare?

În cazul paginării, translatarea unei adrese virtuale în adrese fizică duce la interogarea tabelei de pagini, care rezidă în memorie; diminuarea overhead-ului de acces la memorie se realizează prin folosirea TLB. În cazul unei paginări ierarhice timpul de acces este mai mare.

Dacă descriptorii/selectorii de segment sunt menținuți în registre ale procesorului atunci timpul T1 este mai mic decât timpul T2.

Dacă descriptorii/selectorii de segment sunt menținuți în memorie, atunci T1 este aproximativ egal cu T2 în cazul folosirii unui sistem cu adresare neierarhică și mai mic decât T2 în cazul folosirii unui sistem cu adresare ierarhică.

9. Ce se întâmplă cu sistemul de bază (host) în cazul în care apare o eroare fatală la nivelul nucleului:
a) unei mașini virtuale VMware Workstation;
b) unui container OpenVZ.

Dacă apare o eroare la nivelul unei mașini virtuale VMware, mașina virtuală trebuie repornită (este într-o stare inconsistentă). Sistemul de bază nu este afectat în vreun fel.

OpenVZ este o soluție de operating system level virtualization. Drept urmare, containerele OpenVZ partajează același nucleu de sistem de operare (Linux) cu sistemul de bază (denumit și container-ul 0). Astfel o eroare de nucleu apărută în nucleul unui continaer OpenVZ se manifestă la nivelul tuturor container-elor și a sistemului de bază – este, de fapt, impropriu exprimarea "nucleul unui container OpenVZ" - nucleul este comun tutoror container-elor și sistemului de bază. O astfel de eroare va fi, deci, fatală și sistemului de bază și acesta trebuie repornit.

10. Fie următoarele două secvențe de programe

```
       /* S1 */
fd = open("a.txt", O_RDWR | O_CREAT, 0644);

pid = fork();
if (pid == 0) {
        write(fd, "a", 1);
        close(fd);
        exit(EXIT_SUCCESS);
}

wait(&status);
write(fd, "b", 1);
```

```
       /* S2 */
void *thread_handler(void *arg)
{
        write(fd, "a", 1);
        close(fd);
        return NULL;
}

fd = open("a.txt", O_RDWR | O_CREAT, 0644);
pthread_create(&tid, thread_handler, NULL);
pthread_join(&tid, NULL);
write(fd, "b", 1);
```

În cazul secvenței S2 apelul **`write(fd, "b", 1);`** se întoarce cu eroarea EBADF. Care este explicația? De ce în primul caz nu se întâmplă același lucru?

În cazul secvenței S1, după apelul fork, procesul copil folosește un descriptor propriu (duplicat al descriptorului fd al procesului părinte). Operația close(fd) conduce la închiderea descriptorului doar în procesul copil.

În cazul secvenței S2, thread-ul principal și thread-ul nou creat partajează resursele procesului și, deci, tabela de descriptori a procesului. În consecință, operația close(fd) are sens la nivelul întregului proces și va închide descriptorul. Operația write(fd, "b", 1) executată în thread-ul principal după ce thread-ul creat a închis fișierul folosește un descriptor nevalid. Operația va întoarce EBADFD (Bad file descriptor).

11. Fie următoarea secvență de operații:

```
for (i = 0; i < N; i++)
        a[i] = 1;
```

Secvența este rulată pe două sisteme diferite care nu dispun de TLB sau memorie cache. Pe un sistem au loc N accese la memorie iar pe un alt sistem 2*N accese. Secvența este identică și rulată în aceleași condiții (același program) pe ambele sisteme. Cu ce diferă cele două sisteme?

*Secvența de mai sus conduce la N accese la elemente ale vectorului a[i], aflat în memorie. Într-un caz se produc N accese, deci fiecare acces la un element al vectorului înseamnă 1 acces la memorie. În al doilea caz se produc 2*N accese, deci fiecare acces la un element al vectorului înseamnă 2 accese la memorie.*

Pentru primul caz (N accese) sistemul nu dispune de memorie virtuală – în acest caz un acces în limbajul C se traduce printr-un acces la memoria fizică.

*Pentru al doilea caz (2*N accese) sistemul dispune de memorie virtuală cu adresare neierarhică. Sistemul nu dispune de TLB astfel că fiecare acces la un element al vectorului va însemna un prim acces la tabela de pagini și apoi unul la zona de memorie aferentă elementului, ambele localizate în memorie fizică (RAM) a sistemului.*

Sisteme de operare

22 iunie 2010

Timp de lucru: 90 de minute

NOTĂ: toate răspunsurile trebuie justificate

1. Care dintre secțiunile de memorie de mai jos sunt proprii unui proces dar nu unui program/executabil? Justificați.

```
text, rodata, data, bss, heap, stack
```

Un executabil definește secțiunile text, rodata, data și, fără a aloca spațiu, bss. Secțiunea bss este populată cu zero-uri în momentul creării procesului (load-time). Zonele heap și stack (stivă) sunt zone pur dinamice - țin de evoluția procesului – alocarea memoriei; alocarea pe heap se realizează prin malloc iar alocarea pe stack se realizează în contextul apelurilor de funcții.

2. Precizați o situație în care accesarea unei adrese virtuale valide produce page fault, fără a produce segmentation fault.

Dacă adresa este validă, dar pagina fizică nu este prezentă în RAM (este în swap sau a fost alocată folosind demand-paging), va rezulta page fault, și apoi pagina va fi adusă în RAM sau alocată. Dacă pagina este marcată read-only dar de tip copy-on-write (după un fork) atunci un acces de scriere la pagină va conduce la obținerea unui page fault; page fault-ul va conduce la alocarea unei pagini fizice noi și marcarea acesteia cu drepturi de scriere.

3. Presupunem că avem 3 page frames la dispoziție, toate inițial goale. Se realizează următorul șir de accese (numerele reprezintă pagini virtuale): 3 2 1 0 3 2 4 3 2 1 0 4. Câte page faulturi vor rezulta în urma folosirii algoritmului FIFO? Dar dacă se mărește numărul de page frames la 4?

În tabelul de mai jos, cele 3 linii conțin, respectiv, pagina virtuală aferentă fiecărei pagini fizice (se folosesc 3 frame-uri).

frame1	**3**	3	3	**0**	0	0	**4**	4	4	4	4	**4**
frame2	-	**2**	2	2	**3**	3	3	**3**	3	**1**	1	1
frame3	-	-	**1**	1	1	**2**	2	2	**2**	2	**0**	0

În tabelul de mai jos, cele 4 linii conțin, respectiv, pagina virtuală aferentă fiecărei pagini fizice (se folosesc 4 frame-uri).

frame1	**3**	3	3	3	**3**	3	**4**	4	4	4	**0**	0
frame2	-	**2**	2	2	2	**2**	2	**3**	3	3	3	**4**
frame3	-	-	**1**	1	1	1	1	1	**2**	2	2	2
frame4	-	-	-	**0**	0	0	0	0	0	**1**	1	1

Cu font aldin (bold) au fost marcate paginile virtuale accesate, iar cu font roșu dacă acel acces a generat un page fault. În cazul folosirii a 3 page frame-uri, se obțin 9 page fault-uri, iar în cazul folosirii a 4 page frame-uri se obțin 10 page fault-uri. Acest fenomen poartă numele de anomalia lui Belady (http://en.wikipedia.org/wiki/Belady's_anomaly).

4. Se consideră următoarea schemă de segmentare:

Segment	Base	Length
0	100	1000
1	1200	250
2	1800	300
3	2200	500
4	3000	800

Care dintre următoarele reprezintă adrese logice valide? Adresele sunt de forma (segment, offset) .
a. (0, 820)
b. (1, 430)
c. (2, 13)

În cazul opțiunilor prezentate contează dacă offsetul în cadrul segmentului depășește dimensiunea segmentului (length). Se observă că doar a doua opțiune (b) conduce la depășirea lungimii segmentului (430 > 250), deci nu reprezintă o adresă

logică validă.

5. Dați exemplu de situație în care operația lock(&mutex) conduce la invocarea scheduler-ului și un exemplu de situație în care nu conduce la invocarea scheduler-ului.

Dacă apelul este blocant va conduce la invocarea scheduler-ului. Dacă apelul nu este blocant și nu se produce o analiză a priorităților proceselor, nu va conduce la invocarea scheduler-ului. Apelul este blocant în momentul în care mutex-ul este deja achiziționat. Apelul este neblocant dacă mutexul nu este achiziționat (adică este liber). În caz particular, dacă mutex-ul este implementat în user space (de tip futex) și este liber, nu va genera apel de sistem și nu există "riscul" replanificării acestuia din cauza priorității proceselor sau a altor euristici de planificare ale nucelului.

6. Un sistem de fișiere dispune de un bitmap pentru inode-uri (un bit specifică folosirea sau nu a unui inode) de 32KB. Câte symlink-uri pot fi create? (este suficient ordinul de mărime și justificarea răspunsului)

*32KB = 32*2^10*8biti = 256*2^10 intrări în bitmap. Pot fi create 256*2^10 inode-uri. Întrucât un symbolic link ocupă un inode pot fi create 256*2^10 inode-uri. Se pot scădea câteva inode-uri aferente directorului rădăcină și fișierelor reale către care punctează symbolic link-urile.*

7. Se dă următoarea secvență de execuție :

```
  Thread A             Thread B
-------------       -------------
  work_a1              work_b1
  work_a2              work_b2
```

Realizați sincronizarea celor 2 fire de execuție folosind semafoare astfel încât work_a1 să se execute înainte de work_b2 şi work_b1 să se execute înainte de work_a2. Folosiți primitivele: **`sem_init(sem_t *sem, int count), sem_up(sem_t *sem), sem_down(sem_t *sem)`**.

Presupunem folosirea a două semafoare (s_a și s_b) cu următoarele roluri:
** s_a, thread-ul A a ajuns la punctul de întâlnire*
** s_b, thread-ul B a ajuns la punctul de întâlnire.*

Soluția de sincronizare este cea de mai jos:

```
            sem_t s_a, s_b;
            sem_init(&s_a, 0);
            sem_init(&s_b, 0);

   Thread A              Thread B
-----------------    -----------------
 work_a1               work_b1
    sem_up(&s_a);         sem_up(&s_b);
   sem_down(&s_b);       sem_down(&s_a);
 work_a2               work_b2
```

8. Într-o aplicație multi-threaded există două threaduri. Primul execută o funcție CPU intensive, iar celălalt execută preponderent operații I/O. De ce folosirea implementării threadurilor la nivel user nu este de dorit?

În cazul unei implementări de thread-uri la nivel user, blocarea unui thread conduce la blocarea întregului proces. Thread-ul care execută operații I/O va avea parte de situații dese de blocare (operațiile I/O sunt, în general, blocante). Blocarea acestui thread va conduce la blocarea întregului proces. În acest caz, thread-ul CPU intensive, deși ar putea rula și executa acțiuni utile, este blocat. Acest lucru duce la folosirea necorespunzătoare a procesorului: un thread este pregătit pentru execuție (READY) dar nu poate rula. Pe un sistem cu implementare de thread-uri la nivel kernel, acest lucru nu ar avea loc.

9. De ce, înainte de a realiza un apel exec(), e recomandat să se închidă toate fișierele de care nu are nevoie procesul copil?

Un proces copil moștenește descriptorii procesului părinte. Acest lucru atrage două dezavantaje importante:
** securitate: un proces poate citi, parcurge sau corupe datele din fișierele unui alt proces*
** resurse: menținerea descriptorilor deschiși duce la ocuparea unui număr mare de descriptori de fișier; în cazul în care se creează procese în continuare, tabela de descriptori de fișiere este ocupată în mare măsură de fișiere deschise de alte procese*

10. Care este numărul minim de apeluri de sistem generate de următoarea secvență de pseudocod? (toate apelurile de funcții se

întorc cu succes)

```
acquire_mutex(&m);
write(fd, "abcd", 4);
free(p);
release_mutex(&m);
```

În cazul unei implementări de mutex-uri în user space (de tipul futex) și a unui mutex neocupat, funcțiile acquire_mutex și release_mutex nu generează apel de sistem.

Apelul de bibliotecă free poate să nu genereze apel de sistem (de obicei nu generează) depinzând de implementarea din biblioteca standard C. Atât apelul malloc cât și free alocă, respectiv eliberează, anumite dimensiuni de memorie din heap. În momentul în care se "cumulează" o zonă suficient de mare (de alocat sau eliberat), se realizează apel de sistem (brk).

Apelul de bibliotecă write conduce la invocarea apelului de sistem aferent (sys_write pe Linux).

În consecință, numărul minim de apeluri de sistem generate este 1 (unu), generat de apelul write.

11. De ce nu se poate implementa un mecanism de memorie partajată pentru un sistem cu paginare inversată?

Într-un sistem cu paginare inversată, intrările din tabela de pagini conțin PID-ul procesului și pagina virtuală aferentă. Indexul intrării în tabelă reprezintă frame-ul aferent. Partajarea unei pagini se poate realiza în măsura în care se poate asocia unui frame (unei pagini fizice) mai multe pagini virtuale. Într-un sistem cu paginare inversată, o singură pagină virtuală poate corespunde unei pagini fizice, și nu se poate implementa partajarea memoriei.

Dacă sistemul permite alocarea unei liste de elemente de tip (PID, pagină virtuală) în cadrul unei intrări în tabela de pagini atunci partajarea memoriei se poate implementa, cu dezavantajul unui timp de căutare ridicat (problemă care se poate rezolva prin folosirea de tabele hash).

Sisteme de operare

26 iunie 2010

Timp de lucru: 90 de minute

NOTĂ: toate răspunsurile trebuie justificate

1. Câte procese copil, respectiv părinte poate avea un proces la un moment dat?

Un proces poate avea, la un moment dat, un singur proces părinte și oricâte procese copil, în limita resurselor sistemului. Procesul init poate fi considerat un proces particular care nu are un proces părinte.

2. Un proces execută secvența:

```
for (i = 0; i < 42; i++)
      a++;
```

În timpul execuției secvenței, procesul este preemptat și este planificat alt proces. Dați exemplu de o situație care poate genera preemptarea.

Întrucât secțiunea de mai sus nu este blocantă, procesul poate fi preemptat dacă îi expiră cuanta de timp sau dacă în sistem există un proces cu prioritate superioară pregătit pentru execuție. Această situație este declanșată de apariția întreruperii de ceas.

3. Dați exemplu de o funcție thread safe dar non-reentrantă. Explicați.

O funcție thread safe dar non-reentrantă permite rularea acesteia în context multithreaded dar nu permite existența simultană a două fluxuri de execuție în contextul aceluiași thread/proces. Un exemplu este o funcție care folosește locking. De forma:

```
int my_function(void)
{
    lock(&mutex);
    … /* TODO */ …
    unlock(&mutex);
}
```

4. Se consideră următorul cod:

```
void f()
{
      int *z = malloc(sizeof(int));
      [...]
      printf("z = %p\n", z);
      printf("&z = %p\n", &z);
}
```

După rularea secțiunii se afișează mesajul:

```
      z = 0x12345678
      &z = 0x87654321
```

Asociați adresele z, &z cu secțiunile spațiului de adresă al unui proces: .text, .data, .bss, heap și stack.

z este o variabilă de tip pointer – conținutul acesteia este o adresă. Adresa punctează către o zonă din heap (fiind rezultatul întors de apelul malloc).

&z reprezintă adresa variabilei v. Variabila este o variabilă locală unei funcții deci este alocată pe stivă.

Avem o variabilă alocată pe stivă (adresa ei este o adresă din stivă), iar conținutul acelei variabile (pointer) este o adresă întoarsă de apelulul malloc, adică o adresă din heap.

5. Explicați modul în care se poate produce starvation pe un sistem cu planificare SRTF (Shortest Remaining Time First).

Dacă în cadrul sistemului apar în coada ready procese cu timp de rulare redus, acestea vor fi planificate primele. Presupunând un flux continuu de procese cu timp de rulare redus, procesele cu timp de rulare mare vor ajunge să se execute foarte rar sau deloc, adică să se producă fenomenul de starvation.

6. După schimbarea contextului între două thread-uri, care clase registre au valori diferite (înainte și după schimbarea de context): registrele generale, registrul de stivă, registrele de segment.

La schimbarea de context între două thread-uri, majoritatea registrelor se schimbă. Fiecare thread dispune de valori proprii ale registrelor. Astfel, registrele generale și registrul de stivă se schimbă. Sistemele de operare moderne folosesc rar registrele de segment, astfel că în general acestea nu vor fi schimbate.

În plus, anumite registre interne procesorului, inaccesibile din user space (ring3 pe o arhitectură x86) își pot păstră valorile în cazul thread-urilor diferite (registre precum cr2, cr3 pe o arhitectură x86).

7. De ce anumite zone din bibliotecile partajate sunt mapate read-write? Dati un exemplu.

Bibliotecile partajate conține zone r-x (cod), r-- (read-only data) și rw- (date). Zonele read-write conțin variabile care pot fi scrise de procesul ce folosește biblioteca, precum variabila errno în cazul bibliotecii standard C.

8. Exceptând apelurile de sistem, dați exemplu de situație în care procesorul comută în kernel space.

Procesorul execută cod kernel în cazul unor solicitări din user space (apel de sistem) sau de la hardware (întreruperi). Procesorul execută, astfel, cod kernel, exceptând apelurile de sistem, în momentul sosirii unei întreruperi.

9. Un sistem dispune de N procese. Fiecare proces dispune de M pagini virtuale nealocate. Sistemul dispune de o singură pagină fizică disponibilă. Care este numărul maxim de pagini virtuale care pot fi asociate cu pagina fizică?

*Oricâte pagini virtuale pot fi asociate cu o pagină fizică. Implementări de tipul mmap permit maparea unei zone de memorie virtuale peste o zonă de memorie fizică. În situația de mai sus, un proces poate mapa toate cele M pagini virtuale proprii peste acea pagină fizică. În total, pentru cele N procese, se pot mapa N*M pagini virtuale.*

10. Explicați de ce nu se poate implementa mecanismul de swapping pe un procesor fără unitate de management al memoriei.

Unitatea de management a memoriei (MMU) este responsabilă cu translatarea paginilor virtuale în pagini fizice. Absența MMU conduce la absența mecanismului de memorie virtuală. Mecanismul de memorie virtuală permite existența unui spațiu virtual de adrese care depășește spațiul fizic – spațiul suplimentar poate fi furnizat de disc, prin intermediul swap-ului. În cazul absenței mecanismului de memorie virtuală, nu se poate referi/folosi spațiul de swap, deci nu se poate implementa mecanismul de swapping (evacuare pe disc și recuperarea paginilor de pe disc).

11. Un inode dispune de 10 pointeri de indirectare simplă a blocurilor de date. Un bloc ocupă 4096 de octeți. Știind că un dentry ocupă 64 de octeți, câte intrări poate avea maxim un director?

10 pointeri de indirectare simplă punctează către blocuri care conțin, la rândul lor, pointeri. Considerând că un pointer ocupă 4 de octeți, rezultă că un bloc de pointeri conține 4096/4 = 1024 de pointeri.

*Cei 10 pointeri de indirectare simplă vor referi 10 blocuri care conțin, la rândul lor, 10*1024 pointeri adică 10240.*

*Fiecare dintre cei 10240 pointeri punctează către un bloc de date. Fiecare bloc de date ocupă 4K. Rezultă așadar, că un inode poate referi 10240*4KB de date.*

*În cazul unui director, datele sale sunt un vector (array) de dentry-uri. Numărul maxim de dentry-uri se obține împărțind spațiul maxim ce poate fi referit de un inode la dimensiunea unui dentry. În consecință, un director poate conține 10240*4KB / 64 dentry-uri, adică 10240 * 64 = 655360.*

1. Două procese (P1, P2) folosesc o zonă de memorie partajată, rulează într-un sistem preemptiv și execută secvența de cod de mai jos. Știind că valoarea inițială indicată de pointerul **counter** este 0 și că acesta indică în zona de memorie partajată, care este valoarea indicată de pointerul **counter** la finalul execuției celor două procese?

P1	P2
`if (*counter == 0)` `  (*counter)++;`	`if (*counter == 0)` `  (*counter)++;`

Dacă cele două procese se execută secvențial atunci primul proces va incrementa variabila pe 1, iar al doilea nu va executa instrucțiunea de incrementare din if. Valoarea finală va fi 1.

*În varianta în care procesul P1 este preemptat de procesul P1 după verificarea if (*counter == 0), dar înainte de (*counter)++ atunci procesul P2 va testa, la rândul său, ca fiind adevărată condiția (*counter == 0). În consecință, va incrementa valoarea contorului. P1 va incrementa, de asemenea, valoarea contorului, rezultând valoarea finală 2.*

2. Pe un sistem dual processor cu 128MB de RAM și swap de 256MB rulează un sistem Linux. Câte procese se pot găsi la un moment dat în starea **RUNNING**, **READY** respectiv **WAITING**?

*Caracteristica ce influențează numărul de procese din starea RUNNING este numărul de unități de execuție. Fiind vorba de un sistem dual procesor, pot exista maxim **2 procese în coada RUNNING**.*
***În coada READY și WAITING se pot găsi oricâte procese**, limita fiind dată de constrângerile și resursele sistemului.*

3. Un sistem folosește TLB în care fiecare intrare conține trei câmpuri (pid, frame, pagină virtuală). Care este avantajul acestei implementări față de o implementare care conține doar două câmpuri (frame, pagină virtuală)?

Prezența câmpului pid înseamnă că se poate realiza o selecție după proces a intrărilor din TLB. Acest lucru este util în cazul schimbărilor de context. În cazul unei schimbări de context cea mai mare parte a intrările din TLB sunt anulate. Prezența unui câmp pid înseamnă că, în cazul unei schimbări de context, doar intrările specifice procesului vor fi eliminate rezultând într-un număr mai mic de accese directe la tabela de pagini.

4. Descrieți în pseudocod cum se poate determina dacă stiva crește de la adrese mari la adrese mici sau invers.

O soluție ține cont de construirea stack frame-urilor pentru funcții:

```
int *p_fcaller;
int *p_fcalled;

void f2(void)
{
        int f2_local;
        p_fcalled = &f2_local;

        if (p_fcalled > p_fcaller)
                printf("stack grows up\n");
        else
                printf("stack grows down\n");
}
void f1(void)
{
        int f1_local;
        p_fcaller = &f1_local;
        f2();
}
```

Exemplu complet aici: http://swarm.cs.pub.ro/git/?p=razvan-code.git;a=blob;f=tests/stack_grow.c;h=3ebc0408a4cf30cb8317714fb5c473c35ca7bb3d;hb=21d876d26d3db2145f6be96423cbc14a7ddaf21a

5. Câte pagini fizice vor ocupa stivele celor două procese rezultate în urma apelului **fork** exact înainte de apelul **exit**? Apelul **fork** se întoarce cu succes.

```
int main(void)
{
        char buf[3 * PAGE_SIZE];
        buf[0] = 'a';
        buf[PAGE_SIZE] = 'z';
        fork();
        buf[0] = 'b';
        exit(EXIT_SUCCESS);
}
```

Bufferul buf este alocat folosind demand paging. Acest lucru înseamnă că nu vor fi alocate pagini fizice până în momentul accesului. După buf[0] = 'a' și buf[PAGE_SIZE] = 'z' rezultă două page fault-uri care vor genera alocare a două pagini fizice.

După fork paginile sunt marcate copy-on-write.

După apelul fork() atât procesul părinte cât și procesul copil accesează buf[0]. Acest lucru va rezulta într-un page fault (se duplică pagina și pagina nouă si pagina veche primesc drept de scriere). Se va aloca astfel o pagină fizică nouă.

În final stivele celor două procese vor ocupa 3 pagini fizice.

6. În urma rulării secvenței de cod de mai jos fișierul **a.txt** conține șirul **222313**. După fiecare apel **write** actualizați următorul vector (cursor fd1, cursor fd2, cursor fd3, conținut fișier). Exemplu: inainte de primul write vectorul va fi (0, 0, 0, '').

```
fd1 = open("a.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
fd2 = open("a.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
fd3 = dup(fd1);

write(fd1, "1", 1);
write(fd2, "2", 1);
write(fd3, "3", 1);

pid = fork();


/* continuat pe coloana a doua */
```

```
switch (pid) {
case 0:
        write(fd1, "1", 1);
        write(fd2, "2", 1);
        write(fd3, "3", 1);
        break;
default:
        wait(&status);
        write(fd1, "1", 1);
        write(fd2, "2", 1);
        write(fd3, "3", 1);
        break;
}
```

	inițial	w1	w2	w3	w4	w5	w6	w7	w8	w9
f1.cursor	0	1	1	2	3	3	4	5	5	6
f2.cursor	0	0	1	1	1	2	2	2	3	3
f3.cursor	0	1	1	2	3	3	4	5	5	6
conținut	“”	“1”	“2”	“23”	“231”	“221”	“2213”	“22131”	“22231”	“222313”

Procesul părinte și fiu partajează descriptorii de fișier și cursorul de fișier. Orice modificare a cursorului în procesul copil va fi vizibilă în procesul părinte și invers.
fd3 partajează cursorul de fișier cu fd1 ca urmare a apelului dup. Orice incrementare a cursorului folosind descriptorul fd1 va fi vizibila descriptorului fd3 și invers.

1. Fie următoarele procese și timpii lor de execuție și de intrare (process, timp execuție, timp intrare): (P1, 8, 1), (P2, 4, 2), (P3, 3, 3), (P4, 7, 4). Determinați timpii medii de așteptare pentru First Come First Served (FCFS) și Shortest Job First (SJF).

În cadrul răspunsului, am marcat cu (a, b) intervalul în care un proces așteaptă și cu [a, b] intervalul în care un proces este activ pe procesor

= FCFS =

P1: [1,9]
P2: (2, 9) [9,13]
P3: (3, 13) [13,16]
P4: (4, 16) [16,23]

Procesul P1 intră în sistem la momentul 1 și, fiind primul proces, lucrează 8 unități de timp până la momentul 9. Nu așteaptă.
Procesul P2 intră în sistem la momentul 2 și așteaptă procesul P1; va fi planificat înaintea proceselor P3 și P4. Va aștepta până la încheierea procesului P1 adică intervalul (2, 9) Începe să lucreze în intervalul [9,13] pentru 4 unități de timp.
Procesul P3 intră în sistemul la momentul 3 și așteaptă procesul P1 și P2, adică intervalul (3,13). Va lucra în intervalul [13,16] pentru 3 unități de timp.
Procesul P4 intră în sistemul la momentul 4 și așteaptă procesul P1, P2 și P3, adică intervalul (4,16). Va lucra în intervalul [16,23] pentru 7 unități de timp.

Timp mediu de așteptare = (0 + 7 + 10 + 12) / 4 = 7.25

= SJF =

P1: [1,9]
P2: (2,12)[12,16]
P3: (3,9)[9,12]
P4: (4,16)[16,23]

** Procesul P1 întră în sistem la momentul 1 și rulează pe procesor [1,9].*
** La momentul 2 intră procesul P2 al cărui timp de execuție este 4. Întrucât primul proces rulează, nu va preempta procesorul.*
** La momentul 3 intră procesul P3 al cărui timp de execuție este 3. Întrucât primul proces rulează, nu va preempta.*
** La momentul 4 intră procesul P4 al cărui timp de execuție este 7. Nu preemptează nici un proces și așteaptă.*
** La momentul 9, P1 își încheie execuția. Se planifică procesul cu timpul de execuție cel mai scurt. Se planifică P3 (timp de execuție 3). P3 rulează pe procesor în intervalul [9,12].*
** La momentul 12, P3 își încheie execuția. Se planifică procesul cu timpul de execuție cel mai scurt. Se planifică P2 (timp de execuție 4). P3 rulează pe procesor în intervalul [12,16].*
** La momenutl 16, P2 își încheie execuția. Se rulează procesul P4.*
** La momentul 23, procesul P4 își încheie execuția.*

Timp mediu de așteptare = (0 + 10 + 6 + 12) / 4 = 7

2. Câte procese se vor crea în urma execuției secvenței de mai jos (se exclude procesul curent)? Toate apelurile fork se întorc cu succes.

```
l1)    fork();
l2)    if (fork() == 0)
l3)            fork();
```

Apelul fork() întoarce 0 în procesul copil și cu PID-ul procesului copil în părinte
Fie P1 primul proces (cel existent).
După linia 1 (l1) primul proces creează un proces copil, P2. Întrucât nu se verifică valoarea de ieșire a apelului fork, ambele procese (P1 și P2) vor continua cu linia l2
În cadul liniei 2 cele două procese de mai sus creează câte un proces copil. Fie P3 procesul copil pentru P1 și P4 procesul copil pentru P2. În cadrul liniei se verifică valoarea întoarsă de apelul fork. Doar procesele copil proaspăt create vor satisface condiția if (fork() == 0). Adică foar procesele P3 și P4 vor continua la linia l3.
În cadrul liniei l3, procesele P3 respectiv P4 creează câte un proces copil; fie acestea P5 și P6.
La sfârșitul secțiunii vor exista 6 procese: procesul inițial (P1) și 5 noi procese create (P2, P3, P4, P5, P6).

3. Fie secvențele de pseudocod de mai jos. Care din cele două abordări este optimă?

`mutex_lock(&mutex);`
`a++;`
`mutex_unlock(&mutex);`
	`spin_lock(&spinlock);`
	`a++;`
	`spin_unlock(&spinlock);`

Secvențele de mai sus urmăresc accesul exclusiv pentru incrementarea variabilei a. Incrementarea variabilei a este o operație rapidă și puțin consumatoare de timp. Se dorește, așadar, un mecanism de sincronizare/asigurare a accesului exclusiv cât mai rapid. Acest mecansim este asigurat de spinlock-uri care operează foarte rapid în cadrul primitivelor spin_lock și spin_unlock. În vreme ce o operație pe un mutex impune contactarea planificatorului, spinlock-ul impune o operație de busywaiting care va fi, însă, foarte rapidă ținând cont de dimensiunea redusă a regiunii critice (necesită doar incrementarea variabilei a).

4. În secvența de cod de mai jos, PAGE_SIZE reprezintă dimensiunea unei pagini, iar apelul mmap reușește. În urma rulării a 10 instanțe ale programului de mai jos se ocupă 11 pagini fizice în zonele de date (date inițializate, heap, readonly (.rodata), bss). Argumentați acest comportament.

```
const char x[PAGE_SIZE] = {1, };

int main(void)
{
        char *z = mmap(NULL,  PAGE_SIZE, PROT_READ, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
        printf("%c %c\n", x[0], z[0]);
        while (1)  /* wait forever */
                ;
        return 0;
}
```

Vectorul x ocupă o pagină și fiind declarat const este read-only. Acest lucru înseamnă că pagina fizică aferentă acestuia poate fi partajată între mai multe procese. Nu este nevoie de copie separată pentru fiecare proces în parte.

Vectoul z ocupă, de asemnea, o pagină. Fiind vorba de o mapare privată fiecare proces va dispune de o pagină separată.

Alocările se realizează folosind demand-paging. Doar în momentul accesului la date, se vor aloca pagini fizice.

*Pentru fiecare instanță de proces, se alocă, în momentul accesului z[0] o pagină fizică nouă pentru z. Pagina fizică aferenta lui x va fi identică pentru toate procesele, întrucât este read-only. După rularea a 10 procese vor rezulta 10 * 1 pagină (pentru z) + 1 pagină (pentru x) = 11 pagini fizice.*

5. Care dintre apelurile sigaction și sigemptyset va dura mai mult? sigaction actualizează rutina de tratare a unui semnal, iar sigemptyset initializeaza setul de semnale descris de structura sigset_t la 0.

sigaction este apel de sistem și are un overhead inerent. sigemptyset operează la nivelul biților dintr-o zonă de memorie, operație foarte rapidă și care nu dispune de un overhead suplimentar. În consecință, durata sigaction este vizibil mai mare decât durata sigemptyset. Puteti verifica pe codul de aici: http://swarm.cs.pub.ro/git/?p=razvan-code.git;a=blob;f=tests/measure_time_syscall.c;h=f7046aedb4b8af9ffe65cc84533ca43c3888a1d6;hb=refs/heads/examples

6. Fie un sistem pe 32 biți cu 100MB RAM, 100MB de swap și pagini de 4KB. Spațiul de adresă virtual este împărțit 3GB programe utilizator / 1 GB kernel. Un program aflat în execuție rulează:

```
        void *ptr = malloc(1024 * 1024 * 1024);
```

Apelul se realizează cu success, returnând un pointer valid. Motivați de ce malloc se întoarce cu succes.
Ce se întâmplă și de ce, dacă se rulează:

```
        memset(ptr, 0, 1024 * 1024 * 1024);
```

malloc alocă memorie folosind demand-paging. În concluzie se alocă memorie pur virtuală fără a dispune de suport fizic. Memoria fizică nu este folosită și apelul reușește.

În cadrul apelului memset, se produc page-fault-uri la accesarea diferitelor adrese indicate de ptr. Pentru fiecare page-fault se alocă o nouă pagină fizică. În jurul valorii de 200MB, spațiul din memoria RAM și cel de pe swap se vor fi umplut, drept pentru care sistemul va rămâne fără memorie, se va activa un sistem de management de tipul OOM (Out of memory) (http://en.wikipedia.org/wiki/Out_of_memory) care va începe să omoare procese. În final, sistemul va suferi crash/se va bloca din cauza absenței memoriei fizice.

Nume și grupă:

Sisteme de Operare

2 septembrie 2013

Timp de lucru: 100 de minute

Notă: Toate răspunsurile trebuie justificate

1. (**1 punct**) De ce **NU** este posibil ca un planificator să fie, simultan, echitabil (fair) și productiv (să ofere un throughput mare)?

- Pentru că ai un overhead la schimbarea de task-uri. Ca să fii echitabil, fiecare proces trebuie să aibă un timp mai mult sau mai puțin egal pe procesor, însă schimbarea de pe un proces pe altul durează -> nu poți fi productiv.

2. (**1 punct**) De ce este afectat sistemul gazdă (host) în cazul apariției unei erori fatale la nivelul nucleului unui container OpenVZ?

3. (**1 punct**) Funcția malloc alocă memorie. Funcția calloc alocă memorie și completează spațiul cu zero-uri. De ce un apel calloc generează, de obicei, mai multe page fault-uri decât malloc?

Pentru că malloc nu verifică dacă memoria este alocată sau nu, și nu îți dai seama de asta decât atunci când faci o scriere. Calloc scrie 0 în fiecare bit, deci dacă apar probleme de permisiuni, calloc o să țipe (not the most decent explanation)

4. (**1 punct**) De ce este preferată folosirea apelurilor asincrone, acolo unde există, în locul celor sincrone?

- Pentru că prin folosirea apelurilor sincrone, procesul este blocat, așteptând răspunsul apelului. La folosirea unui apel asincron, procesul poate executa alte operații (sau poate fi înlocuit pe procesor de alt proces) în timp ce așteaptă după răspunsul la apel)

5. (**1 punct**) Trei thread-uri afișează continuu respectiv mesajul "red", "green", "blue". Descrieți, ținând cont de sincronizare, cele trei funcții aferente thread-urilor astfel încât să se afișeze "red green blue red green blue red green blue ...".

6. (**1 punct**) De ce este indicat, din punct de vedere al securității, să fie folosit apelul printf("%s", argv[1]) în locul apelului printf(argv[1])?

- Pentru că în primul apel printf o să afișeze un string. În al doilea apel, printf poate afișa orice (inclusiv cifre, ce ar cauza un overflow sau type mismatch)

7. (**1 punct**) Creați un paragraf adevărat, informativ și argumentat din domeniul sistemelor de operare care să cuprindă ca subiecte principale noțiunile de *fișier mapat în memorie* și *apel de sistem*.

8. (**1 punct**) Creați un paragraf adevărat, informativ și argumentat din domeniul sistemelor de operare care să cuprindă ca subiecte principale noțiunile de *hard link, symbolic link* și *inode*.

9. (**1 punct**) Care sunt asocierile dintre clasele asincron, zero-copying, sincron blocant și apelurile/tehnologiile read, readv, ReadFile, mmap, sendfile, Completion Ports, io_submit?

10. (**1 punct**) Care sunt asocierile dintre perechile de mai jos (unul la unul, unul la mai multe,

Commented [ln1]: It seems you can add comments here, too.

Nume și grupă:

mai multe la unul, mai multe la mai multe)?

(procesor, registru)
- unul la mai multe

(spațiu de adresă, stivă)

- unul la mai multe

(procesor, proces) -

unul la mai multe

(hard link, inode)
-mai multe la unul

(mutex, thread-uri în așteptare)
- unul la mai multe

11. (**2.5 puncte**) Se dă un server TCP pe care trebuie să îl caracterizați din punct de vedere al performanței. În general, asta înseamnă să măsurați, de exemplu:

- numărul de cereri servite pe secundă
- numărul maxim de clienți serviți simultan

throughput de date
Nume și grupă:
și cum variază aceste trei mărimi una în funcție de cealaltă.

Descrieți cum realizați o arhitectură de sistem (topologie, legături, scenarii de folosire) care să poată măsura acești parametri. Elaborați având în vedere faptul că software-ul de server rulează pe un calculator server mult mai performant decât calculatoarele pe care le aveți la dispoziție pentru testare.

12. (**2.5 puncte**) Pentru aceeași situație ca la punctul anterior, descrieți arhitectura software a software-ului de măsură: single threaded, multi-threaded, multi-proces? Ce alți parametri ați mai putea măsura?

Nume și grupă:
10 iunie 2013

1. (**1 punct**) Știind că overhead-ul unui apel de sistem este de 7 ms și overhead-ul tratării unui page fault este de 2 ms, în ce situație un apel memcpy va dura mai mult decât un apel de sistem?

2. (**1 punct**) De ce este o eroare de tip *Segmentation fault* urmată de o schimbare de context?
Pentru că după ce se rulează handler-ul întreruperii de segfault, fie se va pasa contextul sistemului de operare, în cazul în care programul nu are handler pentru segfault, fie contextul va deveni cel al programului în cauză, pentru a își putea reveni din situația creată

3. (**1 punct**) De ce putem crea un container OpenVZ în cadrul unui mașini virtuale VMware Workstation, dar nu și invers?

4. (**1 punct**) Precizați și justificați valoarea de adevăr a următoarei afirmații: *Un apel write blocant apelat de un proces multithreaded cu implementare în user-space* ***NU*** *va cauza un TLB flush*.

Nu va cauza un TLB flush pentru că apelul este executat în user-space, deci se face buffered

5. (**1 punct**) Un proces execută secvența:

```
for (i = 0; i < 42; i++)
        a++;
```

În ce situație poate genera această secvență un TLB flush?

6. (**1 punct**) Majoritatea planificatoarelor I/O (*I/O scheduler-ul*) realizează operații de tip *sorting and merging* pe cererile de lucrul cu discul. De ce aceste planificatoare sunt utile pentru discuri IDE, dar nu sunt potrivite pentru dispozitive de stocare de tip SSD (*Solid State Drive*)?
Pentru ca SSD-urile sunt construite pe o cu totul alta tehnologie fata de HDD-uri, tehnologie in care nu se foloseste un cap de citire deci nu exista latenta la seek, algoritmii de disk-scheduling nu isi au rostul. In orice ordine s-ar realiza operatiile, overhead-ul e acelasi.

7. (**1 punct**) Creați un paragraf adevărat, informativ și argumentat din domeniul sistemelor de operare care să cuprindă ca subiecte principale noțiunile de *ASLR* (*Address Space Layout Randomization*) și *shellcode*.

Folosirea ASLR împiedică rularea de shellcode-uri de pe stivă prin poziționarea stivei în diverse zone din spațiul de adresă; atacatorul nu poate ști (ușor) care este adresa de start a shellcode-ului. Are eficiență în special pe sistemele pe 64 de biți. Pe cele pe 32 de biți se poate folosi brute force.

8. (**1 punct**) Creați un paragraf adevărat, informativ și argumentat din domeniul sistemelor de operare care să cuprindă ca subiecte principale noțiunile de *stack overflow* și *thread*.
Folosirea unui număr mare de thread-uri în cadrul unui proces poate conduce mai rapid la stack overflow. Fiecare thread are stiva proprie, cu dimensiunea fixată la crearea. Dacă se creează prea multe thread-uri, se va ocupa foarte mult stiva. Stivele thread-urilor vor fi apropiate unele de altele astfel că, în cazul unui flux de apeluri mare (apeluri recursive, de

Nume și grupă:

exemplu), există riscul ca stiva unui thread să suprascrie stiva altui thread.

9. (**1 punct**) Fie următoarea comandă rulată într-un shell. Identificați procesele, relațiile părinte-copil, descriptorii de fișier și redirectările existente.

```
cat examen | grep raspuns > corect
cat examen | grep raspuns > corect
```

Procese:
- Bash
- Cat
- Grep

Relații
- Bash – părinte la cat
- Cat – părinte la grep

Descriptori de fișier:
- Pipe va avea doi descriptori de fișier (output-ul de la cat examen și input-ul de la GREP
- Mai avem un descriptor către fișierul corect, în care scrie grep

Redirectări
- Stdout de la cat la stdin grep
- Stdout grep la corect

10. (**1 punct**) Ce drepturi (citire, scriere, execuție) au următoarele zone din spațiul de adresă al unui proces: text, data, rodata, bss, stivă, heap, biblioteci mapate? Justificați.

Executable files include four canonical sections called, by convention, `.text`, `.data`, `.rodata`, and `.bss`. The `.text` section contains executable code and is packed into a segment which has the read and execute access rights. The `.data` and `.bss` sections contain initialized and uninitialized data respectively, and are packed into a segment which has the read and write access rights.

11. (**2.5 puncte**) Se cere să construiți un server care ascultă pe un socket TCP și servește cereri efectuate într-un limbaj propriu împachetat in XML. Este nevoie să ofere suport pentru maxim 1000 de clienți simultan și să servească 10.000 de cereri pe secundă în total din partea acestor clienți. Alcătuiți schema bloc a acestui server și detaliați blocul de comunicație peste TCP în pseudocod. Explicați alegerile făcute.

Blocurile/modulele implicate intro diagrama, despre care ar trebui discutat, sunt:

1. TCP communication
2. I/O model (async vs threading)
3. XML parser

Daca le luam de la coada la cap, parserul XML trebuie sa fie lightweight si sa implementeze un subset necesar comunicarii intre client si server.

Modelul de I/O, daca ne uitm la numarul maxim de clienti cerut de specificatii, de 1000,

putem sa implementam folosind 1000 de threaduri care asculta pe acelasi socket. Daca insa ne uitam la faptul ca trebuie sa serveasca un numar relativ mare de tranzactii pe secunda, probabil ca un model async I/O ar fi mai potrivit, eliminand context switchurile dintre cele 1000 de threaduri din celalalt model. In cele din urma, probabil ca un model hibrind cu un thread care asculta pe un socket si foloseste async IO si un numar de threaduri care este o functie de numarul de core-uri este cel mai eficient.

Modulul de TCP are o particularitate interesanta. Din starea de listen, la aparitia unei conexiuni noi se creeaza un fd de date. TCPul fiind un protocol de tip stream, sunt doua posibilitati. Daca clientii trimit cereri care au toate nevoie de raspuns, atunci este ok, serverul citeste de pe socket pana cand se poate forma un mesaj XML corect, apoi il trimite mai departe catre procesare si trimite raspunsul inapoi pe aceeasi conexiune de date, dupa care inchid conexiunea. Cazul mai complicat este cand un client poate trimite notificari catre server fara sa astepte raspuns, caz in care este necesar sa se detecteze si sa se delimiteze mai multe requesturi in acelasi buffer. In acest caz este posibil sa apara si desincronizari intre client si server, daca clientul trimite notificari foarte rapid, bufferul de TCP poate sa contina requesturi incomplete, ceea ce complica detectia de mesaje bine formate.

12. (**2.5 puncte**)
Nume și grupă:

Un program are nevoie să stocheze 1.000.000 de fișiere pe disc, fiecare de 10MB, care să poată fi accesate pe bază de identificatori numerici unici. Alcătuiți schema bloc a unui sistem de stocare care să ofere suport pentru acest volum de informații și detailați blocul de identificare a fișierului pe disc.

Aici problema consta in faptul ca nu este scalabil sa stochezi 1,000,000 de fisiere in acelasi director, si nici intro baza de date nu prea are sens sa stochezi ca bloburi toata povestea asta care insumeaza 10TB de date. Asadar, e clar ca fisierele vor trebui stocate pe disc in foldere separate, cel mai bine intro structura arborescenta, care sa se poata intinde pe oricate filesystemuri, deoarece e vorba de o capacitate mai mare decat discurile uzuale. Asadar, blocurile implicate in aceasta solutie sunt doua:

1. Bloc de identificare
2. Bloc de stocare

Blocul de identificare se poate implementa cu o tabela simpla in orice sistem de baze de date, care face o mapare de la un identificator unic la o cale de fisier pe disc.

Blocul de stocare are un API simplu, prin care i se cere, pentru un nou fisier de stocat, locul in care se va stoca. Pentru a face asta, trebuie sa stie ce discuri exista in sistem, ce capacitati au si ce filesystem au, pentru a determina un numar optim de intrari in director pentru fiecare dintre ele. Cand se cere un slot pt un fisier nou, sistemul cauta pe volumele existente cel mai bun loc in termeni de spatiu disponibil, intrari in director, etc. Se poate implementa si un load balancer care sa distribuie fisierele cele mai cerute pe discuri diferite, etc.

În conformitate cu ghidul de etică al Departamentului de Calculatoare, declar că nu am copiat și nu voi copia la această lucrare. De asemenea, nu am ajutat și nu voi ajuta pe nimeni să copieze la această lucrare.

Nume și grupă:

Semnătură:.....

Nume și grupă:

Sisteme de Operare

14 iunie 2013

Timp de lucru: 100 de minute

Notă: Toate răspunsurile trebuie justificate

1. (**1 punct**) Fie T1 timpul de mutare a unui fișier pe aceeași partiție și T2 timpul de mutare a aceluiași fișier pe o altă partiție. Ce relație există între T1 și T2 și de ce?

T1 < T2

In cazul in care se muta un fisier pe aceeasi partitie, tot ce se intampla este crearea unui nou hard link catre inode-ul referentiat de fisier si stergerea vechiului link. In situatia in care se muta fisiere intre partitii, datele chiar se muta. Astfel, T2 este mai mare deoarece are overhead de la mutarea datelor.

2. (**1 punct**) Bibliotecile partajate conțin zone de memorie mapate în cadrul proceselor cu următoarele permisiuni: r-x (cod), r-- (read-only data) și rw- (date). Care din aceste zone **NU** trebuie să fie partajate între mai multe procese care folosesc aceeași bibliotecă și de ce?

3. (**1 punct**) Două sisteme identice din punct de vedere al hardware-ului și al sistemului de operare sunt folosite în scopuri diferite. Primul este folosit pentru calculul de transformate Fourier, iar al doilea este folosi drept server (web, e-mail, ssh, dns, dhcp, etc.). Care dintre cele două va petrece mai mult timp în schimbări de context și de ce?

Al doilea!

There are three potential triggers for a context switch:

Multitasking[edit]

Most commonly, within some scheduling scheme, one process needs to be switched out of the CPU so another process can run. This context switch can be triggered by the process making itself unrunnable, such as by waiting for an I/O or synchronization operation to complete. On a pre-emptive multitasking system, the scheduler may also switch out processes which are still runnable. To prevent other processes from being starved of CPU time, preemptive schedulers often configure a timer interrupt to fire when a process exceeds its time slice. This interrupt ensures that the scheduler will gain control to perform a context switch.

Interrupt handling[edit]

Modern architectures are interrupt driven. This means that if the CPU requests data from a disk, for example, it does not need to busy-wait until the read is over; it can issue the request and continue with some other execution. When the read is over, the CPU can be *interrupted* and presented with the read. For interrupts, a program called an *interrupt handler* is installed, and it is the interrupt handler that handles the interrupt from the disk.

When an interrupt occurs, the hardware automatically switches a part of the context (at least enough to allow the handler to return to the interrupted code). The handler may save additional context, depending on details of the particular hardware and software designs. Often only a minimal part of the context is changed in order to minimize the amount of time spent handling the interrupt. The kernel does not spawn or schedule a special

Nume și grupă:

process to handle interrupts, but instead the handler executes in the (often partial) context established at the beginning of interrupt handling. Once interrupt servicing is complete, the context in effect before the interrupt occurred is restored so that the interrupted process can resume execution in its proper state.

User and kernel mode switching[edit]

When a transition between user mode and kernel mode is required in an operating system, a context switch is not necessary; a mode transition is *not* by itself a context switch. However, depending on the operating system, a context switch may also take place at this time.

4. (**1 punct**) În ce situație pot două procese copil ale aceluiași proces să aibă același PID?

Neither `fork()` nor `vfork()` keep the same PID although `clone()` *can* in one scenario [*a]. They are all different ways to achieve roughly the same end, the creation of a *distinct* child.

`clone()` is like `fork()` but there are many things shared by the two processes and this is often used to enable threading.

`vfork()` is a variant of `clone` in which the parent is halted until the child process exits or executes another program. It's more efficient in those cases since it doesn't involve copying page tables and such. Basically, everything is shared between the two processes for as long as it takes the child to load another program.

Contrast that last option with the normal copy-on-write where memory itself is shared (until one of the processes writes to it) but the page tables that reference that memory are copied. In other words, `vfork()` is even *more* efficient than copy-on-write, at least for the fork-followed-by-immediate-exec use case.

But, in *most* cases, the child has a different process ID to the parent.

[*a] Things become tricky when you `clone()` with `CLONE_THREAD`. At that stage, the processes still have different identifiers but what constitutes the PID begins to blur. At the deepest level, the Linux scheduler doesn't care about processes, it schedules threads.

A thread has a thread ID (TID) and a thread group ID (TGID). The TGID is what you get from `getpid()`.

When a thread is cloned *without* `CLONE_THREAD`, it's given a new TID and it also has its TGID set to that value (i.e., a brand new PID).

With `CLONE_THREAD`, it's given a new TID but the TGID (hence the reported process ID) remains the same as the parent so they really have the same PID. However, they can distinguish themselves by getting the TID from `gettid()`.

There's quite a bit of trickery going on there with regard to parent process IDs and delivery of signals (both to the threads within a group and the `SIGCHLD` to the parent), all which can be examined from the `clone()` man page.

5. (**1 punct**) Două procese scriu și citesc un fișier, în mod sincronizat, prin următoarea secvență de pseudo-cod:

Nume și grupă:

```
acquire_mutex (&m);
if (condition)
    write_to_file (content);
else
    read_from_file(content);
release_mutex(&m);
```

Care este numărul minim de apeluri de sistem care sunt generate în secvența de pseudo-cod de mai sus?

6. (**1 punct**) La un moment dat un proces accesează o adresă de memorie, fără a rezulta page fault. După un timp, accesează din nou acea adresă și rezultă page fault. Știind că între cele două accese descrise nu au existat alte accese la pagina aferentă, explicați de ce s-a produs acel page fault.

7. (**1 punct**) Creați un paragraf adevărat, informativ și argumentat din domeniul sistemelor de operare care să cuprindă ca subiecte principale noțiunile de *handler de semnal* și *mutex*.

Nu este recomandat să se folosească mutex-uri într-un handler de semnal. Dacă se face lock pe mutex și, în programul principal, s-a făcut de asemenea, lock pe mutex, există riscul unui deadlock; handler-u de semnal întrerupe programul principal în timp ce acesta încă are ocupat mutex-ul.

8. (**1 punct**) Creați un paragraf adevărat, informativ și argumentat din domeniul sistemelor de operare care să cuprindă ca subiecte principale noțiunile de *pagină fizică (frame)* și *fișier mapat în memorie*.

Un fișier mapat în memorie ocupă pagini fizice (frame-uri) care sunt mapate apoi în spațiuil de adresă al procesului. Un fișier poate fi mapat de mai multe procese, caz în care paginile fizice (frame-urile) aferente pot fi partajate. Scrierile în spațiul de adresă vor ajunge în spațiul fizic și vor fi flushed doar la apeluri specifice sau la închiderea mapării.

9. (**1 punct**) Care sunt asocierile dintre secțiunile de memorie de mai jos și programe/executabile, procese, respectiv thread-uri? (unul la unul, unul la mai multe, etc.)

text, rodata, data, bss, heap, stack

10. (**1 punct**) Asociați conceptele de mai jos cu soluțiile de virtualizare VMware Workstation, KVM, LXC: kernel development, bare-metal virtualization, modul de kernel, native virtualization, full virtualization, containers, disk image file.

Nume și grupă:

11. (**2.5 puncte**) Un programator implementează o bază de date care va stoca obiecte de dimensiune fixă de 1MB. Fiecare obiect are un identificator numeric unic (ID), și este salvat pe disc (HDD) ca un fișier separat, folosind sistemul de fișiere ext2. Numărul total de obiecte poate fi foarte mare, fiind limitat doar de dimensiunea HDD-ului (e.g. 10TB 10 milioane fișiere).

Baza de date trebuie optimizată astfel încât să acceseze cât mai repede un fișier identificat cu un anumit ID.

Vi se cere să sugerați optimizări posibile pentru a atinge acest scop. Explicați care sunt factorii care limitează performanța, și argumentați optimizările alese. Desenați o schemă bloc a sistemului propus.

Pentru deschiderea unui fisier e nevoie sa localizam fisierul in dentry - costul este liniar in nr. de fisiere in director in ext2. Idee de baza: ca sa reducem timpul de acces, trebuie sa ne asiguram ca subiectele sunt stocate in directoare care au un numar redus de fisiere.

Pentru a implementa baza de date, avem nevoie de doua lucruri:

- o tabela de hash care mapeaza ID-ul fisierului in directorul care il contine. Aceasta tabela va fi stocata in memorie. Presupunand ca numarul de directoare este relativ mic, dimensiunea tabelei este data de nr de fisiere * (dim_id + pointer_nume_director) ~ 8 sau 16 octeti. 10 milioane de fisiere ar ocupa numai 160MB de RAM.
- o ierarhie de directoare in care fiecare director are un numar maxim de intrari X (prestabilit).

Se pot folosi multiple structuri ierarhice cu adancime prestabilita. Se creaza astfel mai multe directoare:

- un director "direct" care contine un numar de fisiere X
- un director "indirect" care contine X alte directoare fiecare cu X fisiere
- un director dublu indirect
- un director triplu indirect etc.

Se vor folosi directoarele "directe" dupa care cele indirecte, dublu-indirecte, etc.

Cum alegem X? Timpul de acces la un fisier depinde de X si de adancimea structurii de directoare, citirea unui director dureaza: X(logXN + 1). Stiind N (e.g. 10 milioane), se poate optimiza X alegand valoarea care minimizeaza formula de mai sus.

12. (**2.5 puncte**) Clienții Youtube se conectează la servere via TCP și transmit o cerere care conține numele fișierului dorit, offset-ul de început și dimensiunea dorită (tipic, câteva zeci de KB, pentru a evita download-ul conținutului în mod inutil, dacă utilizatorul închide clipul). Dacă serverul închide conexiunea, clientul se va reconecta atunci când are nevoie de următoarea secvență de clip și va relua download-ul. Altfel, clientul ține conexiunea deschisă și doar va emite o nouă cerere.

Vi se cere să implementați un server Youtube care să permită unui număr cât mai mare de utilizatori să privească clipuri simultan, astfel:

- desenați o schemă bloc cu sistemul propus;
- explicați cum va fi tratat fiecare client (proces nou / thread / etc.) și motivați alegerea făcută;

Nume și grupă:

- explicati ce API veți folosi pentru a citi / scrie din sockets TCP (nonblocking, event-based, blocking?) și de ce;
- atunci când crește numărul de clienți, ce resurse vor limita scalabilitatea sistemului pe care l-ați construit? (e.g. nr. de descriptori, nr. de procese/thread-uri, rețeaua)?

Ideea principala este de a trata o cerere de continut cat mai repede si de a inchide conexiunea – astfel numarul de clienti conectati simultan este mic si acest lucru reduce stresul asupra resurselor sistemtului (thread-uri, descriptori).

Se poate folosi un pool de thread-uri; atunci cand vine o cerere noua se aloca cererea unuia din thread-urile din pool. Folosirea pool-ului de thread-uri minimizeaza costurile de startup, si reduc costurile de switching (no tlb flush), insa toti clientii vor folosi acelasi spatiu de adresa – totusi potentialele probleme de securitate par minore (read-only video data).

Se va folosi blocking API pt. citire din sockets – e cea mai usor de folosit. Non-blocking nu prea are sens cu thread-uri. Event based la fel. Cel mai probabil reteaua va deveni un bottleneck. Din cauza ca folosim un pool de thread-uri nr de thread-uri nu e o problema, si nici cel de descriptori (presupunand ca nu se executa accept atunci cand nu se poate repartiza cererea clientului unui thread).

În conformitate cu ghidul de etică al Departamentului de Calculatoare, declar că nu am copiat și nu voi copia la această lucrare. De asemenea, nu am ajutat și nu voi ajuta pe nimeni să copieze la această lucrare.

Nume și grupă:

Semnătură:.....

Sisteme de Operare

27 mai 2013

Timp de lucru: 60 de minute

Notă: Toate răspunsurile trebuie justificate

1. Comanda "ulimit -s unlimited" stabilește ca dimensiunea stivei să fie maximă posibilă. De ce această comandă poate fi folosită pentru a înlesni atacuri de tip return-to-libc pe sisteme cu suport de ASLR?

Cu cat stiva este mai mare cu atat posibilitatile de pozitionare a acesteia in spatiul de adresa scad. Astfel, se poate face brute force mai usor.

2. Care este asocierea dintre noțiunea de spațiu de adresă și proces? Dar între spațiu de adresă și thread-uri?

Fiecare proces are propriul spatiu de adresa -->(spatiu de adresa, proces) - unu la unu
Threadurile unui proces partajeaza acelasi spatiu de adresa (spatiu de adresa, thread) - unul la mai multe

Nume și grupă:

3. Folosind mecanismul de memorie virtuală, se pot partaja zone din spațiul de adresă din user space al unui proces cu zone din spațiul kernel. Dați exemplu de situație în care o astfel de abordare este utilă.

A processor in a computer running Windows has two different modes: user mode and kernel mode. The processor switches between the two modes depending on what type of code is running on the processor. Applications run in user mode, and core operating system components run in kernel mode. Many drivers run in kernel mode, but some drivers run in user mode.

When you start a user-mode application, Windows creates a process for the application. The process provides the application with a private virtual address space and a private handle table. Because an application's virtual address space is private, one application cannot alter data that belongs to another application. Each application runs in isolation, and if an application crashes, the crash is limited to that one application. Other applications and the operating system are not affected by the crash.

In addition to being private, the virtual address space of a user-mode application is limited. A processor running in user mode cannot access virtual addresses that are reserved for the operating system. Limiting the virtual address space of a user-mode application prevents the application from altering, and possibly damaging, critical operating system data.

All code that runs in kernel mode shares a single virtual address space. This means that a kernel-mode driver is not isolated from other drivers and the operating system itself. If a kernel-mode driver accidentally writes to the wrong virtual address, data that belongs to the operating system or another driver could be compromised. If a kernel-mode driver crashes, the entire operating system crashes.

This diagram illustrates communication between user-mode and kernel-mode components.

4. De ce numărul de intrări în TLB este de ordinul sutelor? De ce nu se alocă dimensiuni mai mari pentru TLB (număr de intrări de ordinul miilor sau zecilor de mii)?

I'd be interested to experiment with different TLB sizes, to see what effect

that has on performance. But I suspect that lack of TLB contexts mean that we

wind up flushing the TLB more often than real hardware does, and therefore a

larger TLB merely takes longer to flush.

Hardware TLBs are limited in size primarily due to the fact that increasing their sizes increases their access latency as well. but software tlb does not

suffer from that problem. so i think the size of the softtlb should be not influenced by the size of the hardware tlb.

Nume și grupă:

Flushing the TLB is minimal unless we have a really really large TLB, e.g. a TLB with 1M entries. I vaguely remember that i see ~8% of the time is spent in

the cpu_x86_mmu_fault function in one of the speccpu2006 workload some time ago. so if we increase the size of the TLB significantly and potential getting

rid of most of the TLB misses, we can get rid of most of the 8%. (there are still compulsory misses and a few conflict misses, but i think compulsory

misses is not the major player here).

5. De ce se preferă folosirea spinlock-urilor pentru regiuni critice de dimensiuni mici iar mutex-urile pentru regiuni critice de dimensiuni mari?
Deoarece spinlock-ul foloseste busy-waiting este indicat sa fie folosit pe portiuni mici unde timpul de asteptare este mic si unde folosirea unui mutex ar aduce un overhead mult prea mare. Mutex-urile sunt folosite pe portiuni critice mari deoarece costul unui busy waiting pe un timp lung e mult mai mare decat asteptarea intr-o coada pana la un unlock pe mutex.

6. Care este legătura și diferența dintre noțiunile de "drepturi de creare a unui fișier" și "drepturi de deschidere a unui fișier"?

7. Care este un avantaj al fiecăreia dintre formele de virtualizare: full virtualization și paravirtualization?
Full virtualization:
*AVANTAJE
One of the most common reasons for implementing a full virtualization solution is for operational efficiency. It allows organizations to use existing hardware more efficiently by placing a greater load on each computer. This means that servers using full virtualization can use more of the computer's processing and memory resources than servers running a single OS instance and a single set of services.
Another reason to use full virtualization is to facilitate desktop virtualization, in which a single PC runs more than one OS instance. There are a number of reasons to do so. This can offer support for applications that only run on a particular OS. It can also allow changes to be made to an OS and later on, revert to the original if necessary. Desktop virtualization has also proven to support better control of OSs, in order to ensure that they meet basic security requirements
*DEZAVANTAJE

- Adds layers of technology, which increase the security management burden as it requires additional security controls.
- Combining a number of systems onto a single physical computer causes a larger impact, should a security compromise occur.
- It's relatively easy to share information between virtualization systems, which can facilitate attack vectors, if not carefully controlled or regulated.
- The dynamic aspect of virtualized environments renders creating and maintaining the necessary security boundaries more complex.

.PARAVIRTUALIZATION

Nume și grupă:

*AVANTAJE

Performance is the most well known advantage that paravirtualization has, however with paravirtualized device drivers in a fully virtualized OS this advantage is actually getting smaller over time.

However, compared to traditional full virtualization, where the virtualization software emulates a complete computer and a completely unmodified guest operating system is run, paravirtualization has very significant performance advantages.

8. De ce un proces care reprezintă un server web are prioritate mai mare decât un proces care este folosit pentru înmulțiri de matrice?

CPU INTENSIVE VS I/O

9. Procesul P folosește thread-uri hibride, având 9 thread-uri user-level, mapate pe 3 thread-uri kernel-level (câte trei user-level threads pe un kernel-level thread). Fie thread-ul user-level TU1 mapat pe thread-ul kernel-level TK1. În ce mod vor fi afectate celelalte thread-uri mapate pe TK1, în cazul în care TU1 realizează un acces nevalid la memorie? Dar thread-urile mapate pe celelalte thread-uri kernel?

10. De ce este necesară operația de "Safe Remove" a dispozitivelor USB, în urma copierii unor fișiere noi pe acestea?

Obviously, yanking out a drive while it's being written to could corrupt the data. However, even if the drive isn't *actively* being written to, you could still corrupt the data. By default, most operating systems use what's called *write caching* to get better performance out of your computer. When you write a file to another drive—like a flash drive—the OS waits to actually perform those actions until it has a number of requests to fulfill, and then it fulfills them all at once (this is more common when writing small files). When you hit that eject button, it tells your OS to flush the cache—that is, make sure all pending actions have been performed—so you can safely unplug the drive without any data corruption.

În conformitate cu ghidul de etică al Catedrei de Calculatoare, declar că nu am copiat și nu voi copia la această lucrare. De asemenea, nu am ajutat și nu voi ajuta pe nimeni să copieze la această lucrare.

Nume și grupă:

Sisteme de Operare

24 mai 2014

Timp de lucru: 60 de minute

Notă: Toate răspunsurile trebuie justificate

1. (**7 puncte**) Precizați o diferență între un thread și un proces.

Procesele au propriul spatiu de adresa pe cand threadurile unui proces sharuiesc acelasi spatiu de adresa.

Processes are the abstraction of running programs: A binary image, virtualized memory, various kernel resources, an associated security context, and so on. **Threads are the unit of execution in a process**: A virtualized processor, a stack, and program state. Put another way, processes are running binaries and threads are the smallest unit of execution schedulable by an operating system's process scheduler.

A process *contains* one or more threads.

Both processes and threads are independent sequences of execution. The typical difference is that threads (of the same process) run in a shared memory space, while processes run in separate memory spaces.

2. (**7 puncte**) Care este legătura între memorie virtuală și spațiu de swap?

Memoria virtuală este memoria pe care o vede un proces care rulează în sistem. Pentru că în RAM nu încape întreg spațiul adresat de memoria virtuală (~4GB pentru procesoarele pe 32 de biți) extindem memoria cu un cache pe hard disk, pe care îl numim swap.

Virtual memory is a combination of RAM and disk space that running processes can use.

Swap space is the portion of virtual memory that is on the hard disk, used when RAM is full.

3. (**7 puncte**) Dați exemplu de dispozitiv de tip caracter. De ce este acesta un dispozitiv de tip caracter?

Tastatura este un dispozitiv de tip caracter pentru ca trasferul de date se realizeaza caracter cu caracter.

A character device is any device that can have streams of characters read from or written to it. A character device has a character device driver associated with it that can be used for a device such as a line printer that handles one character at a time. However, character drivers are not limited to performing I/O a single character at a time (despite the name ``character'' driver). For example, tape drivers frequently perform I/O in 10K chunks. A character device driver can also be used where it is necessary to copy data directly to or from a user process. Because of their flexibility in handling I/O, many drivers are character drivers. Line printers, interactive terminals, and graphics displays are examples of devices that require character device drivers.

(si despre block character, sa-l avem acolo)

exemplu: hard disk

A device, such as a magnetic tape drive or disk drive, that conveys data in blocks through the buffer management code.

A block device is one that is designed to operate in terms of the block I/O supported by Digital UNIX. It is accessed through the buffer cache. A block device has an associated block device driver that performs I/O by using file system block-sized buffers from a buffer cache supplied by the kernel. Block device drivers are particularly well-suited for disk drives, the most common block devices.

4. (**7 puncte**) Descrieți, în pseudocod sau literal, un scenariu în care are loc un atac de tipul buffer overflow. Precizați în ce condiții se realizează acest atac.

exemplU, avem urmatorul program:

```
#include <stdio.h>

#include <string.h>

int main(void) {

char buff[15];

 int pass = 0;

 printf("\n Enter the password : \n");

gets(buff);

if(strcmp(buff, "thegeekstuff"))     {

 printf ("\n Wrong Password \n");

 }     else     {

 printf ("\n Correct Password \n");

  pass = 1;     }


if(pass)     {        /* Now Give root or admin rights to user*/         printf ("\n
Root privileges given to the user \n");     }     return 0; }
```

Daca rulam programul cu parola: thegeekstuff, se produce ceea ce ne asteptam, si primim drepturi de root.

Acest program insa are posibilitatea de a produce buffer overflow. Functia gets() nu verifica marginile unde scrie si poate scrie string-uri de lungime mai mare decat marimea celui in care scrie el de fapt. Asadar, daca atacatorul da o parola mai lunga decat marimea buffer-ului, acesta poate sa ajunga sa scrie peste zona de memorie a variabilei "pass", devenind ceva diferit de 0 => drepturi de root pentru atacator.

5. (**7 puncte**) Un fișier are două link-uri hard (`a.txt` și `b.txt`). Ce se întâmplă dacă ștergem unul dintre link-uri (rm `a.txt`)?

When you create a second, third, fourth, etc link, the counter is incremented (increased) each

time by one. When you delete (`rm`) a link the counter is decremented (reduced) by one. If the link counter reaches 0 the filesystem removes the `inode` and marks the space as available for use.

In short, as long as you do not delete the last link the file will remain.

`Edit:` The file will remain *even if the last link is removed*. This is one of the ways to ensure security of data contained in a file is not accessible to any other process. Removing the data from the filesystem completely is done only if the data has 0 links to it as given in its metadata and is not being used by any process.

raspunzi cam la orice intrebare soft links vs hard links:
Underneath the file system files are represented by inodes (or is it multiple inodes not sure)

A file in the file system is basically a link to an inode.
A hard link then just creates another file with a link to the same underlying inode.

When you delete a file it removes one link to the underlying inode. The inode is only deleted (or deletable/over-writable) when all links to the inode have been deleted.

A symbolic link is a link to another name in the file system.

Once a hard link has been made the link is to the inode. deleting renaming or moving the original file will not affect the hard link as it links to the underlying inode. Any changes to the data on the inode is reflected in all files that refer to that inode.

Note: Hard links are only valid within the same File System. Symbolic links can span file systems as they are simply the name of another file.

6. (**10 puncte**) De ce este necesară prezența bitului setuid (suid) pe executabilul /usr/bin/passwd?

Pentru a executa passwd user-ul nu are neaparat nevoie de privilegii de root (ex: vrea sa isi schimbe parola). Cu toate astea, procesul are nevoie de privilegii pentru a modifica fisierul /etc/passwd. Aici intervine bitul setuid care seteaza effective user id-ul la owner-ul executabilului - root.

7. (**10 puncte**) Într-un sistem rulează la un moment dat 100 de procese. Câte tabele de pagini sunt alocate? Justificați.

By giving each process its own page table, every process can pretend that it has access to the entire address space available from the processor. It doesn't matter that two processes might use the same address, since different page-tables for each process will map it to a different frame of physical memory. Every modern operating system provides each process with its own address space like this.

Over time, physical memory becomes fragmented, meaning that there are "holes" of free space in the physical memory. Having to work around these holes would be at best annoying and would become a serious limit to programmers. For example, if you `malloc` 8 KiB of memory; requiring

the backing of two 4 KiB frames, it would be a huge inconvience if those frames had to be contiguous (i.e., physically next to each other). Using virutal-addresses it does not matter; as far as the process is concerned it has 8 KiB of contiguous memory, even if those pages are backed by frames very far apart. By assigning a virtual address space to each process the programmer can leave working around fragmentation up to the operating system.

8. (**10 puncte**) În urma a două apeluri accept() în codul unui server sunt creați doi socketi care au aceeași adresă IP și port. Cum diferențiază sistemul de operare socketul căruia îi va fi livrat un pachet dat?

Socketii sunt identificati printre altele de urmatoarele atribute: ip sursa, port sursa, ip destinatie, port destinatie. In cazul de fata, desi doua dintre atribute vor fi identice pentru ambii socketi celelalte doua vor putea identifica peer-ul asociat (port sursa, ip sursa).

9. (**10 punct**) De ce este de preferat folosira unei cuante de timp mai mari pentru planificatorul de procese al unui sistem de tip **server**, și a unei cuante de timp mai mici pentru planificatorul de procese al unui sistem de tip **laptop**?

10. (**10 puncte**) Apelul lseek() actualizează cursorul de fișier. De ce această actualizare nu produce nici o modificare a inode-ului fișierului?

If it were associated with the inode, then you would not be able to have multiple processes accessing a file in a sensible manner, since all accesses to that file by one process would affect other processes.

Thus, a single process could have track many different file positions as it has file descriptors for a given file.

Per the lseek docs, the file position is associated with the open file pointed to by a file descriptor, i.e. the thing that is handed to your by open. Because of functions like dup and fork, multiple*descriptors* can point to a single *description*, but it's the description that holds the location cursor.

File position is associated with an *open file description*, not a file descriptor. Many different file descriptors can refer to the same open file description, due to fork, dup, etc.

11. (**15 puncte**) Aveți la dispoziție un sistem cu mai multe core-uri și vreți să dezvoltați o bibliotecă de video transcoding (CPU-bound) pentru stream-uri video dintr-un fișier. Presupunem că aveți detaliile unui algoritm de transcoding paralel. Acest algoritm permite efectuarea operației de transcoding separat pe blocuri diferite din stream (transcode_block()). Este însă nevoie, la finalul trascodingului unui bloc de o operație de unificare la final pentru două blocuri adiacente (merge_adjacent_stream_blocks()); în această parte de unificare se „lipesc", respectiv, părțile de început și sfârșit ale celor două blocuri. Blocurile sunt citite și scrise dintr-un/într-un fișier. Dimensiunea blocului este prestabilită.

Care vor fi principiile de proiectare a bibliotecii? Veți folosi thread-uri sau procese? Câte? Ce mecanisme de comunicare și sincronizare veți folosi în cadrul bibliotecii? Care sunt factorii de overhead din implementare?

În conformitate cu ghidul de etică al Departamentului de Calculatoare, declar că nu am copiat și nu voi copia la această lucrare. De asemenea, nu am ajutat și nu voi ajuta pe nimeni să copieze la această lucrare.

Nume și grupă:

Semnătură:.....

Sisteme de Operare

3 iunie 2014

Timp de lucru: 60 de minute

Notă: Toate răspunsurile trebuie justificate

1. (**7 puncte**) Precizați două entități diferite pe care le poate referi un descriptor de fișier în Linux.

Entati diferite ce le poate referi: standard input/output/error, fisier, pipe, resursa input/output

if there are 100 files opened in your OS then there will be 100 entries in OS (somewhere in kernel). These entries are represented by integers like (...100, 101, 102....). This entry number is the file descriptor. So it is just an integer number that uniquely represents an opened file in operating system. If your process opens 10 files then your Process table will have 10 entries for file descriptors.
Similarly when you open a network socket, it is also represented by an integer and it is called Socket Descriptor.

2. (**7 puncte**) În urma unui buffer overflow adresa de retur este suprascrisă. Către ce zonă poate pointa adresa suprascrisă pentru a genera un atac, în cazul în care sistemul are DEP (*Data Execution Prevention*)? Alegeți dintre *text*, *stack*, *data*. Justificați.

3. (**7 puncte**) De ce este utilă paginarea ierarhică?

The inverted page table keeps a listing of mappings installed for all frames in physical memory. However, this could be quite wasteful. Instead of doing so, we could create a page table structure that contains mappings for virtual pages. It is done by keeping several page tables that cover a certain block of virtual memory. For example, we can create smaller 1024-entry 4K pages that cover 4M of virtual memory.

This is useful since often the top-most parts and bottom-most parts of virtual memory are used in running a process - the top is often used for text and data segments while the bottom for stack, with free memory in between. The multilevel page table may keep a few of the smaller page tables to cover just the top and bottom parts of memory and create new ones only when strictly necessary.

Now, each of these smaller page tables are linked together by a master page table, effectively creating a tree data structure. There need not be only two levels, but possibly multiple ones.

A virtual address in this schema could be split into three parts: the index in the root page table, the index in the sub-page table, and the offset in that page.

Multilevel page tables are also referred to as hierarchical page tables.

Alt raspuns:

You will appreciate the space optimization of multi-level page tables when we go into the 64-bit address space.

Assume you have a 64-bit computer (which means **64 bit virtual address space**), which has **4KB pages** and **4 GB** of physical memory. If we have a single level page table as you suggest, then it should contain one entry per virtual page per process.

One entry per virtual page – 2^{64} addressable bytes / 2^{12} bytes per page = **2^{52} page table entries**

One page table entry contains: Access control bits (Bits like Page present, RW etc) + Physical page number

4 GB of Physical Memory = **2^{32} bytes**.

2^{32} bytes of memory/2^{12} bytes per page = **2^{20} physical pages**

20 bits required for physical page number.

So each page table entry is approx **4 bytes**. (20 bits physical page number is approx 3 bytes and access control contributes 1 byte)

Now, Page table Size = 2^{52} page table entries * 4 bytes = **2^{54} bytes** (**16 petabytes**) !

16 petabytes per process is a very very huge amount of memory.

Now, if we page the pagetable too, ie if we use multi level page tables we can magically bring down the memory required to as low a single page. ie **just 4 KB**.

Now, we shall calculate how many levels are required to squeeze the page table into just 4 KB. 4 KB page / 4 bytes per page table entry = **1024 entries**. **10 bits of address space required.** i.e **52/10**ceiled is 6. ie **6 levels of page table can bring down the page table size to just 4KB.**

6 level accesses are definitely slower. But I wanted to illustrate the space savings out of multi level page tables.

4. (**7 puncte**) Cu ce diferă apelul **fork()** din Linux față de apelul CreateProcess() din Windows?

In Windows when you call CreateProcess() it does just that. It creates a new "clean" process ready to run what you say.

In Linux fork() clones the process's page table and sets the child process's pages to copy-on-write--so no, there is no measurable "waste of time and resources". And if you're *really* that concerned about performance, you can use vfork() instead, which doesn't even bother to clone the page table. In fact, if you'd done your homework at all you would know that spawning new processes on Linux is quite significantly *faster*

than on Windows.

5. (**7 puncte**) Când folosim mutex-uri în loc de operații atomice pentru asigurarea accesului serial la date?

6. (**10 puncte**) Fie instrucțiunea:

a = b;

În ce situație instrucțiunea generează două page fault-uri fără a conduce la terminarea procesului curent?

DEFINITIE:

A page fault is a trap to the software raised by the hardware when a program accesses a page that is mapped in the virtual address space, but not loaded in physical memory.

That's not entirely correct, as explained later in the same article (*Minor page fault*). There are soft page faults, where all the kernel needs to do is add a page to the working set of the process. Here's a table from the Windows Internals book (I've excluded the ones that result in an access violation):

- Reason for Fault - *Result*
- Accessing a page that isn't resident in memory but is on disk in a page file or a mapped file -*Allocate a physical page, and read the desired page from disk and into the relevant working set*
- Accessing a page that is on the standby or modified list - *Transition the page to the relevant process, session, or system working set*
- Accessing a demand-zero page - *Add a zero-filled page to the relevant working set*
- Writing to a copy-on-write page - *Make process-private (or session-private) copy of page, and replace original in process or system working set*

Page faults can occur for a variety of reasons, as you can see above. Only one of them has to do with reading from the disk. If you try to allocate a block from the heap and the heap manager allocates new pages, then accesses those pages, you'll get a demand-zero page fault. If you try to hook a function in kernel32 by writing to kernel32's pages, you'll get a copy-on-write fault because those pages are silently being copied so your changes don't affect other processes.

Now to answer your question more specifically: Process Hacker only seems to have page faults when updating its service information - that is, when it calls EnumServicesStatusEx, which RPCs to the SCM (services.exe). My guess is that in the process, a lot of memory is being allocated, leading to demand-zero page faults (the service information requires several pages to store, IIRC).

Several reasons:

1. Processes are created by memory mapping the code sections from the file. Reading sections of a memory mapped file that aren't yet in memory causes page faults, so each process will at least have the page faults of reading in its own executable and any DLLs whose code wasn't yet in memory. Other memory mapped files used by a process will also cause page faults.
2. When a process requests memory with VirtualAlloc, no physical frames are actually committed to the process until the allocated pages are 'touched' for the first time. This also causes page faults.
3. Even when memory is not full, Windows will trim infrequently used pages from the process' working set and lazily page them out to disk. This enables Windows to better respond to sudden demands for

large amounts of memory. When a process attempts to access such a page, it causes a page fault. In situations where memory consumption is low enough, the page will probably still be in memory so no disk read is necessary, but a page fault is still triggered. This is called a soft page fault.

7. (**10 puncte**) În ce situație folosiți apeluri de tip read/write în loc de maparea fișierului în memorie?

It really depends on what you're trying to do. If all you need to do is hop to a known offset and read out a small tag, `read()` may be faster (`mmap()` has to do some rather complex internal accounting). If you are planning on copying out all 200mb of the MP3, however, or scanning it for some tag that may appear at an unknown offset, then `mmap()` is likely a faster approach.

`read()` on the other hand involves an extra memory-to-memory copy, and can thus be inefficient for large I/O operations, but is simple, and so the fixed overhead is relatively low. In short, use `mmap()` for large bulk I/O, and `read()` or `pread()` for one-off, small I/Os.

8. (**10 puncte**) În cadrul unei conexiuni TCP, transmițătorul realizează apelul:

```
send(s, send_buffer, 5000, 0);      /* send 5000 bytes */
```

iar receptorul realizează apelul

```
recv(s, recv_buffer, 7000, 0);    /* receive 7000 bytes */
```

Câți octeți va primi receptorul (la întoarcerea din apelul recv)?

Receptorul poate primi oricati octeti intre 1 si 5000, presupunand ca niciun alt send nu s-a realizat sau se va realiza din partea tansmitatorului. In functie de congesia retelei, a incarcarii bufferelor din kernel si cele de pe placa de retea aceasta valoare variaza intre 1 si 5000. Daca transmitatorul inchide conexiunea inainte ca datele sa fie transmise, apelul recv se va intoarce cu -1.

9. (**10 punct**) Un thread folosește malloc pentru a aloca memorie. Precizați un set de pași (în pseudocod) în care alt thread al aceluiași proces accesează zona de memorie alocată de primul thread.

- malloc() and free() are not thread-safe functions. You need to protect the calls to those functions with a mutex.
- You need to protect all shared variables with a mutex as well. You can use the same one as you use for malloc/free, one per variable.
- You need to declare variables shared between several threads as `volatile`, to prevent dangerous optimizer bugs on some compilers. Note that this is no replacement for mutex guards.
- Are the buffers arrays, or two-dimensional arrays (like arrays of C strings)? You have declared all buffers as potential two-dimensional arrays, but you never allocate the inner-most dimension.
- Never typecast the result of malloc in C. Read this and this.
- `free(bufferaction)`, not `free(&bufferaction)`.
- Initialize all pointers to NULL explicitly. After free(), make sure to set the pointer to NULL. Before the memory is accessed by either thread, make sure to check the pointer against NULL.

10. (**10 puncte**) Un sistem de fișiere ext2 poate folosi fișiere cu dimensiune de până la 16GB. Ce limitează dimensiunea maximă a fișierelor?

There are various limits imposed by the on-disk layout of ext2. Other limits are imposed by the current

implementation of the kernel code. Many of the limits are determined at the time the filesystem is first created, and depend upon the block size chosen. The ratio of inodes to data blocks is fixed at filesystem creation time, so the only way to increase the number of inodes is to increase the size of the filesystem.

The 2TiB file size is limited by the i_blocks value in the inode which indicates the number of 512-bytes sector rather than the actual number of ext2 blocks allocated.

This limit was also overcome ages ago by the use of a flag in the inode that indicates that the i_blocks value is, in fact, in units of block size rather than 512 bytes. The triple indirect block structure though, can only address just over 4 TiB using a 4k block size.

11. (**15 puncte**) Dorim să implementăm un proxy server pentru conexiuni web. Un proxy server servește cereri din cache-ul local dacă paginile web se găsesc în cache; altfel face cereri către serverul web destinație, obține pagina și apoi o cache-uiește. Facem următoarele presupuneri:

- Paginile cerute sunt, în mare parte, de dimensiuni mici (ordinul kiloocteților).
- Paginile se modifică greu; nu este nevoie să vă gândiți la expirarea paginilor în cache.

- Se poate folosi pentru caching atât memorie cât și spațiu pe disc, ambele limitate.
- Există un număr mare de cereri pe secundă pe care le primește proxy serverul.

Ce tehnologii veți folosi în proiectarea proxy serverului? (operații asincrone, multiplexare, multithreading, multiproces, etc.). Justificați alegerea.
Ce politică de înlocuire a paginilor în cache veți folosi?
Ce pagini veți plasa în cache-ul de memorie și ce pagini veți plasa în cache-ul de pe disc?
Cum veți asigura accesul sincronizat/consecvent/coerent la datele din cache?

În conformitate cu ghidul de etică al Departamentului de Calculatoare, declar că nu am copiat și nu voi copia la această lucrare. De asemenea, nu am ajutat și nu voi ajuta pe nimeni să copieze la această lucrare.

Nume și grupă:

Semnătură:.....

Sisteme de Operare

5 iunie 2014

Timp de lucru: 60 de minute

Notă: Toate răspunsurile trebuie justificate

1. (**7 puncte**) Când are loc un flush de TLB (*Translation Lookaside Buffer*)?
Are loc la orice schimbare asupra structurii paginilor.

If another processor could also be affected by a page table write (because of shared memory, or multiple threads from the same process), you must also flush the TLBs on those 1processors.

2. (**7 puncte**) Ce conțin blocurile de date aferente unui director?

3. (**7 puncte**) De ce considerăm tastatura un dispozitiv de tip caracter?
a mai fost

4. (**7 puncte**) De ce este timpul de creare a unui thread al aceluiași proces mai mic decât timpul de creare a unui proces?

- Less time to create a new thread than a process, because the newly created thread uses the current process address space.

5. (**7 puncte**) Câte tabele de descriptori de fișiere are un proces cu 10 thread-uri?
una!
there is only one file descriptor table per process, and it's shared among all the threads.

The file descriptors are shared between the threads. If you want "thread specific" offsets, why not have each thread use a different file descriptor (open(2) multiple times) ?

6. (**10 puncte**) Care este o legătură între planificatorul de procese și sistemul de întreruperi?
Sistemul de intreruperi furnizeaza intreruperea de ceas, ce determina planificatorul sa ia decizii privind procesul ce va rula in urmatoarea cuanta de timp.

7. (**10 puncte**) În cadrul unei conexiuni TCP un client trimite către un server mesaje într-o buclă:

```
while (1) {
        send(s, buffer, 8192, 0);   /* send 8192 bytes */
}
```

La un moment dat, apelul send se blochează (pentru o perioadă de timp). Care este o cauză posibilă pentru această blocare?
Toate bufferele intre transmitator si destinatar s-au umplut ori reteaua este congestionata. Prin buffere ma refer la atat la bufferele din kernel cat si la cele de pe placile de retea.

8. (**10 puncte**) Descrieți o situație în care un buffer overflow pe un array aflat în zona de date globale conduce la un exploit.
The principle of exploiting a buffer overflow is to overwrite parts of memory which aren't supposed to be overwritten by arbitrary input and making the process execute this code. To see how and where an overflow takes place, lets

take a look at how memory is organized. A page is a part of memory that uses its own relative addressing, meaning the kernel allocates initial memory for the process, which it can then access without having to know where the memory is physically located in RAM. The processes memory consists of three sections:

- code segment, data in this segment are assembler instructions that the processor executes. The code execution is non-linear, it can skip code, jump, and call functions on certain conditions. Therefore, we have a pointer called EIP, or instruction pointer. The address where EIP points to always contains the code that will be executed next.

- data segment, space for variables and dynamic buffers

- stack segment, which is used to pass data (arguments) to functions and as a space for variables of functions. The bottom (start) of the stack usually resides at the very end of the virtual memory of a page, and grows down. The assembler command PUSHL will add to the top of the stack, and POPL will remove one item from the top of the stack and put it in a register. For accessing the stack memory directly, there is the stack pointer ESP that points at the top (lowest memory address) of the stack.

'Lets assume that we exploit a function like this:

```
void lame (void) { char small[30]; gets (small); printf("%s\n", small); }
main() { lame (); return 0; } Compile and disassemble it: # cc -ggdb blah.c
-o blah /tmp/cca017401.o: In function `lame': /root/blah.c:1: the `gets'
function is dangerous and should not be used. # gdb blah /* short
explanation: gdb, the GNU debugger is used here to read the binary file and
disassemble it (translate bytes to assembler code) */ (gdb) disas main Dump
of assembler code for function main: 0x80484c8 : pushl %ebp 0x80484c9 :
movl %esp,%ebp 0x80484cb : call 0x80484a0 0x80484d0 : leave 0x80484d1 : ret
(gdb) disas lame Dump of assembler code for function lame: /* saving the
frame pointer onto the stack right before the ret address */ 0x80484a0 :
pushl %ebp 0x80484a1 : movl %esp,%ebp /* enlarge the stack by 0x20 or 32.
our buffer is 30 characters, but the memory is allocated 4byte-wise
(because the processor uses 32bit words) this is the equivalent to: char
small[30]; */ 0x80484a3 : subl $0x20,%esp /* load a pointer to small[30]
(the space on the stack, which is located at virtual address
0xffffffe0(%ebp)) on the stack, and call the gets function: gets(small); */
0x80484a6 : leal 0xffffffe0(%ebp),%eax 0x80484a9 : pushl %eax 0x80484aa :
call 0x80483ec 0x80484af : addl $0x4,%esp /* load the address of small and
the address of "%s\n" string on stack and call the print function:
printf("%s\n", small); */ 0x80484b2 : leal 0xffffffe0(%ebp),%eax 0x80484b5
: pushl %eax 0x80484b6 : pushl $0x804852c 0x80484bb : call 0x80483dc
0x80484c0 : addl $0x8,%esp /* get the return address, 0x80484d0, from stack
and return to that address. you don't see that explicitly here because it
is done by the CPU as 'ret' */ 0x80484c3 : leave 0x80484c4 : ret End of
```

```
assembler dump. 3a. Overflowing the program # ./blah
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx <- user input xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
# ./blah xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx <- user input
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx Segmentation fault (core dumped) # gdb
blah core (gdb) info registers eax: 0x24 36 ecx: 0x804852f 134513967 edx:
0x1 1 ebx: 0x11a3c8 1156040 esp: 0xbffffdb8 -1073742408 ebp: 0x787878
7895160
```

EBP is 0x787878, this means that we have written more data on the stack than the input buffer could handle. 0x78 is the hex representation of 'x'. The process had a buffer of 32 bytes maximum size. We have written more data into memory than allocated for user input and therefore overwritten EBP and the return address with 'xxxx', and the process tried to resume execution at address 0x787878, which caused it to get a segmentation fault.

9. (**10 punct**) Fie secvența de instrucțiuni de mai jos:

```
printf("%d", *a);
*a = 42;
```

unde a este un pointer la un întreg (`int *`). Dați exemplu de situație în care prima instrucțiune (`printf`) NU cauzează page fault, dar a doua (`*a = 42`) cauzează page fault.

10. (**10 puncte**) Un set de thread-uri lucrează cu o structură de tip listă dublu înlănțuită. Unele thread-uri modifică lista (adaugă, șterg elemente), altele doar parcurg lista. De ce trebuie asigurat accesul exclusiv la listă pentru ambele tipuri de thread-uri, nu doar pentru cele care modifică lista?

11. (**15 puncte**) Dorim să implementăm o bibliotecă de tip engine de baze de date. Această bibliotecă va oferi un API de adăugare, ștergere, inserare, modificare elemente în baza de date și va realiza și stocarea fiecărei baze de date într-un fișier pe disc. Un program care va folosi biblioteca va putea să stocheze informații într-o bază de date dintr-un fișier pe disc într-un format intern. Biblioteca trebuie să fie thread safe. Trebuie ca operațiile executate în thread-uri diferite ale procesului să mențină datele coerente.

Definiți schematic API-ul pe care îl va expune biblioteca: structuri de date și funcționalități expuse ca interfață pentru programul ce va folosi biblioteca. Gândiți-vă doar la interfața expusă nu la internele implementării.

Cum veți asigura în cadrul implementării bibliotecii, în mod eficient, partea de thread safety?

Cum propuneți să asigurați o viteză bună de lucru cu fișierul de bază de date și în același timp să oferiți o asigurare cât mai bună că datele ajung pe disc?

Cum veți implementa partea de tranzacție? Adică un set de operații să fie executate atomic în cadrul bibliotecii.

În conformitate cu ghidul de etică al Departamentului de Calculatoare, declar că nu am copiat și nu voi copia la această lucrare. De asemenea, nu am ajutat și nu voi ajuta pe nimeni să copieze la această lucrare.

Nume și grupă:

Semnătură:.....

Sisteme de Operare

8 iunie 2014

Timp de lucru: 60 de minute

Notă: Toate răspunsurile trebuie justificate

1. (**7 puncte**) Un apel mmap() rezervă 16 pagini de memorie virtuală. Câte pagini de memorie fizică alocă apelul?

2. (**7 puncte**) De ce este dezavantajoasă folosirea unei cuante de timp prea mari pentru planificarea proceselor?

3. (**7 puncte**) Cu ce diferă un spinlock de un mutex?

Mutex Vs Spinlock

Criteria	Mutex	Spinlock
Mechanism	Test for lock. If available, use the resource If not, go to wait queue	Test for lock. If available, use the resource. If not, loop again and test the lock till you get the lock
When to use	Used when putting process is not harmful like user space programs. Use when there will be considerable time before process gets the lock.	Used when process should not be put to sleep like Interrupt service routines. Use when lock will be granted in reasonably short time.
Drawbacks	Incur process context switch and scheduling cost.	Processor is busy doing nothing till lock is granted, wasting CPU cycles.

The Theory

In theory, when a thread tries to lock a mutex and it does not succeed, because the mutex is already locked, it will go to sleep, immediately allowing another thread to run. It will continue to sleep until being woken up, which will be the case once the mutex is being unlocked by whatever thread was holding the lock before. When a thread tries to lock a spinlock and it does not succeed, it will continuously re-try locking it, until it finally succeeds; thus it will not allow another thread to take its place (however, the operating system will forcefully switch to another thread, once the CPU runtime quantum of the current thread has been exceeded, of course).

The Problem

The problem with mutexes is that putting threads to sleep and waking them up again are both rather expensive operations, they'll need quite a lot of CPU instructions and thus also take

some time. If now the mutex was only locked for a very short amount of time, the time spent in putting a thread to sleep and waking it up again might exceed the time the thread has actually slept by far and it might even exceed the time the thread would have wasted by constantly polling on a spinlock. On the other hand, polling on a spinlock will constantly waste CPU time and if the lock is held for a longer amount of time, this will waste a lot more CPU time and it would have been much better if the thread was sleeping instead.

The Solution

Using spinlocks on a single-core/single-CPU system makes usually no sense, since as long as the spinlock polling is blocking the only available CPU core, no other thread can run and since no other thread can run, the lock won't be unlocked either. IOW, a spinlock wastes only CPU time on those systems for no real benefit. If the thread was put to sleep instead, another thread could have ran at once, possibly unlocking the lock and then allowing the first thread to continue processing, once it woke up again.

On a multi-core/multi-CPU systems, with plenty of locks that are held for a very short amount of time only, the time wasted for constantly putting threads to sleep and waking them up again might decrease runtime performance noticeably. When using spinlocks instead, threads get the chance to take advantage of their full runtime quantum (always only blocking for a very short time period, but then immediately continue their work), leading to much higher processing throughput.

The Practice

Since very often programmers cannot know in advance if mutexes or spinlocks will be better (e.g. because the number of CPU cores of the target architecture is unknown), nor can operating systems know if a certain piece of code has been optimized for single-core or multi-core environments, most systems don't strictly distinguish between mutexes and spinlocks. In fact, most modern operating systems have hybrid mutexes and hybrid spinlocks. What does that actually mean?

A hybrid mutex behaves like a spinlock at first on a multi-core system. If a thread cannot lock the mutex, it won't be put to sleep immediately, since the mutex might get unlocked pretty soon, so instead the mutex will first behave exactly like a spinlock. Only if the lock has still not been obtained after a certain amount of time (or retries or any other measuring factor), the thread is really put to sleep. If the same code runs on a system with only a single core, the mutex will not spinlock, though, as, see above, that would not be beneficial.

A hybrid spinlock behaves like a normal spinlock at first, but to avoid wasting too much CPU time, it may have a back-off strategy. It will usually not put the thread to sleep (since you don't want that to happen when using a spinlock), but it may decide to stop the thread (either immediately or after a certain amount of time) and allow another thread to run, thus increasing chances that the spinlock is unlocked (a pure thread switch is usually less expensive than one that involves putting a thread to sleep and waking it up again later on, though not by far).

Summary

If in doubt, use mutexes, they are usually the better choice and most modern systems will allow them to spinlock for a very short amount of time, if this seems beneficial. Using spinlocks can sometimes improve performance, but only under certain conditions and the fact that you are in doubt rather tells me, that you are not working on any project currently where a spinlock might be beneficial. You might consider using your own "lock object", that can either use a spinlock or a mutex internally (e.g. this behavior could be configurable when creating such an object), initially use mutexes everywhere and if you think that using a spinlock somewhere might really help, give it a try and compare the results (e.g. using a profiler), but be sure to test both cases, a single-core and a multi-core system before you jump to conclusions (and possibly different operating systems, if your code will be cross-platform).

4. (**7 puncte**) Dați exemplu de apel de sistem care poate conduce la o schimbare de context între procese. Explicați inclusiv în ce condiții se produce schimbarea de context.

- **Interrupts** - When the CPU is interrupted to return data from a disk read.

schimbare de context la apel de sistem read file

5. (**7 puncte**) Apelurile accept() și recv() au o sintaxă de forma:

accept(sockfd1, **...)**
recv(sockfd2, **...)**

unde sockfd1 și sockfd2 sunt descriptori de socket. Cu ce diferă cei doi socketi?
sockfd1 - socket de tip listen asupra caruia se poate efectua doar operatia de accept deoarece campurile port sursa si ip sursa nu sunt completate
sockfd2 - socket normal cu ajutorul caruia se realizeaza comunicatia. Caracterizat de atributele: ip + port sursa, ip + port destinatie

6. (**10 puncte**) Într-un executabil sunt definite secțiunile text (codul programului), data (variabile globale) și rodata (variabile read-only). Care secțiuni vor fi partajate de două procese pornite separat din acest executabil?

7. (**10 puncte**) Ignorând câmpurile de tip timestamp din cadrul unui inode, dați un exemplu de apel de sistem de lucru cu fișiere (din forma open(), read(), write(), seek(), close(), chmod(), stat() etc.) care modifică un inode și altul care nu modifică un inode.

write() - modifica inode-ul aferent fisierului deoarece se poate modifica dimensiunea sau, mai exact, numarul de block-uri din care este alcatuit fisierul.
seek() - nu modifica inode-ul deoarece cursorul de fisier, ca si entitate, apartine de catre structura ce defineste un fisier deschis si nu de file control block - FCB - inode.

8. (**10 puncte**) Un sistem are suport DEP (*Data Execution Prevention*) dar nu are suport ASLR (*Address Space Layout Randomization*). Precizați cum se face un atac de tipul *return-to-libc*. Cum se obține adresa/adresele necesare?

Address space layout randomization (ASLR) makes this type of attack extremely unlikely to succeed on 64-bit machines as the memory locations of functions are random. For 32-bit systems ASLR provides little benefit since there are only 16 bits available for randomization, and they can be defeated by brute force in a matter of minutes

DEP effectiveness (without ASLR)

In a previous blog post series we went into detail on what DEP is and how it works[part 1, part 2]. In summary, the purpose of DEP is to prevent attackers from being able to execute data as if it were code. This stops an attacker from being able to directly execute code from the stack, heap, and other non-code memory regions. As such, exploitation techniques like heap spraying (of shellcode) or returning into the stack are not immediately possible.

The effectiveness of DEP hinges on the attacker not being able to 1) leverage code that is already executable or 2) make the attacker's data become executable (and thus appear to be code). On platforms without ASLR (that is, versions of Windows prior to Windows Vista), it is often straightforward for an attacker to find and leverage code that exists in modules (DLLs and EXEs) that have been loaded at predictable locations in the address space of a process. Return-oriented programming (ROP) is perhaps the most extensive example of how an attacker can use code from loaded modules in place of (or as a stepping stone to) their shellcode [3,1]. In addition to loaded modules, certain facilities (such as Just-In-Time compilers) can allow an attacker to generate executable code with partially controlled content which enables them to embed shellcode in otherwise legitimate instruction streams ("JIT spraying")[2].

The fact that modules load at predictable addresses without ASLR also makes it possible to turn the attacker's data into executable code. There are a variety of ways in which this can be accomplished, but the basic approach is to use code from loaded modules to invoke system functions like VirtualAlloc or VirtualProtect which can be used to make the attacker's data become executable.

Summary: DEP breaks exploitation techniques that attackers have traditionally relied upon, but DEP without ASLR is not robust enough to prevent arbitrary code execution in most cases.

ASLR effectiveness (without DEP)

Attackers often make assumptions about the address space layout of a process when developing an exploit. For example, attackers will generally assume that a module will be loaded at a predictable address or that readable/writable memory will exist at a specific address on all PCs. ASLR is designed to break these assumptions by making the address space layout of a process unknown to an attacker who does not have local access to the machine. This prevents an attacker from being able to directly and reliably leverage code in loaded modules.

The effectiveness of ASLR hinges on the entirety of the address space layout remaining unknown to the attacker. In some cases memory may be mapped at predictable addresses across PCs despite ASLR. This can happen when DLLs or EXEs load at predictable addresses because they have not opted into ASLR via the /DYNAMICBASE linker flag. Prior to Internet Explorer 8.0 it was also possible for attackers to force certain types of .NET modules to load at a predictable address in the context of the browser[6]. Attackers can also use various address space spraying techniques (such as heap spraying or JIT spraying) to place code or data at a predictable location in the address space.

In cases where the address space is initially unpredictable an attacker can attempt to discover the location of certain memory regions through the use of an *address space information disclosure* or

through *brute forcing*[5]. An address space information disclosure occurs when an attacker is able to coerce an application into leaking one or more address (such as the address of a function inside a DLL). For example, this can occur if an attacker is able to overwrite the NUL terminator of a string and then force the application to read from the string and provide the output back to the attacker [4]. The act of reading from the string will result in adjacent memory being returned up until a NUL terminator is encountered. This is just one example; there are many other forms that address space information disclosures can take.

Brute forcing, on the other hand, can allow an attacker to try their exploit multiple times against all of the possible addresses where useful code or data may exist until they succeed. Brute forcing attacks, while possible in some cases, are traditionally not practical when attacking applications on Windows because an incorrect guess will cause the application to terminate. Applications that may be subjected to brute force attacks (such as Windows services and Internet Explorer) generally employ a restart policy that is designed to prevent the process from automatically restarting after a certain number of crashes have occurred. It is however important to note that there are some circumstances where brute force attacks can be carried out on Windows, such as when targeting an application where the vulnerable code path is contained within a catch-all exception block.

Certain types of vulnerabilities can also make it possible to bypass ASLR using what is referred to as a*partial overwrite*. This technique relies on an attacker being able to overwrite the low order bits of an address (which are not subject to randomization by ASLR) without perturbing the higher order bits (which are randomized by ASLR).

Summary: ASLR breaks an attacker's assumptions about where code and data are located in the address space of a process. ASLR can be bypassed if the attacker can predict, discover, or control the location of certain memory regions (particularly DLL mappings). The absence of DEP can allow an attacker to use heap spraying to place code at a predictable location in the address space.

9. (**10 punct**) Fie secvența de instrucțiuni de mai jos:

```
printf(\%d\n", *a);
printf(\%d\n", *(a+1));
```

unde a este un pointer la un întreg (`int *`). Dați exemplu de situație în care prima instrucțiune **NU** cauzează page fault, dar a doua cauzează page fault.

10. (**10 puncte**) Care este un avantaj, respectiv un dezavantaj al folosirii suportului de *huge pages*? Adică pagini de 2MB (*2 megabytes*) în locul paginilor de 4KB (*4 kilobytes*).
Avantaje:

- Increased performance through increased TLB hits.
- Pages are locked in memory and are never swapped out which guarantees that shared memory like SGA remains in RAM.
- Contiguous pages are preallocated and cannot be used for anything else but for System V shared memory (e.g. SGA)
- Less bookkeeping work for the kernel for that part of virtual memory due to larger page sizes

Dezavantaje:

The amount of wasted memory will increase as a result of internal fragmentation;

extra data dragged around with sparsely-accessed memory can also be costly.

Larger pages take longer to transfer from secondary storage, increasing page fault latency (while decreasing page fault counts).

The time required to simply clear very large pages can create significant kernel latencies.

11. (**15 puncte**) Dorim să implementăm un alocator îmbunătățit de memorie. Alocatorul va expune funcțiile malloc(), calloc(), realloc() și free(), apeluri standard în lucrul cu memoria. În back end va folosi apelurile de sistem de tip mmap() sau brk() expuse de sistemul de operare pentru rezervarea de memorie virtuală. Cerințele alocatorului sunt viteză foarte bună și thread safety.

Ce structuri interne veți folosi în cadrul alocatorului pentru gestiunea alocărilor?

Ce probleme posibile (de viteză/eficiență) pot apărea la nivelul alocatorului?

Cum veți asigura viteză bună de alocare/dezalocare?

Ce fel de aplicații/scenarii de test veți folosi pentru a testa alocatorul?

În conformitate cu ghidul de etică al Departamentului de Calculatoare, declar că nu am copiat și nu voi copia la această lucrare. De asemenea, nu am ajutat și nu voi ajuta pe nimeni să copieze la această lucrare.

Nume și grupă:

Semnătură:.....

Sisteme de Operare

4 septembrie 2014

Timp de lucru: 60 de minute

Notă: Toate răspunsurile trebuie justificate

1. (**7 puncte**) Care este un avantaj al folosirii tabelei de pagini ierarhice față de tabela de pagini simplă?

A mai fost inca o data.

2. (**7 puncte**) De ce un sistem este cu atât mai încărcat cu cât numărul de procese din cozile READY crește (adică mai multe procese în cozile READY înseamnă încărcare mai mare)?

In a real time system, admitting too many processes to the "ready" state may lead to oversaturation and overcontention for the systems resources, leading to an inability to meet process deadlines.

overcontention: **Bus contention**, in computer design, is an undesirable state of the bus in which more than one device on the bus attempts to place values on the bus at the same time.

3. (**7 puncte**) În ce situație o operație de tip lock() pe un mutex blochează thread-ul curent și în ce situație nu îl blochează?

And for default mutexes, attempting to lock a mutex that has been locked by the calling thread leads to undefined behaviour:

If the mutex type is PTHREAD_MUTEX_DEFAULT, attempting to recursively lock the mutex results in undefined behaviour.

lock()

Locks the mutex. If another thread has locked the mutex then this call will block until that thread has unlocked it.

Calling this function multiple times on the same mutex from the same thread is allowed if this mutex is arecursive mutex. If this mutex is a non-recursive mutex, this function will *dead-lock* when the mutex is locked recursively.

4. (**7 puncte**) De ce au dispozitivele de tip bloc nevoie de operații mai rapide (cu throughput mai mare) decât dispozitivele de tip caracter?

5. (**7 puncte**) Cu ce diferă un apel de bibliotecă de un apel de sistem?

System calls are operating system functions, like on UNIX, the `malloc()` function is built on top of the `sbrk()` system call (for resizing process memory space).

Libraries are just application code that's not part of the operating system and will often be available on more than one OS. They're basically the same as function calls within your own program.

The line can be a little blurry but just view system calls as kernel-level functionality.

6. (**10 puncte**) În ce situație se poate rula cod pe stivă, chiar în cazul folosirii unui mecanism de *stack smashing protection* (*canary value*)?

Stack-smashing protection is unable to protect against certain forms of attack. For example, it cannot protect against buffer overflows in the heap. There is no sane way to alter the layout of data within a structure; structures are expected to be the same between modules, especially with shared libraries. Any data in a structure after a buffer is impossible to protect with canaries; thus, programmers must be very careful about how they organize their variables and use their structures.

7. (**10 puncte**) La montarea unui sistem de fișiere se poate folosi opțiunea noatime. Opțiunea înseamnă că **nu** va fi actualizat câmpul atime (timestamp de acces) al unui inode în momentul accesării (citirii sau scrierii inode-ului). De ce este avantajoasă această opțiune în cadrul unui sistem de fișiere încărcat (cu accese dese la fișiere)?

This basically means that the number of writes to a disk for `relatime` mount is close to double relative to a `noatime` mount other thing being equal. It is a serious concern for partitions on flash memory devices.

8. (**10 puncte**) Fie secvența de instrucțiuni de mai jos:

```
*a = 42;        /* first dereferencing */
sleep(5);       /* sleep for 5 seconds */
*a = 42;        /* second dereferencing */
```

unde a este un pointer la un întreg (`int *`). Dați exemplu de situație în care prima dereferențiere **nu** cauzează page fault, dar a doua dereferențiere cauzează page fault.

9. (**10 punct**) Un proces în Linux are, în mod obișnuit, tabela de descriptori de fișiere limitată la 1024 de intrări. De ce în cazul unui server TCP încărcat, care primește multe conexiuni, este nevoie de creșterea acestei limite?

10. (**10 puncte**) De ce este mai probabilă apariția unui *stack overflow* în cazul unui proces multi-threaded față de un proces single-threaded? (*stack overflow* = depășirea limitei stivei în cadrul spațiului de adrese al unui proces)

Folosirea unui număr mare de thread-uri în cadrul unui proces poate conduce mai rapid la stack overflow. Fiecare thread are stiva proprie, cu dimensiunea fixată la crearea. Dacă se creează prea multe thread-uri, se va ocupa foarte mult stiva. Stivele thread-urilor vor fi apropiate unele de altele astfel că, în cazul unui flux de apeluri mare (apeluri recursive, de exemplu), există riscul ca stiva unui thread să suprascrie stiva altui thread.

11. (**15 puncte**) Ne propunem implementarea unui framework de messaging (message queue framework). Cu ajutorul acestui framework, aplicații diferite pot comunica unele cu celelalte. Există patru concepte importante: Publishers (cei care produc mesaje), Consumers (cei care consumă mesaje din cozi), Exchanges (cei care primesc mesajele în cozi de la Publishers și apoi le transmit către Consumers), Queues (cozi de mesaje, create de Consumers și care stochează mesajele). Framework-ul trebuie să asigure scalabilitate și performanță. În general, Consumers, Exchanges și Publishers se găsesc pe sisteme fizic diferite; sunt conectați prin Internet/rețea.

Definiți, la nivel de pseudocod, metodele framework-ului folosite de Publishers și Consumers.

Ce probleme de scalabilitate pot apărea la nivelul framework-ului? (adică de la un nivel în sus se vor resimți probleme de performanță)

Ce soluții de rezolvare a problemelor de scalabilitate există? (atât la nivelul framework-ului, cât și la nivelul infrastrcturii folosite în instalare)

Cum asigurați o performanță ridicată a framework-ului și pentru o utilizare simplă (fără probleme de scalabilitate)?

În conformitate cu ghidul de etică al Departamentului de Calculatoare, declar că nu am copiat și nu voi copia la această lucrare. De asemenea, nu am ajutat și nu voi ajuta pe nimeni să copieze la această lucrare.

Nume și grupă:

Semnătură:.....

Sisteme de Operare

13 septembrie 2014

Timp de lucru: 60 de minute

Notă: Toate răspunsurile trebuie justificate

1. (**7 puncte**) Câte tabele de pagini se găsesc la un moment dat într-un sistem de operare?
Intr-un sistem se gasesc la un moment dat atatea tabele de pagini catre procese exista.

2. (**7 puncte**) De ce overhead-ul unui apel de sistem (oricât de simplu ar fi apelul de sistem) este, în majoritatea cazurilor, semnificativ mai mare decât overhead-ul unui apel de bibliotecă? Presupunem că vorbim de un apel de bibliotecă care nu face în spate apel de sistem

A system call is a call to the kernel for something and acts as an entry point into the operating system. A system call executes in kernel address space and counts as part of the system time. System calls have a high overhead because of the switch to kernel and back, they are specific to each operating system and generally are not portable.

A library call is a call to a routine in a library, such as printf, and is linked with the program. It executes in the user address space that is passed out by the operating system for user programs and has a much lower overhead than a system call. Library calls can be bundled up with a program so that they are portable.

3. (**7 puncte**) În ce situație o operație de tip down() pe un semafor blochează thread-ul curent și i în ce situație nu îl blochează?

If a semaphore has the value 0, a down operation on it will block until someone releases a resource and increments the semaphore.

A non-blocking semaphore does not block on a down operation if the resource is unavailable, but rather yields an error. This can be useful if the program needs that resource immediately or without suspending execution, and if the resource isn't available, the program logic can rather do something else.

4. (**7 puncte**) De ce fiecare thread al unui proces dispune de o stivă proprie?
Thread-ul reprezinta unitatea minimala ce descrie un fir de executie. Fara o stiva nu s-ar putea mentine si memora contextul unui thread atunci cand se face context switching.

5. (**7 puncte**) Un proces CPU intensive este pornit și rulează timp de 30 de minute. La încheierea sa, se observă că acesta a consumat doar 20 de secunde timp de procesor. Cum se explică acest lucru?

6. (**10 puncte**) În general, nu se pot crea hard link-uri la directoare. Cu toate acestea numărul de link-uri aferente unui director diferă între directoare diferite; putem observa acest lucru prin rularea comenzii `stat` pe diverse directoare: `/`, `/home`, `/usr/lib`. De ce diferă numărul de

link-uri între directoare?

Numarul de link-uri al unui director este reprezentat de numarul total de subdirectoare pentru ca fiecare dintre ele au un link catre .. + cele doua intrari default: . si ..
Astfel, putem sa zicem ca numarul de link-uri difera intre directoare deoarece difera numarul de subdirectoare continute.

7. (**10 puncte**) De ce codul dintr-un shellcode se încheie, în general, cu o instrucțiune pentru realizarea unui apel de sistem (de forma `int 0x80`)?

8. (**10 puncte**) Un utilizator rulează, în terminal, **de mai multe ori** cele două comenzi de mai jos (rulăm de mai multe ori ca să fie informațiile citit cache-uite și să nu afecteze rezultatul):

```
find /usr > find.out
find /usr
```

Utilizatorul observă că prima comandă (cu redirectare în fișier) durează semnificativ mai puțin decât a doua (fără redirectare, care afișează pe terminal). De ce?

9. (**10 punct**) Care este avantajul unui server web care folosește mai multe procese pentru servirea cererilor față de unul care folosește mai multe thread-uri?

1. Threads will use up much less resident memory than Processes. Yes, with dynamically linked libraries a lot of memory is shared between the Apache Control Process and it's child Processes, however each new Process will need to instantiate all of the modules you have enabled.
2. This is easily testable by comparing the memory usage of each Process where you have, for example, either 5 Processes and 1 Thread each or 5 Processes and 25 Threads each. In my case here, each child Process takes about 7 MBs regardless of the amount of Threads.
3. +For Threads
4. It takes longer to initiate in terms of time and cpu cycles to load a new Process than it does a Thread. This can be tested by verifying avg amount of pages served via 'ab'.
5. +For Threads
6. A Processes Threads all depend on the Process .. The biggest concern here, is that if something happens to the Process it will affect all the Threads that are associated with it. If you're running with a single Process with a bunch of Threads, then when the Process dies so will the Threads. More Processes would therefore cause a better separation, and thus greater "fault" tolerance if you will.
7. +For Processes
8. Related to (3), for modules such as PHP, their memory is loaded by the Process and shared across all of the Threads. This means that if you have php with memory_limit set to 100Mbs with 25 Threads below, then at max load technically each Thread would be able to allocate a maximum of 4MBs each (course it won't happen this way, some will hog, some will starve).

So in the end, it really depends on your use case .. That being said, you'll want to maximize the amount of Threads used so as to diminish memory usage and increase responsiveness. However, you'll have to balance that with a proper amount of Processes for better fault tolerance.

Course I'm no expert here as I've only recently have had to become concerned with this, so I look forward to see what other answers might pop up here !

10. (**10 puncte**) Un executabil are zona de cod de 1MB. Cu toate acestea, un proces creat din acest executabil ocupă, pe parcursul rulării, maxim 100KB de memorie RAM. Cum explicați?

11. (**15 puncte**) Dorim să implementăm o rețea peer-to-peer. Rețeaua va pune preț mai mult decât orice pe disponibilitatea conținutului (*availability*). Fiecare peer va rezerva un spațiu dat pe hard disk-ul propriu pentru fișiere care nu sunt ale sale dar care ne propunem să fie disponibile.

Perioada de timp cât un peer este activ și lățimea sa de bandă sunt factori în stabilirea nivelului de implicare (*involvement*) al acestui peer. Un peer implicat va putea descărca mai rapid date de la alți peeri; fiecare peer își controlează banda de upload și acordă mai mult peerilor implicați.

Rețeaua este folosită pentru transferuri de fișiere mici (muzică, documentație, mici fișiere video). Transferul se realizează doar între un peer și alt peer.

Care vor fi primitivele protocolului de comunicare între peeri?

Cum ați reține, în cadrul rețelei peer-to-peer, informațile despre implicarea unui peer (involvement), pentru a fi accesate de peeri? Motivați alegerea.

Ce facilități veți folosi pentru o performanță cât mai bună a aplicației specifică unui peer

(pentru transferul fișierelor pe rețea, stocarea fișierelor)?

Ce facilități oferă rețeaua pentru a asigura disponibilitatea conținutului? Adică fiecare fiș ier să fie stocat în cât mai multe locuri.

În conformitate cu ghidul de etică al Departamentului de Calculatoare, de- clar că nu am copiat și nu voi copia la această lucrare. De asemenea, nu am ajutat și nu voi ajuta pe nimeni să copieze la această lucrare.

Nume și grupă: Semnătură:.....

FARA NEGRU

Carina
CristianM
Prj Rosu <3
Remus TODO
Vlaicu
Cosminel
Mares Orange
Tanase
Iuga Turcu'
Anda
Baronescu <!>
Fabi
Trump

Puteti sa adaugati si din alti ani, la final. Oricine doreste acces, sa ceara.

8 iunie 2016

1. Precizati un apel care modifica cursorul de fisier si un apel care modifica dimensiunea unui fisier.
   Modifica dimensiunea: truncate(), write(), open() cu O_TRUNC, close()
   Muta cursorul: lseek, read, write, open - pozitioneaza la inceput sa la sfarsit.

2. Metrica pentru un nivel de interactivitate al proceselor pt un planificator.
   Timpul de asteptare (waiting time) = timpul de asteptare al unui proces in coada READY a planificatorului; acesta trebuie sa fie cat mai scurt pentru a avea un sistem interactiv/responsiv
   Nr. de procese pe secundă
   Cuanta de timp - mica -> interactivitate
3. Fie sectiunea de cod de mai jos:
   char arr[128];
   …
   arr[140] = '\0';
   In ce situatie instructiunea de atribuire rezulta in eroare de acces la memorie (de tip segmentation fault) si in ce situatie nu?

Instructiunea de atribuire rezulta in eroare de acces la memorie (segfault) atunci cand adresa din memorie ce trebuie accesată este într-o pagină la care nu avem drepturi de scriere (sau nu a fost alocată - cuvantul potrivit cred ca e paginata) **si nu rezulta in segfault atunci cand** adresa referită e în aceeaşi pagină cu vectorul alocat, la care avem drepturi (sau într-o altă pagină, eventual cea următoare, care să fie alocată şi la care să avem drepturi).

4. Apelul malloc() aloca memorie, in vreme ce apelul calloc() o si zeroizeaza (umple spatiul alocat cu valori de zero). De ce in cazul apelului calloc() spatiul de memorie fizica/rezidenta (resident set size) a procesului creste, dar in cazul malloc() nu (sau foarte putin)?

Apelul malloc face doar rezervare de pagini în memoria virtuală, fără a face corespondențe cu spaţiu de memorie fizică (care se va întâmpla ulterior, la accesarea efectivă a paginilor virtuale rezervate).

COMPLETARE: ..

Calloc e la la baza malloc() + memset, iar memset implica mai multe apeluri de sistem, deci overhead

5. Pe un sistem de operare dat, stiva unui thread este limitata la 8MB. Care este un avantaj si un dezavantaj al cresterii acestei limite (de exemplu la 32MB)?

Dezavantaj: Poti face mult mai putine threaduri, capra serverasul lui perju (sugi
Avantaj: Faci recursivitate pana vrea neamul lui tanase si ii trece si lui tema la SD

6. (10 puncte) Un sistem pe 64 de biti are suport pentru DEP (Data Execution Prevention) si ASLR (Address Space Layout Randomization). Presupunand ca identificam intr-un program o vulnerabilitate de tip buffer overflow, ce actiuni urmarim pentru a putea obtine un shell?

Daca are DEP, sigur nu se poate injecta cod si apoi sa il executam, prin urmare va trebui sa gasim/cautam locul in care aslr-ul a palasat biblioteca libc pentru a reusi sa apelam /bin/bash prin suprascrierea adresei de return dintr-o finctie catre /bin/bash.

Alternative answer: Putem injecta cod chiar daca are DEP dar doar cod read-only , Dep te impiedica doar sa executi cod read-write. Deci am putea injecta un cod compilat si sa il setam cu mprotect ca readonly iar pentru a scapa de ASLR putem introduce la inceputul acestui cod un block de no-opuri si suprascriind adresa de return de unde se gaseste vulnerabilitatea cu adresa mijlocului blocului de no-opuri din codul nostru aslr-ul o sa dea o adresa de mai sus sau mai jos cel mai probabil din acel bloc de no-oprui si programul nostru va incepe executia.
C: PS putem chiar injecta cod intr-o pagina protejata read only si dupa sa ii dam drepturi cu mprotect

7. (10 puncte) Precizati un scenariu ın care doua fisiere (inode-uri) diferite ajung sa refere acelasi bloc de date.

Se intampla asta daca acele inoduri sunt linkuri simbolice -> pot referi acelasi bloc de date si, simbolic link sunt inoduri de sine statatoare

8. (10 puncte) Un buffer circular este un buffer pentru care operatiile ajunse la sfarsitul sau, continua de la inceput (daca ai ajuns la ultimul element al buffer-ului, continua de la primul element). Buffer-ul are doua capete (doi indecsi): in si out. Un consumator citeste de la out, iar un procucator scrie la in. Precizat, in pseudocod o implementare sincronizata a functiilor consume() si produce(item) folosind un buffer circular.

```
Sem N = 1, M = 0 // cred(99%) ca N e initial dimensiunea bufferului circular, nu 1
in,out
Producator:
While true
        Aux = produce()
        P(N) // acquire
        Buffer[in] = aux
        In = in mod max + 1
        V(M) // release

Consumator:
While true:
        P(M)
        Consuma
        Out = out mod max + 1
        V(N)
```

9. (10 puncte) Pentru un program/executabil putem folosi functionalitatea de setuid pentru actiuni privilegiate. Intrucat aceasta functionalitate este de tip totul sau nimic (obtii ıntregul set de privilegii ale utilizatorului administrativ), se recomanda folosirea de capabilitati. O capabilitate este avantajoasa pentru ca permite doar un anumit tip de actiune privilegiata. De ce nu are sens sa existe o capabilitate pentru actiuni privilegiate cu sistemul de fisiere (adica scrierea si citirea oricarui fisier)?

Un astfel de privilegiu ar permite acces la fisiere precum /etc/shadow, /etc/passwd, iar scrierea si citirea acestor fisiere de catre oricine ar putea avea consecinte nedorite in securitatea sistemului.

10. Load average este o metrica care este proportionala cu numarul de procese aflate in starea READY. De ce load average-ul este mai mare pentru un sistem cu mai multe procese de tip CPU intensive?

Spre deosebire de procesele I/O intensive, care deseori se vor afla în starea WAITING, procesele CPU intensive nu au nevoie de interacţiuni cu utilizatorul şi deci vor solicita mereu accesul la procesor, deci vor fi în READY dacă procesoarele sunt deja ocupate. Astfel, mai multe procese CPU intensive conduc la un load average mai mare.

10 iunie 2016

1. (7 puncte) Fie urmatoarea sectiune de cod:
char a[128];
...
a[900] = '\0';
Intructiunea de atribuire **nu rezulta** in exceptie de memorie (segmentation fault). Acest lucru se ıntampla, insa in momentul folosirii instructiunii a[901] = '\0'.
Cum explicati?

> **Instructiunea de atribuire rezulta in eroare de acces la memorie (segfault) atunci cand** adresa din memorie ce trebuie accesată este într-o pagină la care nu avem drepturi de scriere (sau nu a fost alocată) **si nu rezulta in segfault atunci cand** adresa referită e în aceeaşi pagină cu vectorul alocat, la care avem drepturi (sau într-o altă pagină, eventual cea următoare, care să fie alocată şi la care să avem drepturi).
>
> ____
> Adresa ce trebuie accesata nu se afla intr-o pagina cu drepturi de scriere (sau nu a fost alocata);
> Adresa a[900] se afla chiar la sfarsitul paginii cu drept de write, iar a[901] la inceputul unei pagini fara drept de write.

2. (7 puncte) In ce situatie un apel de tipul write() modifica atat cursorul de fisier cat si dimensiunea fisierului?
write(int *fd***, const void ****buf***, size_t** *count***); -** Write() scrie acolo unde este pozitionat cursorul, nr de octeti trimisi ca argument, iar cursorul ramane acolo unde

a terminat de scris. In cazul in care, argumentul count nu este 0, iar scrierea se face cu succes, atat cursorul cat si dimensiunea se modifica.
Completare: daca cursorul este pozitionat la inceputul fisierului, de exemplu, sau in mijloc si scriem un numar x de caracter x < numarul de caractere ramase pana la sfarsitul fisierului , write va **suprascrie** cele x numar de caractere existente deja in fiser si nu v-a modifica dimensiunea acestuia. Deci write modifica dimensiunea fisierului doar atunci cand cursoru trece de SEEK_ENDUL initial (dinainte de scrierei).

3. (7 puncte) Precizati si justificati o metrica numerica pentru nivelul de productivitate al unui planificator de procese.

Turnaround time cat mai mic. Numar de context switch-uri cat mai mic.

Numarul de context switch-uri într-o unitate de timp. Cu cat sunt mai multe, cu atat nivelul de productivitate este mai mare(?!!!??????????????????!??!!?!??!?!?!?!?!?!?!?!?!?!?)
C:??????^

Cred ca voia sa zica de interactivitate, nu de productivitate
C:agree

4. (7 puncte) De ce operatia de tipul seek() nu are sens pe dispozitive de tip caracter si pe socketi, ci doar pe dispozitive de tip bloc?

Pentru ca pe dispozitive de tip caracter se primesc datele una cate una si nu ai pe ce sa faci seek, in schimb daca ai un bloc de date, ai unde sa faci seek

Pentru că pe dispozitivele de tip caracter datele vin şi sunt citite/ scrise octet cu octet, ca într-o ţeavă. Nu putem anticipa date şi ne putem plasa mai sus sau mai jos pe banda de date. În cazul dispozitivelor de tip bloc însă, datele se găsesc pe un spaţiu de stocare pe care ne putem plimba/glisa; putem "căuta" date prin plasarea pe un sector/bloc al dispozitivului de stocare şi atunci operaţia lseek() are sens. Pe sockeţi se primesc in pachetele datele si nu ai cum sa faci seek la ce se afla in capatul celalalt

5. (7 puncte) De ce este de preferat sa folosim spinlock-uri pentru regiuni critice mici si mutexuri pentru regiuni critice mari?

Spinlock-ul foloseşte busy-waiting şi are operaţii de lock() şi unlock() ieftine prin comparaţie cu mutex-ul. Operaţiilor de lock() şi unlock() pe mutex sunt de obicei costisitoare întrucât pot ajunge să invoce planificatorul. Având operaţii rapide, spinlock-ul este potrivit pe secţiuni critice de mici dimensiuni în care nu se fac operaţii blocante; în aceste cazuri faptul că face busy-waiting nu contează aşa de mult pentru că va intra rapid în regiunea critică. Dacă am folosi un mutex pentru o regiune critică mică, atunci overhead-ul cauzat de operaţiile pe mutex ar fi relativ semnificativ faţă de timpul scurt petrecut în regiunea critică, rezultând în ineficienţa folosirii timpului pe procesor.

Se folosesc spinlock-uri atunci cand codul din regiunea critica se va executa foarte rapid si celelalte threaduri nu vor astepta foarte mult sa ia lock-ul in busy-waiting.

6. (10 puncte) Fisierele executabile contin o sectiune numita PLT (sau function stubs) prin care se intermediaza apelul functiilor de biblioteca. Atacurile de tipul ret-to-plt urmaresc sa foloseasca PLT pentru apelul functiilor de biblioteca. De ce sunt de interes aceste atacuri pe sisteme care folosesc DEP (Data Execution Prevention) si ASLR (Address Space Layout Randomization)?
Sunt de interes deoarece prin intermediul acelei sectiuni putem afla unde a plasat ASLR-ul bibliotecile libc, astfel putem realiza atacuri return-to-libc

ret-to-plt este folosit pentru a trece peste DEP si ASLR. In loc sa faca return la o functie din libc, a carei adresa este randomizata de ASLR, atacatorul face return la un plt a11l functiei, a carui adresa nu este randomizata. Pentru ca function@PLT nu este randomizata, atacatorul nu mai trebuie sa ghiceasca adresa de baza a libc, putand face simplu return la function@PLT pentru a invoca functia function.

7. (10 puncte) Fisierul /etc/shadow contine rezumate hash (functii de tip one-way) ale parolelor. De ce se urmareste ca functiile care calculeaza rezumatul hash al unei parole sa fie costisitoare din punctul de vedere al timpului: o rulare de functie (care calculeaza un hash pentru o parola) sa dureze, sa nu fie obtinut rezultatul instant?
Sa nu fie obtinute parolele prin brute force (dar nu sunt sigura) (e bine, am gasit pe net “bcrypt”)
Altfel ai putea usor implementa un brute force. Iei un string gol, pui oricare caracter (256 posibile) si compari hash-ul obtinut de tine cu cel pt care cauti parola. Si concatenezi caractere in continuare pana ai match. Deci incerci 256^nr_caractere_parola

8. (10 puncte) Thrashing este o problema a unui mecanism de stocare in care componentele acestuia (pagini, intrari) sunt foarte des schimbate, ducand la o performanta scazuta; in loc sa fie folosite, acele componente sunt foarte des schimbate. Cum are loc TLB thrashing?

Cred: ca TLB thrashing are loc cand se realizeaza alocari alternative de blocuri de memorie mai mici cu blocuri de memorie mai mari, de EX: daca in tlb avem mapate pagini pentru date de dimensiune mica(tlb plin) folosite des si avem nevoie de a aceesa un bloc de date mai mare , tlb-ul va mapa multitudinea de pagini asociate blocului mare de date si va scoate paginile asociata blocurilor mici iar cand v-a venii randul de acces al blocurilor mici se vor cauza tlb-misses.

Cred ca e mai simplu de atat: tlb are o dimensiune limitata(desi nowadays e un monstru pe multe nivele). Daca procesul are multe pagini mapate peste ram, atunci se va umple tlb si vei pune info peste inrtarile veche..t

9. (10 puncte) Un kernel preemptiv este unul in care poate intrerupe un proces de pe un procesor atunci cand ii expira cuanta sau este un alt proces prioritar, iar procesul ruleaza in spatiul kernel (kernel space). Un kernel nepreemptiv poate “ıntrerupe procesul” in acele conditii doar daca ruleaza in spatiul utilizator (user space). Un kernel preemptiv este mai util pentru interactivitate sau pentru productivitate?

Interactivitate (procesele nu trebuie sa astepte mult timp pentru a rula pe procesor, se tine cont de o cuanta de timp, de prioritate)

10. (10 puncte) a.txt si b.txt sunt doua link-uri (nume) la acelasi inode. Inode-ul are contorul de link-uri egal cu 2. Pe un sistem dat, dupa operatia mv a.txt /boot/ contorul de link-uri ajunge la 1 pentru inode. Cum explicati?

/boot se afla pe alta partitie .
Poate cineva sa mai dezvolte? E valabil pt cazul in care a este fisierul si b un hard link la fisier si in acest caz b fiind hard link nu poate referi ceva aflat pe alta partitie?
Pana la urma cred ca pe ambele poti sa le interpretezi ca fiind hard link-uri, pentru ca de fapt doar pointeaza catre acelasi inode si nu conteaza, de fapt orice fisier e un hard link catre un inode(chiar daca e doar unul). Mai era o intrebare de genul, de ce un fiser mare se muta pe aceeasi partitie instant iar pe alta partitie dureaza, tocmai pentru ca de fapt pe aceeasi partitie e doar un “hard link” si doar se muta link-ul, inode-ul ramane tot acolo. (E CINEVA CARE CREDE ALTFEL??????????????????????????? POATE GRESESC EU)

16 iunie 2016

1. (7 puncte) Ce tip de apel de sistem nu are loc in cazul folosirii operatorului & (ampersand) in shell (operatorul de rulare in background)?

Apelul wait() nu mai are loc. (parintele nu mai asteapta procesul copil)

2. (7 puncte) O functie intr-un program consuma un procent semnificativ din timpul de rulare si ne propunem sa paralelizam implementarea acesteia. De ce nu are sens folosirea de thread-uri cu implementare user-level?

Threadurile user level nu au suport multi-core, fiind necesara o implementare cu kernel level threads.

3. (7 puncte) Un program are o vulnerabilitate de tip buffer overflow si foloseste canary values (stack smashing protection). Cum putem totusi ataca programul pentru a-i altera fluxul de executie?

Canary, DEP, ASLR
Facem un handler pentru semnalul de stack smashing
Daca reusesti cumva sa gasesti valoarea Canary-ului si in momentul in care suprascrii folosindu-te de buffer overflow sa incepi cu valoarea gasita, am inteles ca a spus ceva de genul la curs, stie cineva sa spuna cum???????

4. (7 puncte) De ce pipe-urile anonime (create cu ajutorul apelului pipe()) pot fi folosite doar intre procese inrudite?

Deoarece procesele copil mostenesc file descriptorii deschisi de procesul parinte prin apelul pipe(). Daca se doreste comunicarea intre procese ce nu sunt inrudite este nevoie named pipes.

5. (7 puncte) De ce numarul de schimbari de context intre procese este mai mare atunci cand se foloseste un planificator de procese cu prioritati (fata de folosirea unui planificator de procese fara prioritati)?

Deoarece procesele IO bound au o prioritate mai mare decat cele CPU bound (foarte posibil sa faca un apel blocant) deci numarul de schimbari de context creste, astfel obtinandu-se interactivitate mare.

6. (10 puncte) memcached este un sistem de caching al obiectelor in memorie folosit in special in conjunctie cu sisteme de baze de date. Pentru o performanta superioara se recomanda folosirea Huge Pages, adica pagini de 2MB in loc de 4KB (pe x86_64). De ce? Alocarile si dezalocarile afecteaza putin performanta, mare parte din alocari facandu-se la inceputul rularii iar dezalocarile sunt rare.
Cautarea in TLB este mult mai rapida. Paginile fiind mai mari e nevoie de mai putine intrari in TLB (ai destula memorie RAM intr-un server).
Cred ca ar mai trebui ceva aici.

7. (10 puncte) Un utilizator porneste Firefox dupa un reboot al sistemului. Dupa un timp inchide procesul Firefox si apoi il reporneste la scurt timp dupa. Observa ca a doua oara procesul a pornit semnificativ mai rapid decat prima oara. Cum explicati?
C: probabil anumite blocuri de date sunt ramase prin nivelele de cache L2/L3 sau ram, si nu mai este nevoie de aducerea lor de pe disk (foarte lenta, mai repede iti vine pizza de la dominos) .
In Windows (nu stiu sigur de Linux) este mecanismul de prefetch care face posibil acest lucru

8. (10 puncte) Un read-write lock este un mecanism de sincronizare care protejeaza o regiune critica tinand cont de tipul thread-urilor: thread-uri de tip reader (care citesc date) sau threaduri de tip writer (care modifica date). Exista doua posibilitati:
i. in regiunea critica se gasesc doar thread-uri de tip reader, oricat de multe;
ii.in regiunea critica se gaseste un singur thread de tip writer (si doar acesta, fara alte thread-uri).
Fat De ce, in general, implementarile de read-write lock sunt de forma write-biased? Adica daca in regiunea critica se gasesc thread-uri reader si sosesc la regiunea critica thread-uri writer si thread-uri reader, thread-urile writer nou sosite vor avea prioritate in fata thread-urilor reader nou sosite.

Daca sosesc la regiunea critica si threaduri writer si reader, prioritate vor avea cele writer deoarece acestea vor actualiza informatia ce va fi citita de threadurile reader. Daca threadurile reader ar avea prioritate acestea ar citi date ce vor fi modificate deci date invalide.

Starvation … Daca prioritizezi threadurlei reader, e posibil ca threadul writer sa nu intre niciodata. (se stie!!! ;))

9. (10 puncte) De ce dimensiunea tabelei FAT nu este afectata de numarul de fisiere aflat pe o partitie formatata FAT32?
Numarul de entries in tabela FAT e stabilit de la ineput. (Nu stiu sigur)

10. (10 puncte) Un atacator identifica o vulnerabilitate intr-un program si o exploateaza obtinand un shell pe un sistem la distanta; shell-ul nu este de utilizator privilegiat. Programul ınsa ruleaza ˆıntr-un chroot jail prevenind atacatorul sa distruga sau sa obtina informat, ii relevante de pe sistem. Cum poate continua atacatorul atacul?

1. Open a file handle to the root of the jail
2. Create a sub directory in the jail and chroot yourself there (you are root, so you are allowed to chroot). You're now even deeper in the jail.
3. Change directory using the file handle to the root of the old jail. You're now outside of the chroot jail you created in the sub directory. You're free! Well, kind of. Actually all locations starting with '/' are still mapped to that sub directory. You don't want this.
4. cd ..; cd ..; …until you reach the real root.
5. Chroot yourself in the real root. Now you're properly free.
6. (DE AICEA AM LUAT)

11. (25 puncte) O firma proiecteaza si implementeaza un sistem de publicare de imagini. Firma doreste sa atraga clienti si publicitate folosind ca diferentiator performanta sistemului sau. Vi se cere sa proiectati si sa implementati o aplicatie/sistem care sa evalueze performanta sistemului de publicare de imagini.

a. Ce metrici veti urmari pentru a evalua performanta sistemului? (5 puncte)
b. Ce scenarii de utilizare veti folosi pentru a masura acele metrici? Cum veti folosi aplicatia voastra pentru testarea sistemului? (6 puncte)
c. Ce particularitati de proiectare si implementare va avea sistemul vostru pentru realizarea scenariilor de mai sus? Cum urmariti testarea limitelor sistemului de publicare de imagini? (7 puncte)
d. Ce veti recomanda firmei proiectante sa modifice (fine tuning) la nivelul sistemului de publicare de imagini pentru a putea determina ce valori sunt adecvate pentru o performanta ridicata? (7 puncte)

17 iunie 2016

1. (7 puncte) Explicati daca si cum poate fi exploatata o vulnerabilitate de tip buffer overflow intr-un program scris in Java.
[Intrebare lucrare curs CA]
De ce nu sunt posibile buffer overflows în Java?
Răspuns: In Java, toate accesele la memorie sunt verificate sa fie “in bounds”. C’est magnifique
Alte opinii?

2. (7 puncte) Ce efect va avea comanda rm a.txt ˆıntr-un sistem de fisiere Unix? (a.txt este un fisier, iar comanda se intoarce cu succes)
C: Se va sterge fisierul de pe disc, se va reduce numarul de inoduri din folderul in care se gasea a.txt (DACA MAI EXISTA UN HARD-LINK CATRE ACELASI INODE???????)

3. (7 puncte) Ce se intampla daca un proces shell foloseste doar apelul exec(), nu si fork(), in rularea unei comenzi?
C: intreaga imagine a procesului curent va fi suprascrisa cu imaginea executabilului de la calea data in exec. (nu stiu sigur ce se intampla cu stiva procesului vechi)

4. (7 puncte) De ce este recomandat ca procesul parinte sa execute un apel wait() sau waitpid() pentru fiecare dintre procesele copil?
C: pentru a afla daca procesul copil s-a terminat cu succes sau nu. Astfel procesul copil nu va ramane zombie si procesul parinte stie daca treaba procesului copil a fost executata bine si daca poate folosi date modificate de acel proces
Cred ca e recomandat si ca sa poata sa elibereze resursele alocate proceselor copil.

5. (7 puncte) Explicati de ce intr-un handler de semnal nu este recomandat sa folosim variabile globale sau functii ne-reentrante.
C: pentru ca un handler de semnal poate fi apelat de mai multe threaduri la un moment-dat, deci trebuie sa fie o functie reentranta (sa se poata fi apelata de mai multe ori in paralel si ea sa scoata acelasi output)

6. (10 puncte) In procesul P1, descriptorul d are offset-ul 0 intr-un fisier cu continutul abc. Procesul P1 executa fork() si rezulta procesul P2. Procesul P2 citeste din d caracterul a. Procesul P1 doarme o secunda dupa fork(), dupa care citeste un octet din d. Ce caracter va citi procesul P1?
C: va citi caracterul b deoarece procesul P1 va avea acelasi cursor de fisier ca si procesul P2 (mostenit de la parinte) si in momentul in care P2 citeste un caracter, cursorul din fisier se va muta si el cu o pozitie)

7. (10 puncte) Explicatii cum este partajat TLB-ul (Translation Lookaside Buffer) intre procesele care ruleaza pe acelasi sistem de calcul.
Fiecare proces are propriul TLB. No sharing. Se face flush la fiecare context switch.

8. (10 puncte) Descriptorul s reprezinta un socket TCP conectat, iar apelul send(s,buf,1000,0) ˆıntoarce valoarea 500. Care e num˘arul minim de octeti pe care i-a primit receptorul pana la ˆıntoarcerea apelului send()?
C: 0 deoarece send se intoarce in momentul cand a pus datele in bufferul de send din kernel si, intoarce numarul de octeti scrisi in acel buffer. Insa este posibil ca mesajul sa nu fie inca trimis cu adevarat catre destinatie sau sa nu fi ajuns inca. In concluzie numarul minim de octeti pe care i-a primit receptorul este 0.

9. (10 puncte) O aplicatie transmitator foloseste urmatorul protocol peste TCP: fiecare mesaj incepe cu 4 octet, i care reprezinta dimensiunea mesajului, urmat, i de mesajul efectiv (maxim 1000 octet, i). Dup˘a fiecare mesaj, se as,teapt˘a primirea unei confirm˘ari de la receptor. Explicat, i problemele care vor ap˘area atunci c ˆand transmit,˘atorul execut˘a dou˘a apeluri send() per mesaj (unul pentru dimensiune s, i altul pentru mesaj) ˆın loc s˘a fac˘a un singur apel send().

10. (10 puncte) ˆIntr-un sistem cu paginare, tabela de pagini ajut˘a la translatarea din adrese virtuale ˆın adrese fizice. Tabela de pagini, ˆıns˘a, este stocat˘a tot ˆın memorie. Cum afl˘a MMU (Memory Management Unit) adresa fizic˘a a tabelei de pagini?
C: Adresa din memoria RAM a tabelei de pagini a procesului curent este dată de un registru dedicat numit generic PTBR (Page Table Base Register). Acest registru este încărcat de sistemul de operare cu valoarea aferentă procesului curent şi este interogat de MMU.

11. (25 puncte) Un programator a implementat un server care primes,te cereri de la client, i cont, inˆand numele unui fisier dorit, s, i transmite fis, ierul folosind sendfile(). Fis, ierele sunt stocate pe un disc de tip SSD atas,at serverului. Serverul foloses,te un pool de thread-uri pentru a procesa aceste cereri. Programatorul crede c˘a un singur parametru va afecta performant,a sistemului: num˘arul T de thread-uri din thread pool. Ajutat, i programatorul s˘a m˘asoare performant,a sistemului s˘au s, i s˘a ˆıl optimizeze.
a. Ce metric˘a este cea mai potrivit˘a pentru m˘asurarea performant,ei server-ului? (5 puncte)

b. Explicat, i cum poate fi mˇasuratˇa aceastˇa metricˇa la server. (5 puncte) Programatorul dores,te sˇa s,tie dacˇa bottleneck-ul ˆıl reprezintˇa SSD-ul, CPU-ul sau placa de ret,ea.
c. Explicat, i ce experimente poate rula pentru a afla acest bottleneck. Ce valoare trebuie sˇa foloseascˇa pentru T dacˇa existˇa N procesoare ˆın sistem? (8 puncte)
d. Argumentat, i de ce implementarea unui cache de fis, iere ˆın memorie nu ar ˆımbunˇatˇat, i performant,a server-ului. (7 puncte)

C: M-am plictisit rauuuu si nu am chef sa invat la PM :(((

30 August 2016

1. (7 puncte) ˆIn urma rulˇarii comenzii stat pe un fis, ier obt, inem cˇa dimensiunea acestuia este 1GB, dar numˇarul de blocuri ocupate este 0. Cum explicat, i?
C: Ce apare acolo la dimensiunea fisierului poate fi modificat printr-o functie (truncate) dar nu ocupa si spatiul real.

2. (7 puncte) Prioritˇat, ile statice nu pot fi schimbate pe parcursul rulˇarii unui proces. Care este problema cu un planificator care foloses,te doar prioritˇat, i statice?
Se poate ajunge la starvation (procesele cu prioritate mica vor sta foarte mult timp in coada de asteptare)

3. (7 puncte) Thrashing-ul este un fenomen negativ al sistemului de memorie virtualˇa ˆın care se fac schimburi continue ˆıntre memoria principalˇa s, i disc (swap in s, i swap out). Descriet, i un scenariu care duce la aparit, ia thrashing-ului.
Thrashing-ul poate fi cauzat de programe sau workload-uri care nu prezinta localitatea referintei (datele accesate frecvent nu se afla in aceeasi parte a memoriei). Astfel, daca intreg setul de date pe care se lucreaza nu poate incapea in memoria fizica, atunci se poate produce swapping constant (thrashing).

4. (7 puncte) De ce un proces intr-un sistem de operare modern nu va dispune niciodata de mai multe pagini fizice decat pagini virtuale?
C: deoarece sistemele moderne sunt majoritatea pe 64 de biti -> un spatiu foarte mare de adrese virtuale, mult mai mare decat ai putea aveea rami.

5. (7 puncte) De ce putem afirma ca paginarea ierarhica este un compromis spatiu-timp (space time tradeoff)?
C: deoarece prin paginarea ierarhica se miscoreaza dimensiunea tabelelor de pagini, insa este ingreunata cautarea. (a doua parte not sure)

6. (10 puncte) De ce este indicat sa folosim capabilitati in loc de setuid pentru executabile care au nevoie de privilegii?

Din motive de securitate. E mai safe folosirea de capabilitati, decat schimbarea userului. Un exemplu e comanda “ping” care trebuie sa execute actiuni privilegiate(deschidere de scoketur) si care e executata folosind capabilitati

7. (10 puncte) De ce unele sisteme pe 32 de bit, si care folosesc memory mapped I/O pot avea maxim 3GB de memorie RAM (numit si 3GB barrier) in loc de maxim 4GB?
C: pentru ca in acel 1 G lipsa sunt mapate functii din kernel pentru a nu fi necesar un context-switch la fiecare apel de sistem

8. (10 puncte) De ce mecanismele de sincronizare sunt mai complexe si mult mai frecvent folosite ˆın codul care ruleaz˘a ˆın kernel mode fat,˘a de codul care ruleaz˘a ˆın user mode?
C: wtf?

9. (10 puncte) ˆIn ce situat, ie apelul send(sockfd, buffer, 1000) ˆıntoarce o valoare cuprins˘a ˆıntre 0 s, i 1000 (excluz ˆand 0 s, i 1000)?
C: 0 nu cred ca are sens, cat despre valorile intre 0 si 1000, apelul send va intoarce numarul de octeti pusi in bufferul send_buffer din kernel, buffer din care urmeaza sa fie trimisi catre destinatie. Datele odata puse acolo sunt considerate ca si trimise (asigura TCP), deci poate sa intoarca oricat. Legat de dimensiunea maxima nu stiu sigur.

10. (10 puncte) Pe un sistem dat un proces poate dispune de un num˘ar maxim de thread-uri. Cum putem cres,te aceast˘a limitare?
Micşorând dimensiunea stivei.

11. (25 puncte) Vi se cere s˘a proiectat, i s, i s˘a implementat, i o aplicat, ie de reverse engineering s, i binary analysis pentru executabile, care s˘a fie capabil˘a at ˆat de analiz˘a static˘a c ˆat s, i de analiz˘a dinamic˘a. Se presupune c˘a nu avet, i acces la codul surs˘a s, i c˘a dorit, i s˘a aflat, i c ˆat mai rapid informat, ii legate de executabil s, i de modul s˘au de funct, ionare. Urm˘arit, i act, iuni precum dezasamblare, construire de graf de flux de control (control flow graph), investigarea apelurilor de sistem, de bibliotec˘a, acoperirea codului (coverage), zone de cod hot s, i zone cold (apelate des, respectiv rar).
a. Propunet, i o arhitectur˘a de principiu a aplicat, iei (ce componente va avea s, i cum se vor conecta ˆıntre ele). (5 puncte)
b. Ce funct, ionalit˘at, i trebuie s˘a ofere sistemul de operare pentru a putea implementa cerint,ele aplicat, iei? (7 puncte)
c. Cum at, i implementa ˆın cadrul aplicat, iei descoperirea zonelor de cod hot s, i cold? (6 puncte)
d. Cum at, i folosi aplicat, ia pentru descoperirea vulnerabilit˘at, ilor de memorie care ar putea conduce la execut, ia de cod arbitrar? (7 puncte)

5 septembrie 2016

1. (7 puncte) Cum putem preveni aparitia de atacuri de tipul fork bomb?
Limitam nr de procese ale unui user (comanda ulimit)

2. (7 puncte) De ce o vulnerabilitate la nivelul nucleului sistemului de operare este mult mai periculoasa decat una la nivelul unei aplicatii ın user space?
C: Deoarece nucleul sistemului de operare ruleaza in acces privilegiat, deci poate executa comenzi ce necesita drept de root. Pe cand o vulnerabilitate a unei aplicatii din user space nu este asa de periculoasa deoarece nu poate executa comenzi ca si root, neruland in mod privilegiat.

3. (7 puncte) Are sens ca un sistem pe 32 de bit, i (cu 4GB spat, iu virtual de adrese pentru un proces) sˇa aibˇa mai mult de 4GB de memorie fizicˇa (RAM)? Justificat, i.
Magistrala de adrese procesor-ram are tot 32 de biti, nu poti adresa mai mult de 4gb de ram. Deci nu are sens sa ai mai mult de 4gb de ram.

4. (7 puncte) De ce, ˆın general, nu are sens operat, ia lseek() pe dispozitive de tip caracter?
C: deoarece lseek poate inainta cursorul de citire/scriere intr-un bloc de date, iar in cazul dispozitivelor de tip caracter, datele sunt primite secvential, nu in blocuri, deci nu am avea pe ce saface seek
16 iunie 2016 - intrebarea 4

5. (7 puncte) De ce este preferat un sistem de fisiere FAT32 in fata unui sistem de fisiere NTFS pentru sisteme de fisiere pentru dispozitive mici (de forma USB flash drive)?
In principal pentru compatibilitatea intre platforme, iar fiind vorba si de dispozitive mici nu e o problema asa de mare limitarea de 4Gb a formatului.

6. (10 puncte) De ce este avantajos ca, in momentul planificarii unei noi entitati, planificatorul sa aleaga un thread din acelasi proces cu al thread-ului care a rulat anterior?
Overhead mai mic de context switch?
Nu se mai face tlb flush, inlocuirea tabelei de pagini.
7. (10 puncte) In ce situatie operatia send() la transmit,ˇator s, i operat, ia recv() la receptor vor fi simultan blocate ˆın cadrul unei conexiuni TCP?
C: pot fi simultam blocate daca pachetele se pierd undeva pe retea, astfel recv va ramane blocat fiindca e blocand, iar send se va bloca deoarece se va umple bufferul de send din kernel. TCP obliga SO sa pastreze cadrele pana se primeste ACK pentru ele.
Receiverul are bufferul de receive gol iar senderul are bufferul de send plin. S-a petrecut ceva pe traseu - la middle box uri

8. (10 puncte) Descriet, i un scenariu prin care douˇa intrˇari din tabela de pagini a unui proces referˇa aceeas, i paginˇa fizicˇa.

C: daca ele refera o biblioteca, acea biblioteca se va gasi intr-o singura pagina fizica(frame), iar cele 2 intrari din tabela de pagini virtuale a unui proces pot referi acea biblioteca (apeluri de functii maybe?)

9. (10 puncte) Folosim spinlock-uri sau mutex-uri pentru realizarea accesului exclusiv la o resurs˘a comun˘a ^ıntre thread-uri cu implementare kernel level. De ce implementarea de spinlockuri poate fi realizat˘a ^ın user space (s, i nu necesit˘a apel de sistem) pe c^and implementarea mutex-urilor are nevoie de suport ^ın kernel space?

C: cred ca deoarece spinlock-urile fac busy waiting si ele incearca acolo sa inttre mereu, pe cand mutexurile realizeaza apelul wait, si asta inseamna ca vor fi trecute din running in coada waiting pana se primeste semnalul ce trebuie sa le deblocheze

10. (10 puncte) Dispozitivul /dev/mem permite accesul din user space la toat˘a memoria fizic˘a a sistemului. Scrierea la al N-lea octet din /dev/mem ^ınseamn˘a scrierea la adresa N de memorie fizic˘a. De ce doar utilizatorul privilegiat (root) are acces la acest dispozitiv?

C: cred ca are voie decat root pt ca din /dev/mem se pot rescrie si chiestii ce tin de kernel/sistem de operare si nu ar trebui sa poata fi scrise/modificate de oricine

/dev/mem is a character device file that is an image of the main memory of the computer. It may be used, for example, to examine (and even patch) the system.

Daca orice tamp ar putea scrie in ram atunci nu si-ar mai avea sens separarea user space-kernel space si tot ce tine de securitatea so ului.

11. (25 puncte) Vi se cere s˘a proiectat, i s, i s˘a implementat, i un remote execution gateway, adic˘a un sistem care preia cereri s, i le execut˘a pe alte sisteme de execut, ie (de tip backend) ^ın medii virtualizate. Gateway-ul primes,te task-uri, care cont, in informat, ii despre ce trebuie rulat (de exemplu, teste pentru teme), s, i o specificat, ie de mas, in˘a virtual˘a. Apoi transmite task-ul s, i comand˘a mas, ina virtual˘a pe unul dintre sistemele de execut, ie (backend). Rezultatele rul˘arii sunt apoi trimise clientului care a comandat task-ul. Serverul trebuie s˘a r˘aspund˘a la un num˘ar mare de cereri.

a. Realizat, i o diagram˘a bloc a componentelor software ale gateway-ului s, i a leg˘aturilor dintre ele. (5 puncte)

b. Stabilit, i cum arat˘a protocolul de comunicare ^ıntre clientul care trimite task-ul s, i gateway. G^andit, i-v˘a la trimiterea task-ului s, i la primirea rezultatului. (7 puncte)

c. Stabilit, i cum arat˘a protocolul de comunicare ^ıntre gateway s, i sistemele de execut, ie. (6 puncte)

d. Precizat, i cum va ar˘ata sistemul de fis, iere la nivelul gateway-ului pentru a ment, ine informat, ii despre task-urile ^ın rulare s, i despre rezultatele obt, inute. Presupunet, i c˘a exist˘a mai mult, i utilizatori s, i trebuie s˘a fie p˘astrat˘a separat, ia sigur˘a ^ıntre submisiile utilizatorilor diferit, i. (7 puncte)

Puteti sa adaugati si din alti ani

11 iunie 2015

1. (7 puncte) Ce cauzeazˇa trecerea din user mode ˆın kernel mode?

Mecanismul de trecere din user mode in kernel mode este declansat de apelurile de sistem, revenirea in user mode facandu-se prin intoarcerea din apelul de sistem.

Separatia kernel mode - user mode este importantăpentru căasigurăun mod privilegiat de execuţie (kernel mode) pentru operatii critice. Un mod privilegiat în care ruleazăsistemul de operare înseamnăcăoperaţiile critice (IPC, lucrul cu I/O, lucrul cu memoria) vor fi validate de sistemul de operare si un proces (aplicatie user space) nu poate face pagube sistemului. Pentru operatii privilegiate va fi necesarătrecerea în kernel mode prin intermediul unui apel de sistem, si astfel invocarea sistemului de operare care actioneazăca un gardian al operaţiilor, garantând securitatea si integritatea sistemului.

2. (7 puncte) Ce cont, ine, ˆın mod uzual, o intrare ˆın tabela de pagini a unui proces?

Mapare între adresa virtuală a unei pagini şi adresa unui frame fizic(RAM). Poate conţine şi bit de prezenţă, dirty bit(de modificare) şi ID de proces.

3. (7 puncte) De ce este, ˆın general, mai rapidˇa schimbarea de context ˆıntre douˇa thread-uri ale aceluias, i proces decˆat schimbarea de context ˆıntre douˇa procese?

Nu se face TLB flush. Dezvolta

Schimbarea de context intre 2 procese presupune inlocuirea tabelei de pagini si flush la TLB => overhead. La schimbarea de context intre 2 threaduri nu este necesar acest lucru deoarece threadurile partajeaza tabela de pagini a procesului si au acelasi spatiu de adresa.

4. (7 puncte) Descrieti secventa de apeluri Linux prin care descriptorul 3 din tabela de descriptori de fisier a unui proces va referi iesirea standard, iar descriptorul 1 va referi fisierul a.txt.

```
dup2(1, 3);
int fd = open("a.txt", O_WRONLY);
dup2(1, fd);   // nu e dup2(fd, 1); ???
```

5. (7 puncte) Ce se ˆıntˆamplˇa ˆın cazul operat, iei up() pe un semafor?

Operatia up incrementeaza valoarea semaforului, daca nu exista nici un proces blocat pe acel semafor. Daca exista, valoarea semaforului ramane zero, dar unul din procesele (de exemplu, la intamplare) blocate vor fi deblocate. Operatiile up si down sunt, evident, atomice (ce este o operatie atomica?)

6. (10 puncte) Pe un sistem de fisiere un inode contine 7 pointeri directi catre blocuri de date si 2 pointeri cu indirectare simpla. Stiind ca un bloc are 4096 de octeti si ca un pointer ocupa 4 octeti, care este dimensiunea maxima a unui fisier pe acest sistem de fisiere?

Logic ar fi să fie 7*4096+2*4 (not sure).

7. (10 puncte) ˆIntr-o zonˇa a unui program existˇa urmˇatoarea secvent,ˇa:

char arr[100]; [...]
/* instructiuni oarecare nu afecteaza array-ul arr */
arr[23] = ’a’;
arr[24] = ’b’;
ˆIn ce situat, ie prima instruct, iune de atribuire NU cauzeazˇa page fault, dar a doua instruct, iune de atribuire cauzeazˇa page fault?

8. (10 puncte) Un programator a scris un program ˆın limbajul C care foloseste apelul recv(s, buf, 2000) iar dimensiunea buf este de 1000 de octeti. Sistemul ın discutie foloseste tehnicile Address Space Layout Randomization (ASLR) si Data Execution Prevention (DEP). Poate fi acest bug exploatat? Daca da, explicati cum.

9. (10 punct) Dat, i exemplu de trei mecanisme ale sistemului de operare care permit restrictionarea daunelor pe care le poate provoca un proces controlat de un atacator.
1.Sandboxing (chroot jail), îl pune într-un spațiu de unde nu poate să iasă și nu poate face damage mult.
2.Nu se permite schimbarea ID-ului(cu setuid).
3....

10. (10 puncte) Explicat, i cum s,tie sistemul de operare cˇarui proces ˆıi este destinat un segment TCP primit. Diferentiat, i ˆıntre segmentele cu bit-ul SYN activat sau dezactivat.

11. (25 puncte) Unui programator i se cere sˇa implementeze un server de imagini care va deservi un numˇar mare de utilizatori. Fiecare utilizator poate ˆıncˇarca sau descˇarca imagini de dimensiune micˇa (<1MB). Sistemul trebuie sˇa poatˇa sust, ine simultan minim 1000 de utilizatori (upload s, i download), iar timpul maxim de upload sau download trebuie sˇa fie 1s per fis, ier. Sistemul trebuie sˇa ret, inˇa pentru fiecare utilizator numˇarul de octet, i incˇarcat, i sau descˇarcat, i, precum s, i numˇarul total de octet, i pentru tot, i utilizatorii.
a. Ajutat, i programatorul sˇa aleagˇa hardware-ul minim necesar pentru acest server. Aleget, i placa de ret,ea de vitezˇa, dimensionat, i memoria RAM, aleget, i numˇarul de core-uri/procese, HDD vs. SSD, etc. Explicat, i toate alegerile facute. (7 puncte)
b. Specificat, i arhitectura serverului: dacˇa vor fi folosite procese sau thread-uri (cˆate?), apeluri blocante sau neblocante, etc. (8 puncte)
c. Pentru arhitectura aleasˇa, descriet, i ˆın pseudocod codul de tratare a unui client s, i modul ˆın care statisticile sunt actualizate. (10 puncte)

15 iunie 2015

1. (7 puncte) De ce procesele I/O intensive primesc, ˆın general, o prioritate mai mare decat procesele CPU intensive?

Pentru ca stau foarte putin in starea running, efectuand o operatie blocanta si trecand in waiting. Astfel nu ocupa mult timp pe procesor, lasand loc altor procese.

2. (7 puncte) De ce consideram apelul chroot() o forma de sandboxing?

Multe sisteme folosesc chroot extensiv în etapa de dezvoltare şi build pentru a detecta problemele generate de dependinţele
dintre diversele module software. Această metodă de build poartă uneori numele de sandboxing.
Comanda chroot poate fi utilizată pentru a crea o copie virtuală a sistemului de operare. Aceasta poate fi utilizată pentru:

Testare şi dezvoltare
Un mediu de test poate fi setat cu chroot, evitându-se astfel rularea testului pe un sistem aflat deja în producţie.
Controlul dependinţelor
chroot este o modalitate foarte bună de control al dependinţelor dintre diferite module software aflate în dezvoltare.
Compatibilitate
Uneori este nevoie să rulăm software mai vechi care necesită versiuni mai vechi ale unor biblioteci. Un mediu chroot este ideal pentru rularea acestui software.
Recuperare
Sisteme care nu mai pot fi pornite de pe hard disc, se pot uneori porni rapid într-un mediu chroot plecând de la un Live CD sau alt mediu de pornire.
Separarea privilegiilor
Programe care în mod potenţial constituie o problemă de securitate se pot rula într-un mediu chroot. Se aplică în general serverelor.

3. (7 puncte) Ce se ˆıntˆampl˘a ˆın cazul operat, iei down() pe un semafor?

Marcheaza semaforul ca este ocupat si astfel nu poate fi accesat de alt proces. Operaţia de down are un overhead ridicat, datorită schimbării de context.

4. (7 puncte) De ce schimbarea de context ˆıntre doua user level threads ale aceluiasi proces este mai rapida decat schimbarea de context ıntre doua kernel level threads ale aceluiasi proces?
Threadurile user level nu necesită schimbarea spaţiului de memorie şi permisiunilor. Ele schimbă doar stackul specific.

5. (7 puncte) Ce contine o intrare ˆın TLB (Translation Lookaside Buffer)?
TLB-ul menţine mapări de pagini fizice şi pagini virtuale. Conţine un subset al tabelei de pagini.(care contine pointeri catre structuri de fisiere deschise)

6. (10 puncte) Pe un sistem cu suport DEP (Data Execution Prevention), un atacator urmareste sa realizeze un apel mprotect(..., PROT_EXEC, ...) pe un set de pagini apartinand stivei. Ce urmareste atacatorul cu un astfel de apel? Cum il va ajuta ın continuarea atacului?
vrea sa obtina drept de executie asupra acelei zone de memorie

7. (10 puncte) Un programator masoara durata celor doua instruct, iuni de mai jos, realizate consecutiv: p = mmap(NULL, PAGE_SIZE, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0); p[0] = 'a'; ˆIn urma masuratorii observa ca prima instructiunie (mmap) dureaza 900 de nanosecunde iar a doua dureaza 3000 de nanosecunde (adica mai mult decat prima). Ce putem spune despre overhead-ul de timp cauzat de un apel de sistem fat,˘a de ovehead-ul de timp cauzat de page fault handler?

8. (10 puncte) ˆIntr-un program se realizeaz˘a apelul: fd = open("/home/student/myfile", O_RDONLY); Apelul reus,es,te. S,tim c˘a /home/student/myfile este un o leg˘atur˘a simbolic˘a (symbolic link). Cˆate inode-uri sunt citite ˆın cadrul apelului open() de mai sus?
Ar trebui să fie citite toate inodeurile începând de la myfile până la inodeul final.

9. (10 puncte) De ce, ˆın general, pornirea unui proces dureaz˘a mai mult la prima sa pornire decˆat la urm˘atoarele porniri?

10. (10 puncte) ˆIn ce situat, ie atˆat apelul send() pe transmit,˘ator, cˆat s, i apelul recv() pe receptor (ˆın aceeas, i conexiune TCP) sunt simultan blocate?

11. (25 puncte) Se propune implementarea unui profiler de aplicat, ii multithreading cu obiectivul de a obt, ine informat, ii legate de profilul de performant,˘a al acestor aplicat, ii.
a. Ce informat, ii/metrici sunt utile de a fi furnizate de profiler dezvoltatorului aplicat, iei? ˆIn ce form˘a sunt utile s˘a fie furnizate acestea dezvoltatorului (numere, medii, grafice, tabele)? (7 puncte)
b. Ce facilit˘at, i trebuie s˘a ofere sistemul de operare s, i hardware-ul pentru funct, ionarea profilerului? (6 puncte)
c. Cum va accesa profilerul facilit˘at, ile oferite de sistemul de operare s, i hardware (apeluri de sistem, memorie partajat˘a, sisteme de fis, iere, dispozitive virtuale etc.)? (5 puncte)
d. Ce facilit˘at, i/metrici vor fi de interes pentru dezvoltatorul unei aplicat, ii de transcoding (conversie video)? Cum va fi folosit profilerul de c˘atre dezvoltator pentru a profila/optimiza aplicat, ia de transcoding? (7 puncte)

1 septembrie 2015

1. (7 puncte) De ce, ˆın general, un prim apel open() reus, it ˆın cadrul unui proces pe un sistem de operare Unix ˆıntoarce valoarea 3?

2. (7 puncte) Care este necesitatea spat, iului de swap ˆıntr-un sistem de operare?

3. (7 puncte) Numit, i un avantaj s, i un dezavantaj al mecanismului de memorie virtual˘a.

4. (7 puncte) Care este un avantaj s, i un dezavantaj al folosirii chroot() ˆın detrimentul virtualiz˘arii pentru izolarea unui grup de procese?

5. (7 puncte) De ce este recomandat s˘a folosim, cˆand se poate, operat, ii atomice pentru sincronizare ˆın loc s˘a folosim alte mecanisme de sincronizare ˆın cadrul unui proces multi-threaded?

6. (10 puncte) Ce limiteaz˘a, ˆın general, num˘arul maxim de thread-uri ce pot fi create ˆın cadrul unui proces? Oferit, i o estimare de formul˘a (nu ceva specific, ci aproximativ) pentru calculul num˘arului maxim de thread-uri ce pot fi create pe un sistem dat.

7. (10 puncte) Un sistem de fis, iere dispune de limit˘ari la num˘arul maxim de fis, iere care pot fi create (inode-uri), num˘arul maxim de nume de fis, iere (dentry-uri), dimensiunea maxim˘a a unui fis, ier s, i spat, iul total ocupat de toate fis, ierele. Un utilizator creeaz˘a ˆıntr-o bucl˘a infinit˘a hard link-uri. Care din limit˘arile de mai sus va cauza oprirea cre˘arii de hard link-uri? Dar ˆın cazul cre˘arii de symbolic link-uri?

8. (10 puncte) Un proces aloc˘a un buffer de dimensiunea unei pagini: char buffer[PAGE_SIZE]; ˆIn ce situat, ie operat, ia de mai jos rezult˘a ˆın primirea unei except, ii de tip Segmentation fault s, i ˆın ce situat, ie nu se ˆıntˆampl˘a acest lucru? buffer[PAGE_SIZE+100] = ’a’;

9. (10 puncte) ˆIntr-un sistem de operare un proces A este planificat des pe procesor dar pentru intervale scurte de timp, ˆın vreme ce un proces B este planificat mai rar, atunci cˆand A nu ruleaz˘a, dar timpul total de rulare pe procesor este mai mare decˆat ˆın cazul procesului A. Ce putet, i spune despre cele dou˘a procese?

10. (10 puncte) Un program are o vulnerabilitate de tip buffer overflow pe un buffer de pe stiv˘a. Sistemul dispune de suport DEP (Data Execution Prevention). Cum poate fi exploatat˘a vulnerabilitatea?

11. (25 puncte) Se propune crearea unei solut, ii de application sandboxing pe un sistem de operare dat. O astfel de solut, ie trebuie s˘a asigure o cˆat mai bun˘a izolare a aplicat, iilor/proceselor sistemului ˆın as,a fel ˆıncˆat s˘a fie diminuate pagubele cauzate de potent, iale vulnerabilit˘at, i de securitate. O astfel de solut, ie trebuie s˘a asigure acces limitat la sistemul de fis, iere, la componente de I/O, la primitive de tip IPC, la ret,ea etc. Solut, ia trebuie s˘a permit˘a definirea ˆın cadrul unor fis, iere a profilelor de sandboxing care apoi s˘a fie folosite pentru sandboxing-ul unei aplicat, ii date. Un profil de sandboxing poate fi folosit de mai multe aplicat, ii.

a. Propunet, i un model de specificare a profilului de sandboxing: ce va cont, ine fis, ierul de profil, cum va fi completat acest fis, ier de profil de sandboxing? (6 puncte)
b. Ce funct, ionalit˘at, i trebuie s˘a ofere nucleul sistemului de operare pentru a putea folosi funct, ionalitatea de sandboxing pe baza profilului? Nucleul este cel care va gestiona aplicarea specificat, iilor din profil. (7 puncte)
c. Cum va fi asociat un profil de sandboxing unui proces dat? Din punctul de vedere al utilizatorului acesta trebuie s˘a porneasc˘a o aplicatie s, i aceasta va folosi automat profilul de sandboxing definit de administrator pentru acea aplicat, ie. (7 puncte)
d. Cum vet, i proteja fis, ierele cu profilele de sandbox? Acestea trebuie s˘a fie editate/accesate doar de administrator. (5 puncte)

10 septembrie 2015

1. (7 puncte) Explicat, i de ce apelul printf nu este un mod robust de a face debugging unui program care primes,te semnalul SIGSEGV.

2. (7 puncte) Dat, i un exemplu in care planificatorul de procese Shortest Job First rezult˘a ˆıntr-o alocare inechitabil˘a a resurselor procesorului.

3. (7 puncte) Numit, i un avantaj s, i un dezavantaj al folosirii memoriei virtuale.

4. (7 puncte) Un atacator descoper˘a o vulnerabilitate ˆıntr-un program care-i permite s˘a execute cod arbitrar. De ce sunt interesante pentru atacator apelurile read s, i write?

5. (7 puncte) Dat, i dou˘a avantaje ale implement˘arii unui server de web cu mai multe procese fat,˘a de mai multe thread-uri.

6. (10 puncte) Explicat, i de ce este necesar˘a starea zombie pentru un proces ˆın sistemele Unix.

7. (10 puncte) Un sistem Unix are spat, iu de adrese pe 32 bit, i s, i spatiu de swap de 40GB. Stiva unui thread are minim 4KB pe acest sistem. a. Estimat, i num˘arul maxim de thread-uri ce pot fi create de un proces pe acest sistem. b. Estimat, i num˘arul maxim de procese ce pot fi create pe acest sistem, presupunˆand c˘a nu exist˘a limit˘ari la num˘arul de descriptori sau identificatori de proces.

8. (10 puncte) Un proces este blocat in apelul de sistem recvmsg. Un pachet este primit la placa de ret,ea. Descriet, i secvent,a de pas, i pe care ˆıi execut˘a placa de ret,ea s, i sistemul de operare pˆan˘a cˆand apelul recvmsg se va ˆıntoarce s, i aplicat, ia ˆıs, i va continua execut, ia.

9. (10 puncte) Explicat, i cum s,tie sistemul de operare c˘arui proces ˆıi este destinat un segment TCP primit. Diferentiat, i ˆıntre segmentele cu bit-ul de SYN activat sau dezactivat.

10. (10 puncte) Un programator foloses,te apelul de sistem write pentru a actualiza un fis, ier. Dup˘a o pan˘a de curent, programatorul descoper˘a ca unele informat, ii au fost scrise cu write dar nu apar ˆın fis, ier. Explicat, i cauza problemelor s, i dat, i o posibil˘a solut, ie.

11. (25 puncte) Vi se cere s˘a implementat, i dou˘a programe, un sender s, i un receiver, care sunt capabile s˘a transmit˘a date ˆıntre dou˘a calculatoare folosind dou˘a interfet,e de ret,ea de 10Gbps. Datele de transmis sunt ˆın memoria sender-ului, s, i se presupune c˘a sunt infinite (e.g. dac˘a se ajunge la sf ˆars, itului bufferului ˆın care sunt stocate se va transmite de la ˆınceputul bufferului). Aplicat, ia receiver are nevoie s˘a stocheze datele ˆın memorie ˆın aceeas, i ordine ˆın care au fost citite de aplicat, ia receiver. Vi se cere s˘a folosit, i API-ul de sockets s, i protocolul TCP pentru a utiliza cele dou˘a interfet,e disponibile la maxim (throughput total 20Gbps):
a. Detaliat, i protocolul pe care ˆıl va folosi aplicat, ia pentru a impr˘as,tia datele pe cele dou˘a c˘ai s, i a le pune ˆın ordine la receiver. (7 puncte)
b. Descriet, i apelurile de sistem necesare pentru stabilirea conexiunii pe mai multe interfet,e. (3 puncte)
c. Descriet, i in pseudocod implementarea pentru sender, atunci cˆand conexiunea este deja deschis˘a. Avet, i ˆın vedere folosirea corect˘a a operat, iei send. (5 puncte)
d. Descriet, i in pseudocod implementarea pentru receiver, atunci cˆand conexiunea este deja deschis˘a. Avet, i ˆın vedere folosirea corect˘a a operat, iei receive. (5 puncte)
e. Analizat, i solut, ia propus˘a, discutˆand minim 2 posibile probleme de performant,˘a care pot ap˘area ˆın practic˘a, s, i oferind solut, ii de remediere. (5 puncte)